CHAPTER

TWENTYEIGHT

NETWORK PROTOCOLS — TCP, IP, IPSEC, FTP, TLS AND CONFIGURATIONS

28.1 EFI TCPv4 Protocol

This section defines the EFI TCPv4 (Transmission Control Protocol version 4) Protocol.

28.1.1 TCP4 Service Binding Protocol

28.1.2 EFI_TCP4_SERVICE_BINDING_PROTOCOL

Summary

The EFI TCPv4 Service Binding Protocol is used to locate EFI TCPv4 Protocol drivers to create and destroy child of the driver to communicate with other host using TCP protocol.

GUID

```
#define EFI_TCP4_SERVICE_BINDING_PROTOCOL_GUID \
  {0x00720665,0x67EB,0x4a99,\
    {0xBA,0xF7,0xD3,0xC3,0x3A,0x1C,0x7C,0xC9}}
```

Description

A network application that requires TCPv4 I/O services can call one of the protocol handler services, such as *BS->LocateHandleBuffer(),* to search devices that publish an EFI TCPv4 Service Binding Protocol GUID. Such device supports the EFI TCPv4 Protocol and may be available for use.

After a successful call to the *EFI_TCP4_SERVICE_BINDING_PROTOCOL .CreateChild()* function, the newly created child EFI TCPv4 Protocol driver is in an un-configured state; it is not ready to do any operation except *Poll()* send and receive data packets until configured as the purpose of the user and perhaps some other indispensable function belonged to TCPv4 Protocol driver is called properly.

Every successful call to the *EFI_TCP4_SERVICE_BINDING_PROTOCOL .CreateChild()* function must be matched with a call to the *EFI_TCP4_SERVICE_BINDING_PROTOCOL.DestroyChild()* function to release the protocol driver.

28.1.3 TCP4 Protocol

28.1.4 EFI_TCP4_PROTOCOL

Summary

The EFI TCPv4 Protocol provides services to send and receive data stream.

GUID

```
#define EFI_TCP4_PROTOCOL_GUID \
  {0x65530BC7,0xA359,0x410f,\
    {0xB0,0x10,0x5A,0xAD,0xC7,0xEC,0x2B,0x62}}
```

Protocol Interface Structure

```
typedef struct _EFI_TCP4_PROTOCOL {
  EFI_TCP4_GET_MODE_DATA         GetModeData;
  EFI_TCP4_CONFIGURE             Configure;
  EFI_TCP4_ROUTES                Routes;
  EFI_TCP4_CONNECT               Connect;
  EFI_TCP4_ACCEPT                Accept;
  EFI_TCP4_TRANSMIT              Transmit;
  EFI_TCP4_RECEIVE               Receive;
  EFI_TCP4_CLOSE                 Close;
  EFI_TCP4_CANCEL                Cancel;
  EFI_TCP4_POLL                  Poll;
}   EFI_TCP4_PROTOCOL;
```

Parameters

GetModeData
Get the current operational status. See the *GetModeData()* function description.

Configure
Initialize, change, or brutally reset operational settings of the EFI TCPv4 Protocol. See the *Configure()* function description.

Routes
Add or delete routing entries for this TCP4 instance. See the *Routes()* function description.

Connect
Initiate the TCP three-way handshake to connect to the remote peer configured in this TCP instance. The function is a nonblocking operation. See the *Connect()* function description.

Accept
Listen for incoming TCP connection request. This function is a nonblocking operation. See the *Accept()* function description.

Transmit
Queue outgoing data to the transmit queue. This function is a nonblocking operation. See the *Transmit()* function description.

Receive
Queue a receiving request token to the receive queue. This function is a nonblocking operation. See the *Receive()* function description.

Close
Gracefully disconnecting a TCP connection follow RFC 793 or reset a TCP connection. This function is a nonblocking operation. See the *Close()* function description.

Cancel
Abort a pending connect, listen, transmit or receive request. See the *Cancel()* function description.

Poll
Poll to receive incoming data and transmit outgoing TCP segments. See the *Poll()* function description.

Description

The *EFI_TCP4_PROTOCOL* defines the EFI TCPv4 Protocol child to be used by any network drivers or applications to send or receive data stream. It can either listen on a specified port as a service or actively connected to remote peer as a client. Each instance has its own independent settings, such as the routing table.

Note: *In this document, all IPv4 addresses and incoming/outgoing packets are stored in network byte order. All other parameters in the functions and data structures that are defined in this document are stored in host byte order unless explicitly specified.*

28.1.5 EFI_TCP4_PROTOCOL.GetModeData()

Summary

Get the current operational status.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP4_GET_MODE_DATA) (
  IN EFI_TCP4_PROTOCOL                 *This,
  OUT EFI_TCP4_CONNECTION_STATE        *Tcp4State OPTIONAL,
  OUT EFI_TCP4_CONFIG_DATA             *Tcp4ConfigData OPTIONAL,
  OUT EFI_IPv4_MODE_DATA               *Ip4ModeData OPTIONAL,
  OUT EFI_MANAGED_NETWORK_CONFIG_DATA  *MnpConfigData OPTIONAL,
  OUT EFI_SIMPLE_NETWORK_MODE          *SnpModeData OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_TCP4_PROTOCOL* instance.

Tcp4State
Pointer to the buffer to receive the current TCP state. Type *EFI_TCP4_CONNECTION_STATE* is defined in "Related Definitions" below.

Tcp4ConfigData
Pointer to the buffer to receive the current TCP configuration. Type *EFI_TCP4_CONFIG_DATA* is defined in "Related Definitions" below.

Ip4ModeData
Pointer to the buffer to receive the current IPv4 configuration data used by the TCPv4 instance. Type *EFI_IP4_MODE_DATA* is defined in *EFI_IP4_PROTOCOL .GetModeData()*.

MnpConfigData
Pointer to the buffer to receive the current MNP configuration data used indirectly by

the TCPv4 instance. Type *EFI_MANAGED_NETWORK_CONFIG_DATA* is defined in *EFI_MANAGED_NETWORK_PROTOCOL.GetModeData()*.

SnpModeData
Pointer to the buffer to receive the current SNP configuration data used indirectly by the TCPv4 instance. Type *EFI_SIMPLE_NETWORK_MODE* is defined in the *EFI_SIMPLE_NETWORK_PROTOCOL*.

Description

The *GetModeData()* function copies the current operational settings of this EFI TCPv4 Protocol instance into user-supplied buffers. This function can also be used to retrieve the operational setting of underlying drivers such as IPv4, MNP, or SNP.

Related Definition

```
typedef struct {
  BOOLEAN               UseDefaultAddress;
  EFI_IPv4_ADDRESS      StationAddress;
  EFI_IPv4_ADDRESS      SubnetMask;
  UINT16                StationPort;
  EFI_IPv4_ADDRESS      RemoteAddress;
  UINT16                RemotePort;
  BOOLEAN               ActiveFlag;
}   EFI_TCP4_ACCESS_POINT;
```

UseDefaultAddress
Set to **TRUE** to use the default IP address and default routing table. If the default IP address is not available yet, then the underlying EFI IPv4 Protocol driver will use *EFI_IP4_CONFIG2_PROTOCOL* to retrieve the IP address and subnet information.

StationAddress
The local IP address assigned to this EFI TCPv4 Protocol instance. The EFI TCPv4 and EFI IPv4 Protocol drivers will only deliver incoming packets whose destination addresses exactly match the IP address. Not used when *UseDefaultAddress* is **TRUE**.

SubnetMask
The subnet mask associated with the station address. Not used when *UseDefaultAddress* is *TRUE*.

StationPort
The local port number to which this EFI TCPv4 Protocol instance is bound. If the instance doesn't care the local port number, set *StationPort* to zero to use an ephemeral port.

RemoteAddress
The remote IP address to which this EFI TCPv4 Protocol instance is connected. If *ActiveFlag* is **FALSE** (i.e., a passive TCPv4 instance), the instance only accepts connections from the *RemoteAddress*. If *ActiveFlag* is **TRUE** the instance is connected to the *RemoteAddress,* i.e., outgoing segments will be sent to this address and only segments from this address will be delivered to the application. When *ActiveFlag* is **FALSE** it can be set to zero and means that incoming connection request from any address will be accepted.

RemotePort
The remote port to which this EFI TCPv4 Protocol instance is connects or connection request from which is accepted by this EFI TCPv4 Protocol instance. If *ActiveFlag* is **FALSE** it can be zero and means that incoming connection request from any port will be accepted. Its value can not be zero when *ActiveFlag* is **TRUE**.

ActiveFlag
Set it to **TRUE** to initiate an active open. Set it to **FALSE** to initiate a passive open to act as a server.

```
typedef struct {
  UINT32             ReceiveBufferSize;
```

(continues on next page)

(continued from previous page)

```
  UINT32             SendBufferSize;
  UINT32             MaxSynBackLog;
  UINT32             ConnectionTimeout;
  UINT32             DataRetries;
  UINT32             FinTimeout;
  UINT32             TimeWaitTimeout;
  UINT32             KeepAliveProbes;
  UINT32             KeepAliveTime;
  UINT32             KeepAliveInterval;
  BOOLEAN            EnableNagle;
  BOOLEAN            EnableTimeStamp;
  BOOLEAN            EnableWindowScaling;
  BOOLEAN            EnableSelectiveAck;
  BOOLEAN            EnablePathMtuDiscovery;
}   EFI_TCP4_OPTION;
```

ReceiveBufferSize
The size of the TCP receive buffer.

SendBufferSize
The size of the TCP send buffer.

MaxSynBackLog
The length of incoming connect request queue for a passive instance. When set to zero, the value is implementation specific.

ConnectionTimeout
The maximum seconds a TCP instance will wait for before a TCP connection established. When set to zero, the value is implementation specific.

DataRetries
The number of times TCP will attempt to retransmit a packet on an established connection. When set to zero, the value is implementation specific.

FinTimeout
How many seconds to wait in the FIN_WAIT_2 states for a final FIN flag before the TCP instance is closed. This timeout is in effective only if the application has called *Close()* to disconnect the connection completely. It is also called FIN_WAIT_2 timer in other implementations. When set to zero, it should be disabled because the FIN_WAIT_2 timer itself is against the standard.

TimeWaitTimeout
How many seconds to wait in TIME_WAIT state before the TCP instance is closed. The timer is disabled completely to provide a method to close the TCP connection quickly if it is set to zero. It is against the related RFC documents.

KeepAliveProbes
The maximum number of TCP keep-alive probes to send before giving up and resetting the connection if no response from the other end. Set to zero to disable keep-alive probe.

KeepAliveTime
The number of seconds a connection needs to be idle before TCP sends out periodical keep-alive probes. When set to zero, the value is implementation specific. It should be ignored if keep-alive probe is disabled.

KeepAliveInterval
The number of seconds between TCP keep-alive probes after the periodical keep-alive probe if no response. When set to zero, the value is implementation specific. It should be ignored if keep-alive probe is disabled.

EnableNagle
Set it to **TRUE** to enable the Nagle algorithm as defined in RFC896. Set it to **FALSE** to disable it.

EnableTimeStamp
Set it to **TRUE** to enable TCP timestamps option as defined in RFC7323. Set to **FALSE** to disable it.

EnableWindowScaling
Set it to **TRUE** to enable TCP window scale option as defined in RFC7323. Set it to **FALSE** to disable it.

EnableSelectiveAck
Set it to **TRUE** to enable selective acknowledge mechanism described in RFC 2018. Set it to **FALSE** to disable it. Implementation that supports SACK can optionally support DSAK as defined in RFC 2883.

EnablePathMtudiscovery
Set it to **TRUE** to enable path MTU discovery as defined in RFC 1191. Set to **FALSE** to disable it.

Option setting with digital value will be modified by driver if it is set out of the implementation specific range and an implementation specific default value will be set accordingly.

```
//*******************************************************
// EFI_TCP4_CONFIG_DATA
//*******************************************************
typedef struct {
  // Receiving Filters
  // I/O parameters
  UINT8                     TypeOfService;
  UINT8                     TimeToLive;

  // Access Point
  EFI_TCP4_ACCESS_POINT     AccessPoint;

  // TCP Control Options
  EFI_TCP4_OPTION           ControlOption;

}   EFI_TCP4_CONFIG_DATA;
```

TypeOfService
TypeOfService field in transmitted IPv4 packets.

TimeToLive
TimeToLive field in transmitted IPv4 packets.

AccessPoint
Used to specify TCP communication end settings for a TCP instance.

ControlOption
Used to configure the advance TCP option for a connection. If set to *NULL,* implementation specific options for TCP connection will be used.

```
//*************************************************************
// EFI_TCP4_CONNECTION_STATE
//*************************************************************

   typedef enum {
     Tcp4StateClosed = 0,
     Tcp4StateListen = 1,
     Tcp4StateSynSent = 2,
     Tcp4StateSynReceived = 3,
```

(continues on next page)

(continued from previous page)

```
    Tcp4StateEstablished = 4,
    Tcp4StateFinWait1 = 5,
    Tcp4StateFinWait2 = 6,
    Tcp4StateClosing = 7,
    Tcp4StateTimeWait = 8,
    Tcp4StateCloseWait = 9,
    Tcp4StateLastAck = 10
  }   EFI_TCP4_CONNECTION_STATE;
```

Status Codes Returned

EFI_SUCCESS	The mode data was read.
EFI_NOT_STARTED	No configuration data is available because this instance hasn't been started.
EFI_INVALID_PARAMETER	*This* is **NULL**.

28.1.6 EFI_TCP4_PROTOCOL.Configure()

Summary

Initialize or brutally reset the operational parameters for this EFI TCPv4 instance.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP4_CONFIGURE) (
  IN EFI_TCP4_PROTOCOL           *This,
  IN EFI_TCP4_CONFIG_DATA        *TcpConfigData OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_TCP4_PROTOCOL* instance.

TcpConfigData
Pointer to the configure data to configure the instance.

Description

The *Configure()* function does the following:

- Initialize this EFI TCPv4 instance, i.e., initialize the communication end setting, specify active open or passive open for an instance.
- Reset this TCPv4 instance brutally, i.e., cancel all pending asynchronous tokens, flush transmission and receiving buffer directly without informing the communication peer.

No other TCPv4 Protocol operation can be executed by this instance until it is configured properly. For an active TCP4 instance, after a proper configuration it may call *Connect()* to initiates the three-way handshake. For a passive TCP4 instance, its state will transit to Tcp4StateListen after configuration, and *Accept()* may be called to listen the incoming TCP connection request. If *TcpConfigData* is set to *NULL,* the instance is reset. Resetting process will be done brutally, the state machine will be set to Tcp4StateClosed directly, the receive queue and transmit queue will be flushed, and no traffic is allowed through this instance.

Status Codes Returned

EFI_SUCCESS	The operational settings are set, changed, or reset successfully.
EFI_NO_MAPPING	When using a default address, configuration (through DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_INVALID_PARAMETER	One or more following conditions are TRUE: • This is NULL. • TcpConfigData -> AccessPoint.StationAddress isn't a valid unicast IPv4 address when TcpConfigData -> AccessPoint.UseDefaultAddress is FALSE. • TcpConfigData -> AccessPoint.SubnetMask isn't a valid IPv4 address mask when TcpConfigData -> AccessPoint.UseDefaultAddress is FALSE. The subnet mask must be contiguous. • TcpConfigData -> AccessPoint.RemoteAddress isn't a valid unicast IPv4 address. • TcpConfigData -> AccessPoint.RemoteAddress is zero or TcpConfigData -> AccessPoint.RemotePort is zero when TcpConfigData -> AccessPoint.ActiveFlag is TRUE. • A same access point has been configured in other TCP instance properly.
EFI_ACCESS_DENIED	Configuring TCP instance when it is configured without calling *Configure()* with **NULL** to reset it.
EFI_DEVICE_ERROR	An unexpected network or system error occurred.
EFI_UNSUPPORTED	One or more of the control options are not supported in the implementation.
EFI_OUT_OF_RESOURCES	Could not allocate enough system resources when executing *Configure()*.

28.1.7 EFI_TCP4_PROTOCOL.Routes()

Summary

Add or delete routing entries.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP4_ROUTES) (
  IN EFI_TCP4_PROTOCOL  *This,
  IN BOOLEAN            DeleteRoute,
  IN EFI_IPv4_ADDRESS   *SubnetAddress,
  IN EFI_IPv4_ADDRESS   *SubnetMask,
  IN EFI_IPv4_ADDRESS   *GatewayAddress
  );
```

Parameters

This
Pointer to the *EFI_TCP4_PROTOCOL* instance.

DeleteRoute
Set it to **TRUE** to delete this route from the routing table. Set it to **FALSE** to add this route to the routing table. *DestinationAddress* and *SubnetMask* are used as the keywords to search route entry.

SubnetAddress
The destination network.

SubnetMask
The subnet mask of the destination network.

GatewayAddress
The gateway address for this route. It must be on the same subnet with the station address unless a direct route is specified.

Description

The *Routes()* function adds or deletes a route from the instance's routing table.

The most specific route is selected by comparing the *SubnetAddress* with the destination IP address's arithmetical *AND* to the *SubnetMask*.

The default route is added with both *SubnetAddress* and *SubnetMask* set to 0.0.0.0. The default route matches all destination IP addresses if there is no more specific route.

Direct route is added with *GatewayAddress* set to 0.0.0.0. Packets are sent to the destination host if its address can be found in the Address Resolution Protocol (ARP) cache or it is on the local subnet. If the instance is configured to use default address, a direct route to the local network will be added automatically.

Each TCP instance has its own independent routing table. Instance that uses the default IP address will have a copy of the *EFI_IP4_CONFIG2_PROTOCOL* 's routing table. The copy will be updated automatically whenever the IP driver reconfigures its instance. As a result, the previous modification to the instance's local copy will be lost.

The priority of checking the route table is specific with IP implementation and every IP implementation must comply with RFC 1122.

Note: *There is no way to set up routes to other network interface cards (NICs) because each NIC has its own independent network stack that shares information only through* EFI TCP4 variable.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_NOT_STARTED	The EFI TCPv4 Protocol instance has not been configured.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *This* is **NULL**. • *SubnetAddress* is **NULL**. • *SubnetMask* is **NULL**. • *GatewayAddress* is **NULL**. • *SubnetAddress* is not **NULL** a valid subnet address. • *SubnetMask* is not a valid subnet mask. • *GatewayAddress* is not a valid unicast IP address or it is not in the same subnet.
EFI_OUT_OF_RESOURCES	Could not allocate enough resources to add the entry to the routing table.
EFI_NOT_FOUND	This route is not in the routing table.

continues on next page

Table 28.3 – continued from previous page

EFI_ACCESS_DENIED	The route is already defined in the routing table.
EFI_UNSUPPORTED	The TCP driver does not support this operation.

28.1.8 EFI_TCP4_PROTOCOL.Connect()

Summary

Initiate a nonblocking TCP connection request for an active TCP instance.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP4_CONNECT) (
  IN EFI_TCP4_PROTOCOL           *This,
  IN EFI_TCP4_CONNECTION_TOKEN   *ConnectionToken,
  );
```

Parameters

This
Pointer to the *EFI_TCP4_PROTOCOL* instance.

ConnectionToken
Pointer to the connection token to return when the TCP three way handshake finishes. Type *EFI_TCP4_CONNECTION_TOKEN* is defined in "Related Definition" below.

Description

The *Connect()* function will initiate an active open to the remote peer configured in current TCP instance if it is configured active. If the connection succeeds or fails due to any error, the *ConnectionToken->CompletionToken.Event* will be signaled and *ConnectionToken->CompletionToken.Status* will be updated accordingly. This function can only be called for the TCP instance in *Tcp4StateClosed* state. The instance will transfer into *Tcp4StateSynSent* if the function returns *EFI_SUCCESS*. If TCP three way handshake succeeds, its state will become *Tcp4StateEstablished,* otherwise, the state will return to *Tcp4StateClosed*.

Related Definition

```
//*******************************************************
// EFI_TCP4_COMPLETION_TOKEN
//*******************************************************
typedef struct {
  EFI_EVENT                Event;
  EFI_STATUS               Status;
}   EFI_TCP4_COMPLETION_TOKEN;
```

Event
The *Event* to signal after request is finished and *Status* field is updated by the EFI TCPv4 Protocol driver. The type of *Event* must be *EVT_NOTIFY_SIGNAL,* and its Task Priority Level (TPL) must be lower than or equal to *TPL_CALLBACK*.

Status
The variable to receive the result of the completed operation. *EFI_NO_MEDIA*. There was a media error

The EFI_TCP4_COMPLETION_TOKEN is used as a common header for various asynchronous tokens.

```
//*******************************************************
// EFI_TCP4_CONNECTION_TOKEN
//*******************************************************
typedef struct {
  EFI_TCP4_COMPLETION_TOKEN      CompletionToken;
}   EFI_TCP4_CONNECTION_TOKEN;
```

Status

The *Status* in the *CompletionToken* will be set to one of the following values if the active open succeeds or an unexpected error happens:

EFI_SUCCESS. The active open succeeds and the instance is in *Tcp4StateEstablished*.

EFI_CONNECTION_RESET. The connect fails because the connection is reset either by instance itself or communication peer.

EFI_CONNECTION_REFUSED: The connect fails because this connection is initiated with an active open and the connection is refused.

EFI_ABORTED. The active open was aborted.

EFI_TIMEOUT. The connection establishment timer expired and no more specific information is available.

EFI_NETWORK_UNREACHABLE. The active open fails because an ICMP network unreachable error is received.

EFI_HOST_UNREACHABLE. The active open fails because an ICMP host unreachable error is received.

EFI_PROTOCOL_UNREACHABLE. The active open fails because an ICMP protocol unreachable error is received.

EFI_PORT_UNREACHABLE. The connection establishment timer times out and an ICMP port unreachable error is received.

EFI_ICMP_ERROR. The connection establishment timer timeout and some other ICMP error is received.

EFI_DEVICE_ERROR. An unexpected system or network error occurred.

Status Codes Returned

EFI_SUCCESS	The connection request is successfully initiated and the state of this TCPv4 instance has been changed to *Tcp4StateSynSent*.
EFI_NOT_STARTED	This EFI TCPv4 Protocol instance has not been configured.
EFI_ACCESS_DENIED	One or more of the following conditions are **TRUE:** • This instance is not configured as an active one. • This instance is not in *Tcp4StateClosed* state.
EFI_INVALID_PARAMETER	One or more of the following are **TRUE**: • *This* is **NULL**. • *ConnectionToken* is **NULL**. • *ConnectionToken* -> *CompletionToken. Event* is **NULL**.
EFI_OUT_OF_RESOURCES	The driver can't allocate enough resource to initiate the active open.
EFI_DEVICE_ERROR	An unexpected system or network error occurred.

28.1.9 EFI_TCP4_PROTOCOL.Accept()

Summary

Listen on the passive instance to accept an incoming connection request. This is a nonblocking operation.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP4_ACCEPT) (
  IN EFI_TCP4_PROTOCOL        *This,
  IN EFI_TCP4_LISTEN_TOKEN    *ListenToken
  );
```

Parameters

This
Pointer to the *EFI_TCP4_PROTOCOL* instance.

ListenToken
Pointer to the listen token to return when operation finishes. Type *EFI_TCP4_LISTEN_TOKEN* is defined in "Related Definition" below.

Related Definition

```
//*******************************************************
// EFI_TCP4_LISTEN_TOKEN
//*******************************************************
typedef struct {
  EFI_TCP4_COMPLETION_TOKEN      CompletionToken;
  EFI_HANDLE                     NewChildHandle;
}   EFI_TCP4_LISTEN_TOKEN;
```

Status
The *Status* in *CompletionToken* will be set to the following value if accept finishes:

EFI_SUCCESS. A remote peer has successfully established a connection to *this* instance. A new TCP instance has also been created for the connection.

EFI_CONNECTION_RESET. The accept fails because the connection is reset either by instance itself or communication peer.

EFI_ABORTED. The accept request has been aborted.

NewChildHandle
The new TCP instance handle created for the established connection.

Description

The *Accept()* function initiates an asynchronous accept request to wait for an incoming connection on the passive TCP instance. If a remote peer successfully establishes a connection with this instance, a new TCP instance will be created and its handle will be returned in *ListenToken->NewChildHandle*. The newly created instance is configured by inheriting the passive instance's configuration and is ready for use upon return. The instance is in the Tcp4StateEstablished state.

The *ListenToken->CompletionToken.Event* will be signaled when a new connection is accepted, user aborts the listen or connection is reset.

This function only can be called when current TCP instance is in Tcp4StateListen state.

Status Codes Returned

EFI_SUCCESS	The listen token has been queued successfully.
EFI_NOT_STARTED	This EFI TCPv4 Protocol instance has not been configured.
EFI_ACCESS_DENIED	One or more of the following are **TRUE**: • This instance is not a passive instance. • This instance is not in *Tcp4StateListen* state. • The same listen token has already existed in the listen token queue of this TCP instance.
EFI_INVALID_PARAMETER	One or more of the following are **TRUE**: • *This* is **NULL**. • *ListenToken* is **NULL**. • *ListentToken->CompletionToken.Event* is **NULL**.
EFI_OUT_OF_RESOURCES	Could not allocate enough resource to finish the operation.
EFI_DEVICE_ERROR	Any unexpected and not belonged to above category error.

28.1.10 EFI_TCP4_PROTOCOL.Transmit()

Summary

Queues outgoing data into the transmit queue.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP4_TRANSMIT) (
  IN EFI_TCP4_PROTOCOL        *This,
  IN EFI_TCP4_IO_TOKEN        *Token
  );
```

Parameters

This
Pointer to the *EFI_TCP4_PROTOCOL* instance.

Token
Pointer to the completion token to queue to the transmit queue. Type *EFI_TCP4_IO_TOKEN* is defined in "Related Definitions" below.

Description

The *Transmit()* function queues a sending request to this TCPv4 instance along with the user data. The status of the token is updated and the event in the token will be signaled once the data is sent out or some error occurs.

Related Definition

```
//*******************************************************
// EFI_TCP4_IO_TOKEN
//*******************************************************
typedef struct {
  EFI_TCP4_COMPLETION_TOKEN    CompletionToken;
  union {
    EFI_TCP4_RECEIVE_DATA      *RxData;
    EFI_TCP4_TRANSMIT_DATA     *TxData;
  }                Packet;
} EFI_TCP4_IO_TOKEN;
```

Status
When transmission finishes or meets any unexpected error it will be set to one of the following values:

EFI_SUCCESS. The receiving or transmission operation completes successfully.

EFI_CONNECTION_FIN: The receiving operation fails because the communication peer has closed the connection and there is no more data in the receive buffer of the instance.

EFI_CONNECTION_RESET. The receiving or transmission operation fails because this connection is reset either by instance itself or communication peer.

EFI_ABORTED. The receiving or transmission is aborted.

EFI_TIMEOUT. The transmission timer expires and no more specific information is available.

EFI_NETWORK_UNREACHABLE. The transmission fails because an ICMP network unreachable error is received.

EFI_HOST_UNREACHABLE. The transmission fails because an ICMP host unreachable error is received.

EFI_PROTOCOL_UNREACHABLE. The transmission fails because an ICMP protocol unreachable error is received.

EFI_PORT_UNREACHABLE. The transmission fails and an ICMP port unreachable error is received.

EFI_ICMP_ERROR. The transmission fails and some other ICMP error is received.

EFI_DEVICE_ERROR. An unexpected system or network error occurs.

EFI_NO_MEDIA. There was a media error

RxData
When this token is used for receiving, *RxData* is a pointer to *EFI_TCP4_RECEIVE_DATA*. Type *EFI_TCP4_RECEIVE_DATA* is defined below.

TxData
When this token is used for transmitting, *TxData* is a pointer to *EFI_TCP4_TRANSMIT_DATA*. Type *EFI_TCP4_TRANSMIT_DATA* is defined below.

The *EFI_TCP4_IO_TOKEN* structures are used for both transmit and receive operations.

When used for transmitting, the *CompletionToken.Event* and *TxData* fields must be filled in by the user. After the transmit operation completes, the *CompletionToken.Status* field is updated by the instance and the *Event* is signaled.

- When used for receiving, the *CompletionToken.Event* and *RxData* fields must be filled in by the user. After a receive operation completes, *RxData* and *Status* are updated by the instance and the *Event* is signaled.

```
**********************************************************
// TCP4 Token Status definition
//
**********************************************************
#define EFI_CONNECTION_FIN EFIERR      (104)
#define EFI_CONNECTION_RESET EFIERR    (105)
#define EFI_CONNECTION_REFUSED EFIERR  (106)
```

NOTE: *EFIERR() sets the maximum bit. Similar to how error codes are described in Status Codes.*

```
//*******************************************************
// EFI_TCP4_RECEIVE_DATA
//*******************************************************
typedef struct {
  BOOLEAN                    UrgentFlag;
  UINT32                     DataLength;
  UINT32                     FragmentCount;
  EFI_TCP4_FRAGMENT_DATA     FragmentTable[1];
}   EFI_TCP4_RECEIVE_DATA;
```

UrgentFlag
Whether those data are urgent. When this flag is set, the instance is in urgent mode. The implementations of this specification should follow RFC793 to process urgent data, and should NOT mix the data across the urgent point in one token.

DataLength
When calling Receive() function, it is the byte counts of all Fragmentbuffer in FragmentTable allocated by user. When the token is signaled by TCPv4 driver it is the length of received data in the fragments.

FragmentCount
Number of fragments.

FragmentTable
An array of fragment descriptors. Type *EFI_TCP4_FRAGMENT_DATA* is defined below.

When TCPv4 driver wants to deliver received data to the application, it will pick up the first queued receiving token, update its *Token->Packet.RxData* then signal the *Token->CompletionToken.Event*.

- The *FragmentBuffers* in *FragmentTable* are allocated by the application when calling *Receive()* function and received data will be copied to those buffers by the driver. *FragmentTable* may contain multiple buffers that are NOT in the continuous memory locations. The application should combine those buffers in the *FragmentTable* to process data if necessary.

```
//*******************************************************
// EFI_TCP4_FRAGMENT_DATA
//*******************************************************
typedef struct {
  UINT32                FragmentLength;
  VOID                  *FragmentBuffer;
}   EFI_TCP4_FRAGMENT_DATA;
```

FragmentLength
Length of data buffer in the fragment.

FragmentBuffer
Pointer to the data buffer in the fragment.

EFI_TCP4_FRAGMENT_DATA allows multiple receive or transmit buffers to be specified. The purpose of this structure is to provide scattered read and write.

```
//*******************************************************
// EFI_TCP4_TRANSMIT_DATA
//*******************************************************
typedef struct {
  BOOLEAN                    Push;
  BOOLEAN                    Urgent;
```

(continues on next page)

(continued from previous page)

```
  UINT32                      DataLength;
  UINT32                      FragmentCount;
  EFI_TCP4_FRAGMENT_DATA      FragmentTable[1];
}   EFI_TCP4_TRANSMIT_DATA;
```

Push
If *TRUE,* data must be transmitted promptly, and the PUSH bit in the last TCP segment created will be set. If *FALSE,* data transmission may be delay to combine with data from subsequent *Transmit()* s for efficiency.

Urgent
The data in the fragment table are urgent and urgent point is in effect if **TRUE**. Otherwise, those data are NOT considered urgent.

DataLength
Length of the data in the fragments.

FragmentCount
Number of fragments.

FragmentTable
A array of fragment descriptors. Type *EFI_TCP4_FRAGMENT_DATA* is defined above.

The EFI TCPv4 Protocol user must fill this data structure before sending a packet. The packet may contain multiple buffers in non-continuous memory locations.

Status Codes Returned

EFI_SUCCESS	The data has been queued for transmission.
EFI_NOT_STARTED	This EFI TCPv4 Protocol instance has not been configured.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_INVALID_PARAMETER	One or more of the following are **TRUE**: • *This* is **NULL**. • *Token* is **NULL**. • *Token->CompletionToken.Event* is **NULL**. • *Token->Packet.TxData* is **NULL**. • *Token->Packet.FragmentCount* is zero. • *Token->Packet.DataLength* is not equal to the sum of fragment lengths.
EFI_ACCESS_DENIED	One or more of the following conditions is **TRUE**: • A transmit completion token with the same *Token->CompletionToken.Event* was already in the transmission queue. • The current instance is in Tcp4StateClosed state. • The current instance is a passive one and it is in *Tcp4StateListen* state. • User has called *Close()* to disconnect this connection.
EFI_NOT_READY	The completion token could not be queued because the transmit queue is full.
EFI_OUT_OF_RESOURCES	Could not queue the transmit data because of resource shortage.
EFI_NETWORK_UNREACHABLE	There is no route to the destination network or address.

continues on next page

Table 28.6 – continued from previous page

EFI_NO_MEDIA	There was a media error.

28.1.10.1 EFI_TCP4_PROTOCOL.Receive()

Summary

Places an asynchronous receive request into the receiving queue.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP4_RECEIVE) (
  IN EFI_TCP4_PROTOCOL        *This,
  IN EFI_TCP4_IO_TOKEN        *Token
  );
```

Parameters

This
Pointer to the *EFI_TCP4_PROTOCOL* instance.

Token
Pointer to a token that is associated with the receive data descriptor. Type *EFI_TCP4_IO_TOKEN* is defined in *EFI_TCP4_PROTOCOL .Transmit()*.

Description

The *Receive()* function places a completion token into the receive packet queue. This function is always asynchronous. The caller must allocate the *Token->CompletionToken.Event* and the *FragmentBuffer* used to receive data. He also must fill the *DataLength* which represents the whole length of all *FragmentBuffer*. When the receive operation completes, the EFI TCPv4 Protocol driver updates the *Token->CompletionToken.Status* and *Token->Packet.RxData* fields and the *Token->CompletionToken.Event* is signaled. If got data the data and its length will be copy into the *FragmentTable,* in the same time the full length of received data will be recorded in the *DataLength* fields. Providing a proper notification function and context for the event will enable the user to receive the notification and receiving status. That notification function is guaranteed to not be re-entered.

Status Codes Returned

EFI_SUCCESS	The receive completion token was cached.
EFI_NOT_STARTED	This EFI TCPv4 Protocol instance has not been configured.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.

continues on next page

Table 28.7 – continued from previous page

EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *This* is **NULL**. • *Token* is **NULL**. • *Token->CompletionToken.Event* is **NULL**. • *Token->Packet.RxData* is **NULL**. • *Token->Packet.RxData->DataLength* is *0*. • The *Token->Packet. RxData->DataLength* is not the sum of all *FragmentBuffer* length in *FragmentTable*.
EFI_OUT_OF_RESOURCES	The receive completion token could not be queued due to a lack of system resources (usually memory).
EFI_DEVICE_ERROR	An unexpected system or network error occurred. The EFI TCPv4 Protocol instance has been reset to startup defaults.
EFI_ACCESS_DENIED	One or more of the following conditions is **TRUE**: • A receive completion token with the same *Token->CompletionToken.Event* was already in the receive queue. • The current instance is in Tcp4StateClosed state. • The current instance is a passive one and it is in *Tcp4StateListen* state. • User has called *Close()* to disconnect this connection.
EFI_CONNECTION_FIN	The communication peer has closed the connection and there is no any buffered data in the receive buffer of this instance.
EFI_NOT_READY	The receive request could not be queued because the receive queue is full.
EFI_NO_MEDIA	There was a media error.

28.1.11 EFI_TCP4_PROTOCOL.Close()

Summary

Disconnecting a TCP connection gracefully or reset a TCP connection. This function is a nonblocking operation.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP4_CLOSE)(
  IN EFI_TCP4_PROTOCOL            *This,
  IN EFI_TCP4_CLOSE_TOKEN         *CloseToken
  );
```

Parameters

This
Pointer to the *EFI_TCP4_PROTOCOL* instance.

CloseToken
Pointer to the close token to return when operation finishes. Type *EFI_TCP4_CLOSE_TOKEN* is defined in "Related Definition" below.

Related Definition

```
//*******************************************************
// EFI_TCP4_CLOSE_TOKEN
//*******************************************************
typedef struct {
  EFI_TCP4_COMPLETION_TOKEN         CompletionToken;
  BOOLEAN                           AbortOnClose;
}   EFI_TCP4_CLOSE_TOKEN;
```

Status

When close finishes or meets any unexpected error it will be set to one of the following values:

EFI_SUCCESS. The close operation completes successfully.

EFI_ABORTED. User called configure with NULL without close stopping.

AbortOnClose

Abort the TCP connection on close instead of the standard TCP close process when it is set to **TRUE**. This option can be used to satisfy a fast disconnect.

Description

Initiate an asynchronous close token to TCP driver. After *Close()* is called, any buffered transmission data will be sent by TCP driver and the current instance will have a graceful close working flow described as RFC 793 if *AbortOnClose* is set to *FALSE,* otherwise, a rest packet will be sent by TCP driver to fast disconnect this connection. When the close operation completes successfully the TCP instance is in Tcp4StateClosed state, all pending asynchronous operation is signaled and any buffers used for TCP network traffic is flushed.

Status Codes Returned

EFI_SUCCESS	The *Close()* is called successfully.
EFI_NOT_STARTED	This EFI TCPv4 Protocol instance has not been configured.
EFI_ACCESS_DENIED	One or more of the following are **TRUE**: • *Configure()* has been called with *TcpConfigData* set to **NULL** and this function has not returned. • Previous *Close()* call on this instance has not finished.
EFI_INVALID_PARAMETER	One or more of the following are **TRUE**: • *This* is **NULL**. • *CloseToken* is **NULL**. • *CloseToken->CompletionToken. Event* is **NULL**.
EFI_OUT_OF_RESOURCES	Could not allocate enough resource to finish the operation.
EFI_DEVICE_ERROR	Any unexpected and not belonged to above category error.

28.1.12 EFI_TCP4_PROTOCOL.Cancel()

Summary

Abort an asynchronous connection, listen, transmission or receive request.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP4_CANCEL)(
  IN EFI_TCP4_PROTOCOL              *This,
  IN EFI_TCP4_COMPLETION_TOKEN      *Token OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_TCP4_PROTOCOL* instance.

Token
Pointer to a token that has been issued by

EFI_TCP4_PROTOCOL.Connect(),
EFI_TCP4_PROTOCOL.Accept(),
EFI_TCP4_PROTOCOL.Transmit() or
EFI_TCP4_PROTOCOL.Receive(). If **NULL**, all pending tokens issued by above four functions will be aborted. Type
EFI_TCP4_COMPLETION_TOKEN is defined in
EFI_TCP4_PROTOCOL.Connect().

Description

The *Cancel()* function aborts a pending connection, listen, transmit or receive request. If *Token* is not **NULL** and the token is in the connection, listen, transmission or receive queue when it is being cancelled, its *Token->Status* will be set to *EFI_ABORTED* and then *Token->Event* will be signaled. If the token is not in one of the queues, which usually means that the asynchronous operation has completed, *EFI_NOT_FOUND* is returned. If *Token* is **NULL** all asynchronous token issued by *Connect(), Accept(), Transmit()* and *Receive()* will be aborted.

Status Codes Returned

EFI_SUCCESS	The asynchronous I/O request is aborted and *Token->Event* is signaled.
EFI_INVALID_PARAMETER	*This* is **NULL**.
EFI_NOT_STARTED	This instance hasn't been configured.
EFI_NO_MAPPING	When using the default address, configuration (DHCP, BOOTP, RARP, etc.) hasn't finished yet.
EFI_NOT_FOUND	The asynchronous I/O request isn't found in the transmission or receive queue. It has either completed or wasn't issued by *Transmit()* and *Receive()*.
EFI_UNSUPPORTED	The implementation does not support this function.

28.1.13 EFI_TCP4_PROTOCOL.Poll()

Summary

Poll to receive incoming data and transmit outgoing segments.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP4_POLL) (
  IN EFI_TCP4_PROTOCOL        *This
  );
```

Parameters

This
Pointer to the *EFI_TCP4_PROTOCOL* instance.

Description

The *Poll()* function increases the rate that data is moved between the network and application and can be called when the TCP instance is created successfully. Its use is optional.

In some implementations, the periodical timer in the MNP driver may not poll the underlying communications device fast enough to avoid drop packets. Drivers and applications that are experiencing packet loss should try calling the *Poll()* function in a high frequency.

Status Codes Returned

EFI_SUCCESS	Incoming or outgoing data was processed.
EFI_INVALID_PARAMETER	*This* is **NULL**.
EFI_DEVICE_ERROR	An unexpected system or network error occurred.
EFI_NOT_READY	No incoming or outgoing data is processed.
EFI_TIMEOUT	Data was dropped out of the transmission or receive queue. Consider increasing the polling rate.

28.2 EFI TCPv6 Protocol

This section defines the EFI TCPv6 (Transmission Control Protocol version 6) Protocol.

28.2.1 TCPv6 Service Binding Protocol

28.2.2 EFI_TCP6_SERVICE_BINDING_PROTOCOL

Summary

The EFI TCPv6 Service Binding Protocol is used to locate EFI TCPv6 Protocol drivers to create and destroy protocol child instance of the driver to communicate with other host using TCP protocol.

GUID

```
#define EFI_TCP6_SERVICE_BINDING_PROTOCOL_GUID \
  {0xec20eb79,0x6c1a,0x4664,\
    {0x9a,0x0d,0xd2,0xe4,0xcc,0x16,0xd6, 0x64}}
```

Description

A network application that requires TCPv6 I/O services can call one of the protocol handler services, such as *BS->LocateHandleBuffer(),* to search devices that publish an EFI TCPv6 Service Binding Protocol GUID. Such device supports the EFI TCPv6 Protocol and may be available for use.

After a successful call to the **EFI_TCP6_SERVICE_BINDING_PROTOCOL.CreateChild()** function, the newly created child EFI TCPv6 Protocol driver is in an un-configured state; it is not ready to do any operation except *Poll()* send and receive data packets until configured.

Every successful call to the **EFI_TCP6_SERVICE_BINDING_PROTOCOL.CreateChild()** function must be matched with a call to the **EFI_TCP6_SERVICE_BINDING_PROTOCOL.DestroyChild()** function to release the protocol driver.

28.2.3 TCPv6 Protocol

28.2.4 EFI_TCP6_PROTOCOL

Summary

The EFI TCPv6 Protocol provides services to send and receive data stream.

GUID

```
#define EFI_TCP6_PROTOCOL_GUID \
  {0x46e44855,0xbd60,0x4ab7,\
    {0xab,0x0d,0xa6,0x79,0xb9,0x44,0x7d,0x77}}
```

Protocol Interface Structure

```
typedef struct _EFI_TCP6_PROTOCOL {
  EFI_TCP6_GET_MODE_DATA      GetModeData;
  EFI_TCP6_CONFIGURE          Configure;
  EFI_TCP6_CONNECT            Connect;
  EFI_TCP6_ACCEPT             Accept;
  EFI_TCP6_TRANSMIT           Transmit;
  EFI_TCP6_RECEIVE            Receive;
  EFI_TCP6_CLOSE              Close;
  EFI_TCP6_CANCEL             Cancel;
  EFI_TCP6_POLL               Poll;
}   EFI_TCP6_PROTOCOL;
```

Parameters

GetModeData
Get the current operational status. See the *GetModeData()* function description.

Configure
Initialize, change, or brutally reset operational settings of the EFI TCPv6 Protocol. See the *Configure()* function description.

Connect
Initiate the TCP three-way handshake to connect to the remote peer configured in this TCP instance. The function is a nonblocking operation. See the *Connect()* function description.

Accept
Listen for incoming TCP connection requests. This function is a nonblocking operation. See the *Accept()* function description.

Transmit
Queue outgoing data to the transmit queue. This function is a nonblocking operation. See the *Transmit()* function description.

Receive
Queue a receiving request token to the receive queue. This function is a nonblocking operation. See the *Receive()* function description.

Close
Gracefully disconnect a TCP connection follow RFC 793 or reset a TCP connection. This function is a nonblocking operation. See the *Close()* function description.

Cancel
Abort a pending connect, listen, transmit or receive request. See the *Cancel()* function description.

Poll
Poll to receive incoming data and transmit outgoing TCP segments. See the *Poll()* function description.

Description

The *EFI_TCP6_PROTOCOL* defines the EFI TCPv6 Protocol child to be used by any network drivers or applications to send or receive data stream. It can either listen on a specified port as a service or actively connect to remote peer as a client. Each instance has its own independent settings.

Note: *Byte Order: In this document, all IPv6 addresses and incoming/outgoing packets are stored in network byte order. All other parameters in the functions and data structures that are defined in this document are stored in host byte order unless explicitly specified.*

28.2.5 EFI_TCP6_PROTOCOL.GetModeData()

Summary

Get the current operational status.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP6_GET_MODE_DATA) (
  IN EFI_TCP6_PROTOCOL                 *This,
  OUT EFI_TCP6_CONNECTION_STATE        *Tcp6State OPTIONAL,
  OUT EFI_TCP6_CONFIG_DATA             *Tcp6ConfigData OPTIONAL,
  OUT EFI_IPv6_MODE_DATA               *Ip6ModeData OPTIONAL,
  OUT EFI_MANAGED_NETWORK_CONFIG_DATA  *MnpConfigData OPTIONAL,
  OUT EFI_SIMPLE_NETWORK_MODE          *SnpModeData OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_TCP6_PROTOCOL* instance.

Tcp6State
The buffer in which the current TCP state is returned. Type *EFI_TCP6_CONNECTION_STATE* is defined in "Related Definitions" below.

Tcp6ConfigData
The buffer in which the current TCP configuration is returned. Type *EFI_TCP6_CONFIG_DATA* is defined in "Related Definitions" below.

Ip6ModeData
The buffer in which the current IPv6 configuration data used by the TCP instance is returned. Type *EFI_IP6_MODE_DATA* is defined in *EFI_IP6_PROTOCOL.GetModeData()*.

MnpConfigData
The buffer in which the current MNP configuration data used indirectly by the TCP instance is returned. Type *EFI_MANAGED_NETWORK_CONFIG_DATA* is defined in *EFI_MANAGED_NETWORK_PROTOCOL.GetModeData()*.

SnpModeData
The buffer in which the current SNP mode data used indirectly by the TCP instance is returned. Type *EFI_SIMPLE_NETWORK_MODE* is defined in the *EFI_SIMPLE_NETWORK_PROTOCOL*.

Description

The GetModeData() function copies the current operational settings of this EFI TCPv6 Protocol instance into user-supplied buffers. This function can also be used to retrieve the operational setting of underlying drivers such as IPv6, MNP, or SNP.

Related Definition

```
typedef struct {
  EFI_IPv6_ADDRESS         StationAddress;
  UINT16                   StationPort;
  EFI_IPv6_ADDRESS         RemoteAddress;
  UINT16                   RemotePort;
  BOOLEAN                  ActiveFlag;
}   EFI_TCP6_ACCESS_POINT;
```

StationAddress
The local IP address assigned to this TCP instance. The EFI TCPv6 driver will only deliver incoming packets whose destination addresses exactly match the IP address. Set to zero to let the underlying IPv6 driver choose a source address. If not zero it must be one of the configured IP addresses in the underlying IPv6 driver.

StationPort
The local port number to which this EFI TCPv6 Protocol instance is bound. If the instance doesn't care the local port number, set StationPort to zero to use an ephemeral port.

RemoteAddress
The remote IP address to which this EFI TCPv6 Protocol instance is connected. If *ActiveFlag* is **FALSE** (i.e., a passive TCPv6 instance), the instance only accepts connections from the *RemoteAddress*. If *ActiveFlag* is **TRUE** the instance will connect to the *RemoteAddress,* i.e., outgoing segments will be sent to this address and only segments from this address will be delivered to the application. When *ActiveFlag* is *FALSE,* it can be set to zero and means that incoming connection requests from any address will be accepted.

RemotePort
The remote port to which this EFI TCPv6 Protocol instance connects or from which connection request will be accepted by this EFI TCPv6 Protocol instance. If *ActiveFlag* is **FALSE** it can be zero and means that incoming connection request from any port will be accepted. Its value can not be zero when *ActiveFlag* is **TRUE**.

ActiveFlag
Set it to **TRUE** to initiate an active open. Set it to **FALSE** to initiate a passive open to act as a server.

```
//*******************************************************
// EFI_TCP6_OPTION
//*******************************************************
typedef struct {
  UINT32         ReceiveBufferSize;
```

(continues on next page)

(continued from previous page)

```
  UINT32         SendBufferSize;
  UINT32         MaxSynBackLog;
  UINT32         ConnectionTimeout;
  UINT32         DataRetries;
  UINT32         FinTimeout;
  UINT32         TimeWaitTimeout;
  UINT32         KeepAliveProbes;
  UINT32         KeepAliveTime;
  UINT32         KeepAliveInterval;
  BOOLEAN        EnableNagle;
  BOOLEAN        EnableTimeStamp;
  BOOLEAN        EnableWindowScaling;
  BOOLEAN        EnableSelectiveAck;
  BOOLEAN        EnablePathMtuDiscovery;
}   EFI_TCP6_OPTION;
```

ReceiveBufferSize
The size of the TCP receive buffer.

SendBufferSize
The size of the TCP send buffer.

MaxSynBackLog
The length of incoming connect request queue for a passive instance. When set to zero, the value is implementation specific.

ConnectionTimeout
The maximum seconds a TCP instance will wait for before a TCP connection established. When set to zero, the value is implementation specific.

DataRetries
The number of times TCP will attempt to retransmit a packet on an established connection. When set to zero, the value is implementation specific.

FinTimeout
How many seconds to wait in the FIN_WAIT_2 states for a final FIN flag before the TCP instance is closed. This timeout is in effective only if the application has called Close() to disconnect the connection completely. It is also called FIN_WAIT_2 timer in other implementations. When set to zero, it should be disabled because the FIN_WAIT_2 timer itself is against the standard.

TimeWaitTimeout
How many seconds to wait in TIME_WAIT state before the TCP instance is closed. The timer is disabled completely to provide a method to close the TCP connection quickly if it is set to zero. It is against the related RFC documents.

KeepAliveProbes
The maximum number of TCP keep-alive probes to send before giving up and resetting the connection if no response from the other end. Set to zero to disable keep-alive probe.

KeepAliveTime
The number of seconds a connection needs to be idle before TCP sends out periodical keep-alive probes. When set to zero, the value is implementation specific. It should be ignored if keep-alive probe is disabled.

KeepAliveInterval
The number of seconds between TCP keep-alive probes after the periodical keep-alive probe if no response. When set to zero, the value is implementation specific. It should be ignored if keep-alive probe is disabled.

EnableNagle
Set it to **TRUE** to enable the Nagle algorithm as defined in RFC896. Set it to **FALSE** to disable it.

EnableTimeStamp
Set it to **TRUE** to enable TCP timestamps option as defined in RFC7323. Set to **FALSE** to disable it.

EnableWindowScaling
Set it to **TRUE** to enable TCP window scale option as defined in RFC7323. Set it to **FALSE** to disable it.

EnableSelectiveAck
Set it to **TRUE** to enable selective acknowledge mechanism described in RFC 2018. Set it to **FALSE** to disable it. Implementation that supports SACK can optionally support DSAK as defined in RFC 2883.

EnablePathMtudiscovery
Set it to **TRUE** to enable path MTU discovery as defined in RFC 1191. Set to **FALSE** to disable it.

Option setting with digital value will be modified by driver if it is set out of the implementation specific range and an implementation specific default value will be set accordingly.

```
//*******************************************************
// EFI_TCP6_CONFIG_DATA
//*******************************************************
typedef struct {
  UINT8                    TrafficClass;
  UINT8                    HopLimit;
  EFI_TCP6_ACCESS_POINT    AccessPoint;
  EFI_TCP6_OPTION          *ControlOption;
}   EFI_TCP6_CONFIG_DATA;
```

TrafficClass
TrafficClass field in transmitted IPv6 packets.

HopLimit
HopLimit field in transmitted IPv6 packets.

AccessPoint
Used to specify TCP communication end settings for a TCP instance.

ControlOption
Used to configure the advance TCP option for a connection. If set to *NULL,* implementation specific options for TCP connection will be used.

```
//*******************************************************
// EFI_TCP6_CONNECTION_STATE
//*******************************************************

typedef enum {
  Tcp6StateClosed = 0,
  Tcp6StateListen = 1,
  Tcp6StateSynSent = 2,
  Tcp6StateSynReceived = 3,
  Tcp6StateEstablished = 4,
  Tcp6StateFinWait1 = 5,
  Tcp6StateFinWait2 = 6,
  Tcp6StateClosing = 7,
  Tcp6StateTimeWait = 8,
  Tcp6StateCloseWait = 9,
```

(continues on next page)

(continued from previous page)

```
  Tcp6StateLastAck = 10
}   EFI_TCP6_CONNECTION_STATE;
```

Status Codes Returned

EFI_SUCCESS	The mode data was read.
EFI_NOT_STARTED	No configuration data is available because this instance hasn't been started.
EFI_INVALID_PARAMETER	*This* is **NULL**.

28.2.6 EFI_TCP6_PROTOCOL.Configure()

Summary

Initialize or brutally reset the operational parameters for this TCP instance.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP6_CONFIGURE) (
  IN EFI_TCP6_PROTOCOL        *This,
  IN EFI_TCP6_CONFIG_DATA     *Tcp6ConfigData OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_TCP6_PROTOCOL* instance.

Tcp6ConfigData
Pointer to the configure data to configure the instance.

Description

The *Configure()* function does the following:

- Initialize this TCP instance, i.e., initialize the communication end settings and specify active open or passive open for an instance.
- Reset this TCP instance brutally, i.e., cancel all pending asynchronous tokens, flush transmission and receiving buffer directly without informing the communication peer.

No other TCPv6 Protocol operation except *Poll()* can be executed by this instance until it is configured properly. For an active TCP instance, after a proper configuration it may call *Connect()* to initiates the three-way handshake. For a passive TCP instance, its state will transit to *Tcp6StateListen* after configuration, and *Accept()* may be called to listen the incoming TCP connection requests. If *Tcp6ConfigData* is set to *NULL,* the instance is reset. Resetting process will be done brutally, the state machine will be set to *Tcp6StateClosed* directly, the receive queue and transmit queue will be flushed, and no traffic is allowed through this instance.

Status Codes Returned

EFI_SUCCESS	The operational settings are set, changed, or reset successfully.
EFI_NO_MAPPING	The underlying IPv6 driver was responsible for choosing a source address for this instance, but no source address was available for use.

continues on next page

Table 28.12 – continued from previous page

EFI_INVALID_PARAMETER	One or more of the following conditions are **TRUE**: • *This* is **NULL**. • *Tcp6Config Data->AccessPoint.StationAddress* is neither zero nor one of the configured IP addresses in the underlying IPv6 driver. • *Tcp6Confi gData->AccessPoint.RemoteAddress* isn't a valid unicast IPv6 address. • *Tcp6Confi gData->AccessPoint.RemoteAddress* is zero or *Tcp6Co nfigData->AccessPoint.RemotePort* is zero when *Tcp6Co nfigData->AccessPoint.ActiveFlag* is **TRUE**. • A same access point has been configured in other TCP instance properly.
EFI_ACCESS_DENIED	Configuring TCP instance when it is configured without calling *Configure()* with **NULL** to reset it.
EFI_UNSUPPORTED	One or more of the control options are not supported in the implementation.
EFI_OUT_OF_RESOURCES	Could not allocate enough system resources when executing *Configure()*.
EFI_DEVICE_ERROR	An unexpected network or system error occurred.

28.2.7 EFI_TCP6_PROTOCOL.Connect()

Summary

Initiate a nonblocking TCP connection request for an active TCP instance.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP6_CONNECT) (
  IN EFI_TCP6_PROTOCOL              *This,
  IN EFI_TCP6_CONNECTION_TOKEN      *ConnectionToken
  );
```

Parameters

This
Pointer to the *EFI_TCP6_PROTOCOL* instance.

ConnectionToken
Pointer to the connection token to return when the TCP three-way handshake finishes. Type *EFI_TCP6_CONNECTION_TOKEN* is defined in Related Definition below.

Description

The *Connect()* function will initiate an active open to the remote peer configured in current TCP instance if it is configured active. If the connection succeeds or fails due to any error, the *ConnectionToken->CompletionToken.Event* will be signaled and *ConnectionToken->CompletionToken.Status* will be updated accordingly. This function can only be called for the TCP instance in *Tcp6StateClosed* state. The instance will transfer into *Tcp6StateSynSent* if the function returns *EFI_SUCCESS*. If TCP three-way handshake succeeds, its state will become *Tcp6StateEstablished,* otherwise, the state will return to *Tcp6StateClosed*.

Related Definition

```
//*******************************************************
// EFI_TCP6_COMPLETION_TOKEN
//*******************************************************
typedef struct {
  EFI_EVENT          Event;
  EFI_STATUS         Status;
}   EFI_TCP6_COMPLETION_TOKEN;
```

Event
The Event to signal after request is finished and Status field is updated by the EFI TCPv6 Protocol driver. The type of Event must be *EVT_NOTIFY_SIGNAL*.

Status
The result of the completed operation. *EFI_NO_MEDIA*. There was a media error

The *EFI_TCP6_COMPLETION_TOKEN* is used as a common header for various asynchronous tokens.

```
//*******************************************************
// EFI_TCP6_CONNECTION_TOKEN
//*******************************************************
typedef struct {
  EFI_TCP6_COMPLETION_TOKEN         CompletionToken;
}   EFI_TCP6_CONNECTION_TOKEN;
```

Status
The *Status* in the *CompletionToken* will be set to one of the following values if the active open succeeds or an unexpected error happens:

EFI_SUCCESS: The active open succeeds and the instance's state is Tcp6StateEstablished.

EFI_CONNECTION_RESET: The connect fails because the connection is reset either by instance itself or the communication peer.

EFI_CONNECTION_REFUSED: The receiving or transmission operation fails because this connection is refused.

EFI_ABORTED: The active open is aborted.

EFI_TIMEOUT: The connection establishment timer expires and no more specific information is available.

EFI_NETWORK_UNREACHABLE: The active open fails because an ICMP network unreachable error is received.

EFI_HOST_UNREACHABLE: The active open fails because an ICMP host unreachable error is received.

EFI_PROTOCOL_UNREACHABLE: The active open fails because an ICMP protocol unreachable error is received.

EFI_PORT_UNREACHABLE: The connection establishment timer times out and an ICMP port unreachable error is received.

EFI_ICMP_ERROR: The connection establishment timer times out and some other ICMP error is received.

EFI_DEVICE_ERROR: An unexpected system or network error occurred.

EFI_SECURITY_VIOLATION: The active open was failed because of IPSec policy check.

Status Codes Returned

EFI_SUCCESS	The connection request is successfully initiated and the state of this TCP instance has been changed to *Tcp6StateSynSent*.
EFI_NOT_STARTED	This EFI TCPv6 Protocol instance has not been configured.
EFI_ACCESS_DENIED	One or more of the following conditions are **TRUE:** • This instance is not configured as an active one. • This instance is not in *Tcp6StateClosed* state.
EFI_INVALID_PARAMETER	One or more of the following are **TRUE:**. *This* is **NULL**. • *ConnectionToken* is **NULL**. • *ConnectionToken->CompletionToken.Event* is **NULL**.
EFI_OUT_OF_RESOURCES	The driver can't allocate enough resource to initiate the active open.
EFI_DEVICE_ERROR	An unexpected system or network error occurred.

28.2.8 EFI_TCP6_PROTOCOL.Accept()

Summary

Listen on the passive instance to accept an incoming connection request. This is a nonblocking operation.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP6_ACCEPT) (
  IN EFI_TCP6_PROTOCOL          *This,
  IN EFI_TCP6_LISTEN_TOKEN      *ListenToken
  );
```

Parameters

This
Pointer to the EFI_TCP6_PROTOCOL instance.

ListenToken
Pointer to the listen token to return when operation finishes. Type EFI_TCP6_LISTEN_TOKEN is defined in Related Definition below.

Related Definition

```
//*******************************************
// EFI_TCP6_LISTEN_TOKEN
//*******************************************
typedef struct {
  EFI_TCP6_COMPLETION_TOKEN      CompletionToken;
  EFI_HANDLE                     NewChildHandle;
}   EFI_TCP6_LISTEN_TOKEN;
```

Status
The *Status* in *CompletionToken* will be set to the following value if accept finishes:

EFI_SUCCESS: A remote peer has successfully established a connection to this instance. A new TCP instance has also been created for the connection.

EFI_CONNECTION_RESET: The accept fails because the connection is reset either by instance itself or communication peer.

EFI_ABORTED: The accept request has been aborted.

EFI_SECURITY_VIOLATION: The accept operation was failed because of IPSec policy check.

NewChildHandle
The new TCP instance handle created for the established connection.

Description

The *Accept()* function initiates an asynchronous accept request to wait for an incoming connection on the passive TCP instance. If a remote peer successfully establishes a connection with this instance, a new TCP instance will be created and its handle will be returned in *ListenToken->NewChildHandle*. The newly created instance is configured by inheriting the passive instance's configuration and is ready for use upon return. The new instance is in the *Tcp6StateEstablished* state.

The *ListenToken->CompletionToken.Event* will be signaled when a new connection is accepted, user aborts the listen or connection is reset.

This function only can be called when current TCP instance is in *Tcp6StateListen* state.

Status Codes Returned

EFI_SUCCESS	The listen token has been queued successfully.
EFI_NOT_STARTED	This EFI TCPv6 Protocol instance has not been configured.

continues on next page

Table 28.14 – continued from previous page

EFI_ACCESS_DENIED	One or more of the following are **TRUE:** • This instance is not a passive instance. • This instance is not in *Tcp6StateListen* state. • The same listen token has already existed in the listen token queue of this TCP instance.
EFI_INVALID_PARAMETER	One or more of the following are **TRUE:** • *This* is **NULL**. • *ListenToken* is **NULL**. • *ListentToken->CompletionToken.Event* is **NULL**.
EFI_OUT_OF_RESOURCES	Could not allocate enough resource to finish the operation.
EFI_DEVICE_ERROR	Any unexpected and not belonged to above category error.

28.2.9 EFI_TCP6_PROTOCOL.Transmit()

Summary

Queues outgoing data into the transmit queue.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP6_TRANSMIT) (
  IN EFI_TCP6_PROTOCOL           *This,
  IN EFI_TCP6_IO_TOKEN           *Token
  );
```

Parameters

This
Pointer to the *EFI_TCP6_PROTOCOL* instance.

Token
Pointer to the completion token to queue to the transmit queue. Type *EFI_TCP6_IO_TOKEN* is defined in "Related Definitions" below.

Description

The *Transmit()* function queues a sending request to this TCP instance along with the user data. The status of the token is updated and the event in the token will be signaled once the data is sent out or some error occurs.

Related Definition

```
//*******************************************************
// EFI_TCP6_IO_TOKEN
//*******************************************************
typedef struct {
EFI_TCP6_COMPLETION_TOKEN     CompletionToken;
```

(continues on next page)

(continued from previous page)

```
union {
  EFI_TCP6_RECEIVE_DATA       *RxData;
  EFI_TCP6_TRANSMIT_DATA      *TxData;
}               Packet;
}   EFI_TCP6_IO_TOKEN;
```

Status
When transmission finishes or meets any unexpected error it will be set to one of the following values:

EFI_SUCCESS: The receiving or transmission operation completes successfully.

EFI_CONNECTION_FIN: The receiving operation fails because the communication peer has closed the connection and there is no more data in the receive buffer of the instance.

EFI_CONNECTION_RESET: The receiving or transmission operation fails because this connection is reset either by instance itself or the communication peer.

EFI_ABORTED: The receiving or transmission is aborted.

EFI_TIMEOUT: The transmission timer expires and no more specific information is available.

EFI_NETWORK_UNREACHABLE: The transmission fails because an ICMP network unreachable error is received.

EFI_HOST_UNREACHABLE: The transmission fails because an ICMP host unreachable error is received.

EFI_PROTOCOL_UNREACHABLE: The transmission fails because an ICMP protocol unreachable error is received.

EFI_PORT_UNREACHABLE: The transmission fails and an ICMP port unreachable error is received.

EFI_ICMP_ERROR: The transmission fails and some other ICMP error is received.

EFI_DEVICE_ERROR: An unexpected system or network error occurs.

EFI_SECURITY_VIOLATION: The receiving or transmission operation was failed because of IPSec policy check.

RxData
When this token is used for receiving, *RxData* is a pointer to *EFI_TCP6_RECEIVE_DATA*. Type *EFI_TCP6_RECEIVE_DATA* is defined below.

TxData
When this token is used for transmitting, *TxData* is a pointer to *EFI_TCP6_TRANSMIT_DATA*. Type *EFI_TCP6_TRANSMIT_DATA* is defined below.

The EFI_TCP6_IO_TOKEN structure is used for both transmit and receive operations.

When used for transmitting, the CompletionToken.Event and TxData fields must be filled in by the user. After the transmit operation completes, the CompletionToken.Status field is updated by the instance and the Event is signaled.

When used for receiving, the CompletionToken.Event and RxData fields must be filled in by the user. After a receive operation completes, RxData and Status are updated by the instance and the Event is signaled.

```
//*******************************************************
// EFI_TCP6_RECEIVE_DATA
//*******************************************************
typedef struct {
  BOOLEAN                  UrgentFlag;
  UINT32                   DataLength;
  UINT32                   FragmentCount;
```

(continues on next page)

(continued from previous page)

```
  EFI_TCP6_FRAGMENT_DATA   FragmentTable[1];
}   EFI_TCP6_RECEIVE_DATA;
```

UrgentFlag
Whether the data is urgent. When this flag is set, the instance is in urgent mode. The implementations of this specification should follow RFC793 to process urgent data, and should NOT mix the data across the urgent point in one token.

DataLength
When calling *Receive()* function, it is the byte counts of all *Fragmentbuffer* in *FragmentTable* allocated by user. When the token is signaled by TCPv6 driver it is the length of received data in the fragments.

FragmentCount
Number of fragments.

FragmentTable
An array of fragment descriptors. Type *EFI_TCP6_FRAGMENT_DATA* is defined below.

When TCPv6 driver wants to deliver received data to the application, it will pick up the first queued receiving token, update its *Token->Packet.RxData* then signal the *Token->CompletionToken.Event*.

The *FragmentBuffer* in *FragmentTable* is allocated by the application when calling *Receive()* function and received data will be copied to those buffers by the driver. FragmentTable may contain multiple buffers that are NOT in the continuous memory locations. The application should combine those buffers in the FragmentTable to process data if necessary.

```
//*******************************************
// EFI_TCP6_FRAGMENT_DATA
//*******************************************
typedef struct {
  UINT32             FragmentLength;
  VOID               *FragmentBuffer;
}   EFI_TCP6_FRAGMENT_DATA;
```

FragmentLength
Length of data buffer in the fragment.

FragmentBuffer
Pointer to the data buffer in the fragment. *EFI_TCP6_FRAGMENT_DATA* allows multiple receive or transmit buffers to be specified. The purpose of this structure is to provide scattered read and write.

```
//*********************************************
// EFI_TCP6_TRANSMIT_DATA
//*********************************************
typedef struct {
  BOOLEAN                  Push;
  BOOLEAN                  Urgent;
  UINT32                   DataLength;
  UINT32                   FragmentCount;
  EFI_TCP6_FRAGMENT_DATA   FragmentTable[1];
}   EFI_TCP6_TRANSMIT_DATA;
```

Push
If **TRUE,** data must be transmitted promptly, and the PUSH bit in the last TCP segment created will be set. If **FALSE,** data transmission may be delayed to combine with data from subsequent *Transmit()* s for efficiency.

Urgent
The data in the fragment table are urgent and urgent point is in effect if **TRUE**. Otherwise, those data are NOT considered urgent.

DataLength
Length of the data in the fragments.

FragmentCount
Number of fragments.

FragmentTable
An array of fragment descriptors. Type *EFI_TCP6_FRAGMENT_DATA* is defined above.

The EFI TCPv6 Protocol user must fill this data structure before sending a packet. The packet may contain multiple buffers in non-continuous memory locations.

Status Codes Returned

EFI_SUCCESS	The data has been queued for transmission.
EFI_NOT_STARTED	This EFI TCPv6 Protocol instance has not been configured.
EFI_NO_MAPPING	The underlying IPv6 driver was responsible for choosing a source address for this instance, but no source address was available for use.
EFI_INVALID_PARAMETER	One or more of the following are **TRUE:** • *This* is **NULL**. • *Token* is **NULL**. • *Token->CompletionToken.Event* is **NULL**. • *Token->Packet.TxData* is **NULL**. • *Token->Packet.FragmentCount* is zero. • *Token->Packet.DataLength* is not equal to the sum of fragment lengths.
EFI_ACCESS_DENIED	One or more of the following conditions are **TRUE:** • A transmit completion token with the same *Token->CompletionToken.Event* was already in the transmission queue. • The current instance is in Tcp6StateClosed state. • The current instance is a passive one and it is in *Tcp6StateListen* state. • User has called *Close()* to disconnect this connection.
EFI_NOT_READY	The completion token could not be queued because the transmit queue is full.
EFI_OUT_OF_RESOURCES	Could not queue the transmit data because of resource shortage.
EFI_NETWORK_UNREACHABLE	There is no route to the destination network or address.
EFI_NO_MEDIA	There was a media error.

28.2.10 EFI_TCP6_PROTOCOL.Receive()

Summary

Places an asynchronous receive request into the receiving queue.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP6_RECEIVE) (
  IN EFI_TCP6_PROTOCOL        *This,
  IN EFI_TCP6_IO_TOKEN        *Token
  );
```

Parameters

This
Pointer to the *EFI_TCP6_PROTOCOL* instance.

Token
Pointer to a token that is associated with the receive data descriptor. Type *EFI_TCP6_IO_TOKEN* is defined in *EFI_TCP6_PROTOCOL.Transmit()*.

Description

The *Receive* () function places a completion token into the receive packet queue. This function is always asynchronous. The caller must allocate the *Token->CompletionToken.Event* and the *FragmentBuffer* used to receive data. The caller also must fill the DataLength which represents the whole length of all *FragmentBuffer*. When the receive operation completes, the EFI TCPv6 Protocol driver updates the *Token->CompletionToken.Status* and *Token->Packet.RxData* fields and the *Token->CompletionToken.Event* is signaled. If got data the data and its length will be copied into the *FragmentTable,* at the same time the full length of received data will be recorded in the *DataLength* fields. Providing a proper notification function and context for the event will enable the user to receive the notification and receiving status. That notification function is guaranteed to not be re-entered.

Status Codes Returned

EFI_SUCCESS	The receive completion token was cached.
EFI_NOT_STARTED	This EFI TCPv6 Protocol instance has not been configured.
EFI_NO_MAPPING	The underlying IPv6 driver was responsible for choosing a source address for this instance, but no source address was available for use.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE:** • *This* is **NULL**. • *Token* is **NULL**. • *Token->CompletionToken.Event* is **NULL**. • *Token->Packet.RxData* is **NULL**. • *Token->Packet.RxData->DataLength* is 0. • The *Token->Packet.RxData->DataLength* is not the sum of all *FragmentBuffer* length in *FragmentTable*.
EFI_OUT_OF_RESOURCES	The receive completion token could not be queued due to a lack of system resources (usually memory).
EFI_DEVICE_ERROR	An unexpected system or network error occurred. The EFI TCPv6 Protocol instance has been reset to startup defaults.

continues on next page

Table 28.16 – continued from previous page

EFI_ACCESS_DENIED	One or more of the following conditions is *TRUE:* • A receive completion token with the same *Token->CompletionToken.Event* was already in the receive queue. • The current instance is in Tcp6StateClosed state. • The current instance is a passive one and it is in *Tcp6StateListen* state. • User has called *Close()* to disconnect this connection.
EFI_CONNECTION_FIN	The communication peer has closed the connection and there is no any buffered data in the receive buffer of this instance.
EFI_NOT_READY	The receive request could not be queued because the receive queue is full.
EFI_NO_MEDIA	There was a media error.

28.2.11 EFI_TCP6_PROTOCOL.Close()

Summary

Disconnecting a TCP connection gracefully or reset a TCP connection. This function is a nonblocking operation.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP6_CLOSE)(
  IN EFI_TCP6_PROTOCOL              *This,
  IN EFI_TCP6_CLOSE_TOKEN           *CloseToken
  );
```

Parameters

This
Pointer to the *EFI_TCP6_PROTOCOL* instance.

CloseToken
Pointer to the close token to return when operation finishes. Type *EFI_TCP6_CLOSE_TOKEN* is defined in Related Definition below.

Related Definition

```
//*******************************************
// EFI_TCP6_CLOSE_TOKEN
//*******************************************
typedef struct {
  EFI_TCP6_COMPLETION_TOKEN   CompletionToken;
  BOOLEAN                     AbortOnClose;
}   EFI_TCP6_CLOSE_TOKEN;
```

Status
When close finishes or meets any unexpected error it will be set to one of the following values:

EFI_SUCCESS: The close operation completes successfully.

EFI_ABORTED: User called configure with NULL without close stopping.

EFI_SECURITY_VIOLATION: The close operation was failed because of IPSec policy check

AbortOnClose
Abort the TCP connection on close instead of the standard TCP close process when it is set to **TRUE**. This option can be used to satisfy a fast disconnect.

Description

Initiate an asynchronous close token to TCP driver. After *Close()* is called, any buffered transmission data will be sent by TCP driver and the current instance will have a graceful close working flow described as RFC 793 if *AbortOnClose* is set to *FALSE,* otherwise, a rest packet will be sent by TCP driver to fast disconnect this connection. When the close operation completes successfully the TCP instance is in *Tcp6StateClosed* state, all pending asynchronous operations are signaled and any buffers used for TCP network traffic are flushed.

Status Codes Returned

EFI_SUCCESS	The *Close()* is called successfully.
EFI_NOT_STARTED	This EFI TCPv6 Protocol instance has not been configured.
EFI_ACCESS_DENIED	One or more of the following conditions are **TRUE:** • *CloseToken* or *CloseToken->CompletionToken.Event* is already in use. • Previous Close() call on this instance has not finished.
EFI_INVALID_PARAMETER	One or more of the following conditions are **TRUE:** • *This* is **NULL**. • *CloseToken* is **NULL**. • *CloseToken->CompletionToken.Event* is **NULL**.
EFI_OUT_OF_RESOURCES	Could not allocate enough resource to finish the operation.
EFI_DEVICE_ERROR	Any unexpected and not belonged to above category error.

28.2.12 EFI_TCP6_PROTOCOL.Cancel()

Summary

Abort an asynchronous connection, listen, transmission or receive request.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP6_CANCEL)(
  IN EFI_TCP6_PROTOCOL              *This,
  IN EFI_TCP6_COMPLETION_TOKEN      *Token OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_TCP6_PROTOCOL* instance.

Token
Pointer to a token that has been issued by

EFI_TCP6_PROTOCOL.Connect(),
EFI_TCP6_PROTOCOL.Accept(),
EFII_TCP6_PROTOCOL.Transmit() or
EFI_TCP6_PROTOCOL.Receive(). If **NULL**, all pending tokens issued by above four functions will be aborted. Type
EFI_TCP6_COMPLETION_TOKEN is defined in
EFI_TCP6_PROTOCOL.Connect().

Description

The *Cancel()* function aborts a pending connection, listen, transmit or receive request. If *Token* is not **NULL** and the token is in the connection, listen, transmission or receive queue when it is being cancelled, its *Token->Status* will be set to *EFI_ABORTED* and then *Token->Event* will be signaled. If the token is not in one of the queues, which usually means that the asynchronous operation has completed, *EFI_NOT_FOUND* is returned. If *Token* is **NULL** all asynchronous token issued by *Connect(), Accept(), Transmit()* and *Receive()* will be aborted.

Status Codes Returned

EFI_SUCCESS	The asynchronous I/O request is aborted and *Token->Event* is signaled.
EFI_INVALID_PARAMETER	*This* is **NULL**.
EFI_NOT_STARTED	This instance hasn't been configured.
EFI_NOT_FOUND	The asynchronous I/O request isn't found in the transmission or receive queue. It has either completed or wasn't issued by *Transmit()* and *Receive()*.
EFI_UNSUPPORTED	The implementation does not support this function.

28.2.13 EFI_TCP6_PROTOCOL.Poll()

Summary

Poll to receive incoming data and transmit outgoing segments.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TCP6_POLL) (
  IN EFI_TCP6_PROTOCOL       *This
  );
```

Parameters

This
Pointer to the *EFI_TCP6_PROTOCOL* instance.

Description

The *Poll()* function increases the rate that data is moved between the network and application and can be called when the TCP instance is created successfully. Its use is optional.

In some implementations, the periodical timer in the MNP driver may not poll the underlying communications device fast enough to avoid drop packets. Drivers and applications that are experiencing packet loss should try calling the *Poll()* function in a high frequency.

Status Codes Returned

EFI_SUCCESS	Incoming or outgoing data was processed.
EFI_INVALID_PARAMETER	*This* is **NULL**.
EFI_DEVICE_ERROR	An unexpected system or network error occurred.
EFI_NOT_READY	No incoming or outgoing data is processed.
EFI_TIMEOUT	Data was dropped out of the transmission or receive queue. Consider increasing the polling rate.

28.3 EFI IPv4 Protocol

This section defines the EFI IPv4 (Internet Protocol version 4) Protocol interface. It is split into the following three main sections:

- **EFI IPv4 Service Binding Protocol**
- **EFI IPv4 Variable**
- **EFI IPv4 Protocol**

The EFI IPv4 Protocol provides basic network IPv4 packet I/O services, which includes support for a subset of the Internet Control Message Protocol (ICMP) and may include support for the Internet Group Management Protocol (IGMP).

The EFI IPv4 Protocol supports IPv4 classless IP addressing, and deprecates the original IPv4 classful IP addressing. Please see links to the following RFC documents at http://uefi.org/uefi :

1. RFC 1122 — "Requirements for Internet Hosts –Communication Layers",**
2. RFC 4632 — "Classless Inter-domain Routing (CIDR): The Internet Address Assignment and Aggregation Plan",
3. RFC 3021 — "Using 31-Bit Prefixes on IPv4 Point-to-Point Links"

28.3.1 IP4 Service Binding Protocol

28.3.2 EFI_IP4_SERVICE_BINDING_PROTOCOL

Summary

The EFI IPv4 Service Binding Protocol is used to locate communication devices that are supported by an EFI IPv4 Protocol driver and to create and destroy instances of the EFI IPv4 Protocol child protocol driver that can use the underlying communications device.

GUID

```
#define EFI_IP4_SERVICE_BINDING_PROTOCOL_GUID \
  {0xc51711e7,0xb4bf,0x404a,\
    {0xbf,0xb8,0x0a,0x04,0x8e,0xf1,0xff,0xe4}}
```

Description

A network application that requires basic IPv4 I/O services can use one of the protocol handler services, such as *BS->LocateHandleBuffer(),* to search for devices that publish an EFI IPv4 Service Binding Protocol GUID. Each device with a published EFI IPv4 Service Binding Protocol GUID supports the EFI IPv4 Protocol and may be available for use.

After a successful call to the *EFI_IP4_SERVICE_BINDING_PROTOCOL .CreateChild()* function, the newly created child EFI IPv4 Protocol driver is in an unconfigured state; it is not ready to send and receive data packets.

Before a network application terminates execution, every successful call to the *EFI_IP4_SERVICE_BINDING_PROTOCOL .CreateChild()* function must be matched with a call to the *EFI_IP4_SERVICE_BINDING_PROTOCOL.DestroyChild()* function.

28.3.3 IP4 Protocol

28.3.4 EFI_IP4_PROTOCOL

Summary

The EFI IPv4 Protocol implements a simple packet-oriented interface that can be used by drivers, daemons, and applications to transmit and receive network packets.

GUID

```
#define EFI_IP4_PROTOCOL_GUID \
  {0x41d94cd2,0x35b6,0x455a,\
    {0x82,0x58,0xd4,0xe5,0x13,0x34,0xaa,0xdd}}
```

Protocol Interface Structure

```
typedef struct _EFI_IP4_PROTOCOL {
  EFI_IP4_GET_MODE_DATA       GetModeData;
  EFI_IP4_CONFIGURE           Configure;
  EFI_IP4_GROUPS              Groups;
  EFI_IP4_ROUTES              Routes;
  EFI_IP4_TRANSMIT            Transmit;
  EFI_IP4_RECEIVE             Receive;
  EFI_IP4_CANCEL              Cancel;
  EFI_IP4_POLL                Poll;
}   EFI_IP4_PROTOCOL;
```

Parameters

GetModeData
Gets the current operational settings for this instance of the EFI IPv4 Protocol driver. See the *GetModeData()* function description.

Configure
Changes or resets the operational settings for the EFI IPv4 Protocol. See the *Configure()* function description.

Groups
Joins and leaves multicast groups. See the *Groups()* function description.

Routes
Adds and deletes routing table entries. See the *Routes()* function description.

Transmit
Places outgoing data packets into the transmit queue. See the *Transmit()* function description.

Receive
Places a receiving request into the receiving queue. See the *Receive()* function description.

Cancel
Aborts a pending transmit or receive request. See the *Cancel()* function description.

Poll
Polls for incoming data packets and processes outgoing data packets. See the *Poll()* function description.

Description

The *EFI_IP4_PROTOCOL* defines a set of simple IPv4, ICMPv4, and IGMPv4 services that can be used by any network protocol driver, daemon, or application to transmit and receive IPv4 data packets.

NOTE: *All the IPv4 addresses that are described in* EFI_IP4_PROTOCOL *are stored in network byte order. Both incoming and outgoing IP packets are also in network byte order. All other parameters that are defined in functions or data structures are stored in host byte order.*

28.3.5 EFI_IP4_PROTOCOL.GetModeData()

Summary

Gets the current operational settings for this instance of the EFI IPv4 Protocol driver.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP4_GET_MODE_DATA) (
  IN EFI_IP4_PROTOCOL                  *This,
  OUT EFI_IP4_MODE_DATA                *Ip4ModeData OPTIONAL,
  OUT EFI_MANAGED_NETWORK_CONFIG_DATA  *MnpConfigData OPTIONAL,
  OUT EFI_SIMPLE_NETWORK_MODE          *SnpModeData OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_IP4_PROTOCOL* instance.

Ip4ModeData
Pointer to the EFI IPv4 Protocol mode data structure. Type *EFI_IP4_MODE_DATA* is defined in "Related Definitions" below.

MnpConfigData
Pointer to the managed network configuration data structure. Type *EFI_MANAGED_NETWORK_CONFIG_DATA* is defined in *EFI_MANAGED_NETWORK_PROTOCOL.GetModeData()*.

SnpData
Pointer to the simple network mode data structure. Type *EFI_SIMPLE_NETWORK_MODE* is defined in the *EFI_SIMPLE_NETWORK_PROTOCOL*.

Description

The *GetModeData()* function returns the current operational mode data for this driver instance. The data fields in *EFI_IP4_MODE_DATA* are read only. This function is used optionally to retrieve the operational mode data of underlying networks or drivers.

Related Definition

```
//*******************************************
// EFI_IP4_MODE_DATA
//*******************************************
typedef struct {
  BOOLEAN               IsStarted;
  UINT32                MaxPacketSize;
  EFI_IP4_CONFIG_DATA   ConfigData;
  BOOLEAN               IsConfigured;
  UINT32                GroupCount;
  EFI_IPv4_ADDRESS      *GroupTable;
  UINT32                RouteCount;
  EFI_IP4_ROUTE_TABLE   RouteTable;
  UINT32                IcmpTypeCount;
  EFI_IP4_ICMP_TYPE     *IcmpTypeList;
}   EFI_IP4_MODE_DATA;
```

IsStarted
Set to **TRUE** after this EFI IPv4 Protocol instance has been successfully configured with operational parameters by calling the *Configure()* interface when EFI IPv4 Protocol instance is stopped All other fields in this structure are undefined until this field is **TRUE**.

Set to **FALSE** when the instance's operational parameter has been reset.

MaxPackeSize
The maximum packet size, in bytes, of the packet which the upper layer driver could feed.

ConfigData
Current configuration settings. Undefined until *IsStarted* is **TRUE**. Type *EFI_IP4_CONFIG_DATA* is defined below.

IsConfigured
Set to **TRUE** when the EFI IPv4 Protocol instance has a station address and subnet mask. If it is using the default address, the default address has been acquired.

Set to **FALSE** when the EFI IPv4 Protocol driver is not configured.

GroupCount
Number of joined multicast groups. Undefined until *IsConfigured* is **TRUE**.

GroupTable
List of joined multicast group addresses. Undefined until *IsConfigured* is **TRUE**.

RouteCount
Number of entries in the routing table. Undefined until *IsConfigured* is **TRUE**.

RouteTable
Routing table entries. Undefined until *IsConfigured* is **TRUE**. Type *EFI_IP4_ROUTE_TABLE* is defined below.

IcmpTypeCount
Number of entries in the supported ICMP types list.

IcmpTypeList
Array of ICMP types and codes that are supported by this EFI IPv4 Protocol driver. Type *EFI_IP4_ICMP_TYPE* is defined below.

The *EFI_IP4_MODE_DATA* structure describes the operational state of this IPv4 interface.

```
//*******************************************
// EFI_IP4_CONFIG_DATA
//*******************************************
typedef struct {
  UINT8              DefaultProtocol;
  BOOLEAN            AcceptAnyProtocol;
  BOOLEAN            AcceptIcmpErrors;
  BOOLEAN            AcceptBroadcast;
  BOOLEAN            AcceptPromiscuous;
  BOOLEAN            UseDefaultAddress;
  EFI_IPv4_ADDRESS   StationAddress;
  EFI_IPv4_ADDRESS   SubnetMask;
  UINT8              TypeOfService;
  UINT8              TimeToLive;
  BOOLEAN            DoNotFragment;
  BOOLEAN            RawData;
  UINT32             ReceiveTimeout;
  UINT32             TransmitTimeout;
} EFI_IP4_CONFIG_DATA;
```

DefaultProtocol
The default IPv4 protocol packets to send and receive. Ignored when *AcceptPromiscuous* is **TRUE**. An updated list of protocol numbers can be found at "Links to UEFI-Related Documents" (*http://uefi.org/uefi)* under the heading "IANA Assigned Internet Protocol Numbers list".

AcceptAnyProtocol
Set to **TRUE** to receive all IPv4 packets that get through the receive filters. Set to **FALSE** to receive only the *DefaultProtocol* IPv4 packets that get through the receive filters. Ignored when *AcceptPromiscuous* is **TRUE**.

AcceptIcmpErrors
Set to **TRUE** to receive ICMP error report packets. Ignored when *AcceptPromiscuous* or *AcceptAnyProtocol* is **TRUE**.

AcceptBroadcast
Set to **TRUE** to receive broadcast IPv4 packets. Ignored when *AcceptPromiscuous* is **TRUE**. Set to **FALSE** to stop receiving broadcast IPv4 packets.

AcceptPromiscuous
Set to **TRUE** to receive all IPv4 packets that are sent to any hardware address or any protocol address. Set to **FALSE** to stop receiving all promiscuous IPv4 packets.

UseDefaultAddress
Set to **TRUE** to use the default IPv4 address and default routing table. If the default IPv4 address is not available yet, then the EFI IPv4 Protocol driver will use *EFI_IP4_CONFIG2_PROTOCOL* to retrieve the IPv4 address and subnet information. (This field can be set and changed only when the EFI IPv4 driver is transitioning from the stopped to the started states.)

StationAddress
The station IPv4 address that will be assigned to this EFI IPv4Protocol instance. The EFI IPv4 Protocol driver will deliver only incoming IPv4 packets whose destination matches this IPv4 address exactly. Address 0.0.0.0 is also accepted as a special case in which incoming packets destined to any station IP address are always delivered. When *EFI_IP4_CONFIG_DATA* is used in *Configure (),* it is ignored if *UseDefaultAddress* is **TRUE;** When *EFI_IP4_CONFIG_DATA* is used in *GetModeData (),* it contains the default address if *UseDefaultAddress* is **TRUE** and the default address has been acquired.

SubnetMask
The subnet address mask that is associated with the station address. When *EFI_IP4_CONFIG_DATA* is used

in *Configure ()*, it is ignored if *UseDefaultAddress* is **TRUE** ; When *EFI_IP4_CONFIG_DATA* is used in *GetModeData ()*, it contains the default subnet mask if *UseDefaultAddress* is **TRUE** and the default address has been acquired.

TypeOfService
TypeOfService field in transmitted IPv4 packets.

TimeToLive
TimeToLive field in transmitted IPv4 packets.

DoNotFragment
State of the *DoNotFragment* bit in transmitted IPv4 packets.

RawData
Set to **TRUE** to send and receive unformatted packets. The other IPv4 receive filters are still applied. Fragmentation is disabled for *RawData* mode. NOTE: Unformatted packets include the IP header and payload. The media header is appended automatically for outgoing packets by underlying network drivers.

ReceiveTimeout
The timer timeout value (number of microseconds) for the receive timeout event to be associated with each assembled packet. Zero means do not drop assembled packets.

TransmitTimeout
The timer timeout value (number of microseconds) for the transmit timeout event to be associated with each outgoing packet. Zero means do not drop outgoing packets.

The *EFI_IP4_CONFIG_DATA* structure is used to report and change IPv4 session parameters.

```
//*******************************************
// EFI_IP4_ROUTE_TABLE
//*******************************************
typedef struct {
  EFI_IPv4_ADDRESS        SubnetAddress;
  EFI_IPv4_ADDRESS        SubnetMask;
  EFI_IPv4_ADDRESS        GatewayAddress;
}   EFI_IP4_ROUTE_TABLE;
```

SubnetAddress
The subnet address to be routed.

SubnetMask
The subnet mask. If (*DestinationAddress* & *SubnetMask* == *SubnetAddress*), then the packet is to be directed to the *GatewayAddress*.

GatewayAddress
The IPv4 address of the gateway that redirects packets to this subnet. If the IPv4 address is 0.0.0.0, then packets to this subnet are not redirected.

EFI_IP4_ROUTE_TABLE is the entry structure that is used in routing tables.

```
//*******************************************
// EFI_IP4_ICMP_TYPE
//*******************************************
typedef struct {
  UINT8        Type;
  UINT8        Code;
}   EFI_IP4_ICMP_TYPE
```

Type
The type of ICMP message. See RFC 792 and RFC 950.

Code
The code of the ICMP message, which further describes the different ICMP message formats under the same *Type*. See RFC 792 and RFC 950.

EFI_IP4_ICMP_TYPE is used to describe those ICMP messages that are supported by this EFI IPv4 Protocol driver.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_INVALID_PARAMETER	*This* is **NULL**.
EFI_OUT_OF_RESOURCES	The required mode data could not be allocated.

28.3.6 EFI_IP4_PROTOCOL.Configure()

Summary

Assigns an IPv4 address and subnet mask to this EFI IPv4 Protocol driver instance.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP4_CONFIGURE) (
  IN EFI_IP4_PROTOCOL         *This,
  IN EFI_IP4_CONFIG_DATA      *IpConfigData OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_IP4_PROTOCOL* instance.

IpConfigData
Pointer to the EFI IPv4 Protocol configuration data structure. Type *EFI_IP4_CONFIG_DATA* is defined in *EFI_IP4_PROTOCOL .GetModeData()*.

Description

The *Configure()* function is used to set, change, or reset the operational parameters and filter settings for this EFI IPv4 Protocol instance. Until these parameters have been set, no network traffic can be sent or received by this instance. Once the parameters have been reset (by calling this function with *IpConfigData* set to **NULL**), no more traffic can be sent or received until these parameters have been set again. Each EFI IPv4 Protocol instance can be started and stopped independently of each other by enabling or disabling their receive filter settings with the *Configure()* function.

When *IpConfigData.UseDefaultAddress* is set to *FALSE,* the new station address will be appended as an alias address into the addresses list in the EFI IPv4 Protocol driver. While set to *TRUE, Configure()* will trigger the *EFI_IP4_CONFIG2_PROTOCOL* to retrieve the default IPv4 address if it is not available yet. Clients could frequently call *GetModeData()* to check the status to ensure that the default IPv4 address is ready.

If operational parameters are reset or changed, any pending transmit and receive requests will be cancelled. Their completion token status will be set to *EFI_ABORTED* and their events will be signaled.

Status Codes Returned

EFI_SUCCESS	The driver instance was successfully opened.
EFI_NO_MAPPING	When using the default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_IP_ADDRESS_CONFLICT	There is an address conflict in response to the Arp invocation
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE:** • *This* is **NULL**. • *IpConfigData.StationAddress* is not a unicast IPv4 address. • *IpConfigData.SubnetMask* is not a valid IPv4 subnet mask.
EFI_UNSUPPORTED	One or more of the following conditions is **TRUE:** • A configuration protocol (DHCP, BOOTP, RARP, etc.) could not be located when clients choose to use the default IPv4 address. This EFI IPv4 Protocol implementation does not support this requested filter or timeout setting.
EFI_OUT_OF_RESOURCES	The EFI IPv4 Protocol driver instance data could not be allocated.
EFI_ALREADY_STARTED	The interface is already open and must be stopped before the IPv4 address or subnet mask can be changed. The interface must also be stopped when switching to/from raw packet mode.
EFI_DEVICE_ERROR	An unexpected system or network error occurred. The EFI IPv4 Protocol driver instance is not opened.

28.3.7 EFI_IP4_PROTOCOL.Groups()

Summary

Joins and leaves multicast groups.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP4_GROUPS) (
  IN EFI_IP4_PROTOCOL      *This,
  IN BOOLEAN               JoinFlag,
  IN EFI_IPv4_ADDRESS      *GroupAddress OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_IP4_PROTOCOL* instance.

JoinFlag
Set to **TRUE** to join the multicast group session and **FALSE** to leave.

GroupAddress
Pointer to the IPv4 multicast address.

Description

The *Groups()* function is used to join and leave multicast group sessions. Joining a group will enable reception of matching multicast packets. Leaving a group will disable the multicast packet reception.

If *JoinFlag* is **FALSE** and *GroupAddress* is **NULL,** all joined groups will be left.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_INVALID_PARAMETER	One or more of the following is **TRUE:** • *This* is **NULL**. • *JoinFlag* is **TRUE** and *GroupAddress* is **NULL**. • *GroupAddress* is not **NULL** and ** GroupAddress* is not a multicast IPv4 address.
EFI_NOT_STARTED	This instance has not been started.
EFI_NO_MAPPING	When using the default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_OUT_OF_RESOURCES	System resources could not be allocated.
EFI_UNSUPPORTED	This EFI IPv4 Protocol implementation does not support multicast groups.
EFI_ALREADY_STARTED	The group address is already in the group table (when *JoinFlag* is **TRUE**).
EFI_NOT_FOUND	The group address is not in the group table (when *JoinFlag* is **FALSE**).
EFI_DEVICE_ERROR	An unexpected system or network error occurred.

28.3.8 EFI_IP4_PROTOCOL.Routes()

Summary

Adds and deletes routing table entries.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP4_ROUTES) (
  IN EFI_IP4_PROTOCOL      *This,
  IN BOOLEAN               DeleteRoute,
  IN EFI_IPv4_ADDRESS      *SubnetAddress,
  IN EFI_IPv4_ADDRESS      *SubnetMask,
  IN EFI_IPv4_ADDRESS      *GatewayAddress
  );
```

Parameters

This
Pointer to the *EFI_IP4_PROTOCOL* instance.

DeleteRoute
Set to **TRUE** to delete this route from the routing table. Set to **FALSE** to add this route to the routing table. *SubnetAddress* and *SubnetMask* are used as the key to each route entry.

SubnetAddress
The address of the subnet that needs to be routed.

SubnetMask
The subnet mask of *SubnetAddress*.

GatewayAddress
The unicast gateway IPv4 address for this route.

Description

The *Routes()* function adds a route to or deletes a route from the routing table.

Routes are determined by comparing the *SubnetAddress* with the destination IPv4 address arithmetically *AND* -ed with the *SubnetMask*. The gateway address must be on the same subnet as the configured station address.

The default route is added with *SubnetAddress* and *SubnetMask* both set to 0.0.0.0. The default route matches all destination IPv4 addresses that do not match any other routes.

A *GatewayAddress* that is zero is a nonroute. Packets are sent to the destination IP address if it can be found in the ARP cache or on the local subnet. One automatic nonroute entry will be inserted into the routing table for outgoing packets that are addressed to a local subnet (gateway address of 0.0.0.0).

Each EFI IPv4 Protocol instance has its own independent routing table. Those EFI IPv4 Protocol instances that use the default IPv4 address will also have copies of the routing table that was provided by the *EFI_IP4_CONFIG2_PROTOCOL,* and these copies will be updated whenever the EIF IPv4 Protocol driver reconfigures its instances. As a result, client modification to the routing table will be lost.

NOTE: *There is no way to set up routes to other network interface cards because each network interface card has its own independent network stack that shares information only through* EFI IPv4 variable.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_NOT_STARTED	The driver instance has not been started.
EFI_NO_MAPPING	When using the default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE:** • *This* is **NULL**. • *SubnetAddress* is **NULL**. • *SubnetMask* is **NULL**. • *GatewayAddress* is **NULL**. • * *SubnetAddress* is not a valid subnet address. • * *SubnetMask* is not a valid subnet mask. • * *GatewayAddress* is not a valid unicast IPv4 address.
EFI_OUT_OF_RESOURCES	Could not add the entry to the routing table.
EFI_NOT_FOUND	This route is not in the routing table (when *DeleteRoute* is **TRUE**).
EFI_ACCESS_DENIED	The route is already defined in the routing table (when *DeleteRoute* is **FALSE**).

28.3.9 EFI_IP4_PROTOCOL.Transmit()

Summary

Places outgoing data packets into the transmit queue.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP4_TRANSMIT) (
  IN EFI_IP4_PROTOCOL            *This,
  IN EFI_IP4_COMPLETION_TOKEN    *Token
  );
```

Parameters

This
Pointer to the *EFI_IP4_PROTOCOL* instance.

Token
Pointer to the transmit token. Type *EFI_IP4_COMPLETION_TOKEN* is defined in "Related Definitions" below.

Description

The *Transmit()* function places a sending request in the transmit queue of this EFI IPv4 Protocol instance. Whenever the packet in the token is sent out or some errors occur, the event in the token will be signaled and the status is updated.

Related Definition

```
//*******************************************
// EFI_IP4_COMPLETION_TOKEN
//*******************************************
typedef struct {
  EFI_EVENT                  Event;
  EFI_STATUS                 Status;
  union {
    EFI_IP4_RECEIVE_DATA     *RxData;
    EFI_IP4_TRANSMIT_DATA    *TxData;
  }              Packet;
} EFI_IP4_COMPLETION_TOKEN;
```

Event
This *Event* will be signaled after the *Status* field is updated by the EFI IPv4 Protocol driver. The type of *Event* must be EFI_NOTIFY_SIGNAL. The Task Priority Level (TPL) of *Event* must be lower than or equal to TPL_CALLBACK.

Status
Will be set to one of the following values:

EFI_SUCCESS. The receive or transmit completed successfully.

EFI_ABORTED. The receive or transmit was aborted.

EFI_TIMEOUT. The transmit timeout expired.

EFI_ICMP_ERROR. An ICMP error packet was received.

EFI_DEVICE_ERROR. An unexpected system or network error occurred.

EFI_NO_MEDIA. There was a media error

RxData
When this token is used for receiving, *RxData* is a pointer to the *EFI_IP4_RECEIVE_DATA*. Type *EFI_IP4_RECEIVE_DATA* is defined below.

TxData
When this token is used for transmitting, *TxData* is a pointer to the *EFI_IP4_TRANSMIT_DATA*. Type *EFI_IP4_TRANSMIT_DATA* is defined below.

EFI_IP4_COMPLETION_TOKEN structures are used for both transmit and receive operations.

When the structure is used for transmitting, the *Event* and *TxData* fields must be filled in by the EFI IPv4 Protocol client. After the transmit operation completes, EFI IPv4 Protocol updates the *Status* field and the *Event* is signaled.

When the structure is used for receiving, only the *Event* field must be filled in by the EFI IPv4 Protocol client. After a packet is received, the EFI IPv4 Protocol fills in the *RxData* and *Status* fields and the *Event* is signaled. If the packet is an ICMP error message, the *Status* is set to EFI_ICMP_ERROR, and the packet is delivered up as usual. The protocol from the IP head in the ICMP error message is used to de-multiplex the packet.

```
//*******************************************
// EFI_IP4_RECEIVE_DATA
//*******************************************
typedef struct {
  EFI_TIME              TimeStamp;
  EFI_EVENT             RecycleSignal;
  UINT32                HeaderLength;
  EFI_IP4_HEADER        *Header;
  UINT32                OptionsLength;
  VOID                  *Options;
  UINT32                DataLength;
  UINT32                FragmentCount;
  EFI_IP4_FRAGMENT_DATA FragmentTable[1];
}   EFI_IP4_RECEIVE_DATA;
```

TimeStamp
Time when the EFI IPv4 Protocol driver accepted the packet. *TimeStamp* is zero filled if receive timestamps are disabled or unsupported.

RecycleSignal
After this event is signaled, the receive data structure is released and must not be referenced.

HeaderLength
Length of the IPv4 packet header. Zero if *ConfigData.RawData* is **TRUE**.

Header
Pointer to the IPv4 packet header. If the IPv4 packet was fragmented, this argument is a pointer to the header in the first fragment. **NULL** if *ConfigData.RawData* is *TRUE*. Type *EFI_IP4_HEADER* is defined below.

OptionsLength
Length of the IPv4 packet header options. May be zero.

Options
Pointer to the IPv4 packet header options. If the IPv4 packet was fragmented, this argument is a pointer to the options in the first fragment. May be **NULL**.

DataLength
Sum of the lengths of IPv4 packet buffers in *FragmentTable*. May be zero.

FragmentCount
Number of IPv4 payload (or raw) fragments. If *ConfigData.RawData* is *TRUE,* this count is the number of raw IPv4 fragments received so far. May be zero.

FragmentTable
Array of payload (or raw) fragment lengths and buffer pointers. If *ConfigData.RawData* is *TRUE,* each buffer points to a raw IPv4 fragment and thus IPv4 header and options are included in each buffer. Otherwise, IPv4 headers and options are not included in these buffers. Type *EFI_IP4_FRAGMENT_DATA* is defined below.

The EFI IPv4 Protocol receive data structure is filled in when IPv4 packets have been assembled (or when raw packets have been received). In the case of IPv4 packet assembly, the individual packet fragments are only verified and are not reorganized into a single linear buffer.

The *FragmentTable* contains a sorted list of zero or more packet fragment descriptors. The referenced packet fragments may not be in contiguous memory locations.

```
//*******************************************
// EFI_IP4_HEADER
//*******************************************
#pragma pack(1)
typedef struct {
  UINT8              HeaderLength:4;
  UINT8              Version:4;
  UINT8              TypeOfService;
  UINT16             TotalLength;
  UINT16             Identification;
  UINT16             Fragmentation;
  UINT8              TimeToLive;
  UINT8              Protocol;
  UINT16             Checksum;
  EFI_IPv4_ADDRESS   SourceAddress;
  EFI_IPv4_ADDRESS   DestinationAddress;
}   EFI_IP4_HEADER;
#pragma pack()
```

The fields in the IPv4 header structure are defined in the Internet Protocol version 4 specification, which can be found at "Links to UEFI-Related Documents" (*http://uefi.org/uefi*) under the heading "Internet Protocol version 4 Specification".

```
//*******************************************
// EFI_IP4_FRAGMENT_DATA
//*******************************************
typedef struct {
  UINT32                FragmentLength;
  VOID                  *FragmentBuffer;
}   EFI_IP4_FRAGMENT_DATA;
```

FragmentLength
Length of fragment data. This field may not be set to zero.

FragmentBuffer

Pointer to fragment data. This field may not be set to **NULL**.

The *EFI_IP4_FRAGMENT_DATA* structure describes the location and length of the IPv4 packet fragment to transmit or that has been received.

```
//*******************************************
// EFI_IP4_TRANSMIT_DATA
//*******************************************
typedef struct {
  EFI_IPv4_ADDRESS         DestinationAddress;
  EFI_IP4_OVERRIDE_DATA    *OverrideData;
  UINT32                   OptionsLength;
  VOID                     *OptionsBuffer;
  UINT32                   TotalDataLength;
  UINT32                   FragmentCount;
  EFI_IP4_FRAGMENT_DATA    FragmentTable[1];
}   EFI_IP4_TRANSMIT_DATA;
```

DestinationAddress
The destination IPv4 address. Ignored if *RawData* is **TRUE**.

OverrideData
If not **NULL,** the IPv4 transmission control override data. Ignored if *RawData* is **TRUE**. Type *EFI_IP4_OVERRIDE_DATA* is defined below.

OptionsLength
Length of the IPv4 header options data. Must be zero if the IPv4 driver does not support IPv4 options. Ignored if *RawData* is **TRUE**.

OptionsBuffer
Pointer to the IPv4 header options data. Ignored if *OptionsLength* is zero. Ignored if *RawData* is **TRUE**.

TotalDataLength
Total length of the *FragmentTable* data to transmit.

FragmentCount
Number of entries in the fragment data table.

FragmentTable
Start of the fragment data table. Type *EFI_IP4_FRAGMENT_DATA* is defined above.

The *EFI_IP4_TRANSMIT_DATA* structure describes a possibly fragmented packet to be transmitted.

```
//*******************************************
// EFI_IP4_OVERRIDE_DATA
//*******************************************
typedef struct {
  EFI_IPv4_ADDRESS         SourceAddress;
  EFI_IPv4_ADDRESS         GatewayAddress;
  UINT8                    Protocol;
  UINT8                    TypeOfService;
  UINT8                    TimeToLive;
  BOOLEAN                  DoNotFragment;
}   EFI_IP4_OVERRIDE_DATA;
```

SourceAddress
Source address override.

GatewayAddress
Gateway address to override the one selected from the routing table. This address must be on the same subnet as this station address. If set to 0.0.0.0, the gateway address selected from routing table will not be overridden.

Protocol
Protocol type override.

TypeOfService
Type-of-service override.

TimeToLive
Time-to-live override.

DoNotFragment
Do-not-fragment override.

The information and flags in the override data structure will override default parameters or settings for one *Transmit()* function call.

Status Codes Returned

EFI_SUCCESS	The data has been queued for transmission.
EFI_NOT_STARTED	This instance has not been started.
EFI_NO_MAPPING	When using the default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_INVALID_PARAMETER	One or more of the following is **TRUE:** • *This* is **NULL**. • *Token* is **NULL**.. • *Token.Event* is **NULL**. • *Token.Packet.TxData* is **NULL**.. • *Token.Packet.TxData.OverrideData. GatewayAddress* in the override data structure is not a unicast IPv4 address if *OverrideData* is not **NULL**.. • *Token.Packet.TxData.OverrideData. SourceAddress* is not a unicast IPv4 address if *OverrideData* is not **NULL**. • *Token.Packet.OptionsLength* is not zero and *Token.Packet.OptionsBuffer* is **NULL**.. • *Token.Packet.FragmentCount* is zero. • One or more of the *Tok en.Packet.TxData.FragmentTable[]. FragmentLength* fields is zero. • One or more of the *Tok en.Packet.TxData.FragmentTable[]. FragmentBuffer* fields is **NULL**. • *Tok en.Packet.TxData.TotalDataLength* is zero or not equal to the sum of fragment lengths. • The IP header in *FragmentTable* is not a well-formed header when *RawData* is **TRUE**.
EFI_ACCESS_DENIED	The transmit completion token with the same *Token.Event* was already in the transmit queue.
EFI_NOT_READY	The completion token could not be queued because the transmit queue is full.
EFI_NOT_FOUND	Not route is found to destination address.
EFI_OUT_OF_RESOURCES	Could not queue the transmit data.

continues on next page

Table 28.24 – continued from previous page

EFI_BUFFER_TOO_SMALL	*Tok en.Packet.TxData.TotalDataLength* is too short to transmit.
EFI_BAD_BUFFER_SIZE	The length of the IPv4 header + option length + total data length is greater than MTU (or greater than the maximum packet size if * Token.Packet.TxData.OverrideData. DoNotFragment* is *TRUE*.)
EFI_NO_MEDIA	There was a media error.

28.3.10 EFI_IP4_PROTOCOL.Receive()

Summary

Places a receiving request into the receiving queue.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP4_RECEIVE) (
  IN EFI_IP4_PROTOCOL            *This,
  IN EFI_IP4_COMPLETION_TOKEN    *Token
  );
```

Parameters

This
Pointer to the *EFI_IP4_PROTOCOL* instance.

Token
Pointer to a token that is associated with the receive data descriptor. Type *EFI_IP4_COMPLETION_TOKEN* is defined in "Related Definitions" of above *Transmit()*.

Description

The *Receive()* function places a completion token into the receive packet queue. This function is always asynchronous.

The *Token.Event* field in the completion token must be filled in by the caller and cannot be **NULL**. When the receive operation completes, the EFI IPv4 Protocol driver updates the *Token.Status* and *Token.Packet.RxData* fields and the *Token.Event* is signaled.

Status Codes Returned

EFI_SUCCESS	The receive completion token was cached.
EFI_NOT_STARTED	This EFI IPv4 Protocol instance has not been started.
EFI_NO_MAPPING	When using the default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE:** • *This* is **NULL**. • *Token* is **NULL**. • *Token.Event* is **NULL**.
EFI_OUT_OF_RESOURCES	The receive completion token could not be queued due to a lack of system resources (usually memory).

continues on next page

Table 28.25 – continued from previous page

EFI_DEVICE_ERROR	An unexpected system or network error occurred. The EFI IPv4 Protocol instance has been reset to startup defaults.
EFI_ACCESS_DENIED	The receive completion token with the same *Token.Event* was already in the receive queue.
EFI_NOT_READY	The receive request could not be queued because the receive queue is full.
EFI_ICMP_ERROR	An ICMP error packet was received.
EFI_NO_MEDIA	There was a media error.

28.3.11 EFI_IP4_PROTOCOL.Cancel()

Summary

Abort an asynchronous transmit or receive request.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP4_CANCEL)(
  IN EFI_IP4_PROTOCOL            *This,
  IN EFI_IP4_COMPLETION_TOKEN    *Token OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_IP4_PROTOCOL* instance.

Token
Pointer to a token that has been issued by *EFI_IP4_PROTOCOL*.Transmit() or *EFI_IP4_PROTOCOL*.Receive(). If NULL, all pending tokens are aborted. Type *EFI_IP4_COMPLETION_TOKEN* is defined in *EFI_IP4_PROTOCOL.Transmit()*.

Description

The Cancel() function is used to abort a pending transmit or receive request. If the token is in the transmit or receive request queues, after calling this function, *Token->Status* will be set to *EFI_ABORTED* and then *Token->Event* will be signaled. If the token is not in one of the queues, which usually means the asynchronous operation has completed, this function will not signal the token and *EFI_NOT_FOUND* is returned.

Status Codes Returned

EFI_SUCCESS	The asynchronous I/O request was aborted and *Token.->Event* was signaled. When *Token* is **NULL,** all pending requests were aborted and their events were signaled.
EFI_INVALID_PARAMETER	*This* is **NULL**.
EFI_NOT_STARTED	This instance has not been started.
EFI_NO_MAPPING	When using the default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_NOT_FOUND	When *Token* is not **NULL,** the asynchronous I/O request was not found in the transmit or receive queue. It has either completed or was not issued by *Transmit()* and *Receive()*.

28.3.12 EFI_IP4_PROTOCOL.Poll()

Summary

Polls for incoming data packets and processes outgoing data packets.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP4_POLL) (
  IN EFI_IP4_PROTOCOL         *This
  );
```

Parameters

This
Pointer to the *EFI_IP4_PROTOCOL* instance.

Description

The *Poll()* function polls for incoming data packets and processes outgoing data packets. Network drivers and applications can call the *EFI_IP4_PROTOCOL .Poll()* function to increase the rate that data packets are moved between the communications device and the transmit and receive queues.

In some systems the periodic timer event may not poll the underlying communications device fast enough to transmit and/or receive all data packets without missing incoming packets or dropping outgoing packets. Drivers and applications that are experiencing packet loss should try calling the *EFI_IP4_PROTOCOL .Poll()* function more often.

Status Codes Returned

EFI_SUCCESS	Incoming or outgoing data was processed.
EFI_NOT_STARTED	This EFI IPv4 Protocol instance has not been started.
EFI_INVALID_PARAMETER	*This* is **NULL**.
EFI_DEVICE_ERROR	An unexpected system or network error occurred.
EFI_NOT_READY	No incoming or outgoing data is processed.
EFI_TIMEOUT	Data was dropped out of the transmit and/or receive queue. Consider increasing the polling rate.

28.4 EFI IPv4 Configuration II Protocol

This section provides a detailed description of the EFI IPv4 Configuration II Protocol.

28.4.1 EFI_IP4_CONFIG2_PROTOCOL

Summary

The *EFI_IP4_CONFIG2_PROTOCOL* provides the mechanism to set and get various types of configurations for the EFI IPv4 network stack.

GUID

```
#define EFI_IP4_CONFIG2_PROTOCOL_GUID \
  { 0x5b446ed1, 0xe30b, 0x4faa,\
    { 0x87, 0x1a, 0x36, 0x54, 0xec, 0xa3, 0x60, 0x80 }}
```

Protocol Interface Structure

```
typedef struct _EFI_IP4_CONFIG2_PROTOCOL {
  EFI_IP4_CONFIG2_SET_DATA             SetData;
  EFI_IP4_CONFIG2_GET_DATA             GetData;
  EFI_IP4_CONFIG2_REGISTER_NOTIFY      RegisterDataNotify;
  EFI_IP4_CONFIG2_UNREGISTER_NOTIFY    UnregisterDataNotify;
}   EFI_IP4_CONFIG2_PROTOCOL;
```

Parameters

SetData
Set the configuration for the EFI IPv4 network stack running on the communication device this EFI IPv4 Configuration II Protocol instance manages. See the *SetData()* function description.

GetData
Get the configuration for the EFI IPv4 network stack running on the communication device this EFI IPv4 Configuration II Protocol instance manages. See the *GetData()* function description.

RegiseterDataNotify
Register an event that is to be signaled whenever a configuration process on the specified configuration data is done.

UnregisterDataNotify
Remove a previously registered event for the specified configuration data.

Description

The *EFI_IP4_CONFIG2_PROTOCOL* is designed to be the central repository for the common configurations and the administrator configurable settings for the EFI IPv4 network stack.

An EFI IPv4 Configuration II Protocol instance will be installed on each communication device that the EFI IPv4 network stack runs on.

NOTE: *All the network addresses described in* EFI_IP4_CONFIG2_PROTOCOL *are stored in network byte order. All other parameters defined in functions or data structures are stored in host byte order.*

28.4.2 EFI_IP4_CONFIG2_PROTOCOL.SetData()

Summary

Set the configuration for the EFI IPv4 network stack running on the communication device this EFI IPv4 Configuration II Protocol instance manages.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP4_CONFIG2_SET_DATA) (
  IN EFI_IP4_CONFIG2_PROTOCOL    *This,
  IN EFI_IP4_CONFIG2_DATA_TYPE   DataType,
  IN UINTN                       DataSize,
  IN VOID                        *Data
  );
```

Parameters

This
Pointer to the *EFI_IP4_CONFIG2_PROTOCOL* instance.

DataType
The type of data to set. Type *EFI_IP4_CONFIG2_DATA_TYPE* is defined in "Related Definitions" below.

DataSize
Size of the buffer pointed to by *Data* in bytes.

Data
The data buffer to set. The type of the data buffer is associated with the *DataType*. The various types are defined in "Related Definitions" below.

Description

This function is used to set the configuration data of type DataType for the EFI IPv4 network stack running on the communication device this EFI IPv4 Configuration II Protocol instance manages. The successfully configured data is valid after system reset or power-off.

The DataSize is used to calculate the count of structure instances in the Data for some DataType that multiple structure instances are allowed.

This function is always non-blocking. When setting some type of configuration data, an asynchronous process is invoked to check the correctness of the data, such as doing address conflict detection on the manually set local IPv4 address. *EFI_NOT_READY* is returned immediately to indicate that such an asynchronous process is invoked and the process is not finished yet. The caller willing to get the result of the asynchronous process is required to call *RegisterDataNotify()* to register an event on the specified configuration data. Once the event is signaled, the caller can call *GetData()* to get back the configuration data in order to know the result. For other types of configuration data that do not require an asynchronous configuration process, the result of the operation is immediately returned.

Related Definition

```
//*******************************************
// EFI_IP4_CONFIG2_DATA_TYPE
//*******************************************
typedef enum {
  Ip4Config2DataTypeInterfaceInfo,
  Ip4Config2DataTypePolicy,
  Ip4Config2DataTypeManualAddress,
  Ip4Config2DataTypeGateway,
  Ip4Config2DataTypeDnsServer,
  Ip4Config2DataTypeMaximum
} EFI_IP4_CONFIG2_DATA_TYPE;
```

Ip4Config2DataTypeInterfaceInfo
The interface information of the communication device this EFI IPv4 Configuration II Protocol instance manages. This type of data is read only. The corresponding *Data* is of type *EFI_IP4_CONFIG2_INTERFACE_INFO*.

Ip4Config2DataTypePolicy
The general configuration policy for the EFI IPv4 network stack running on the communication device this EFI IPv4 Configuration II Protocol instance manages. The policy will affect other configuration settings. The corresponding *Data* is of type *EFI_IP4_CONFIG2_POLICY*.

Ip4Config2DataTypeManualAddress
The station addresses set manually for the EFI IPv4 network stack. It is only configurable when the policy is *Ip4Config2PolicyStatic*. The corresponding *Data* is of type *EFI_IP4_CONFIG2_MANUAL_ADDRESS*. When *DataSize* is 0 and *Data* is *NULL*, the existing configuration is cleared from the EFI IPv4 Configuration II Protocol instance.

Ip4Config2DataTypeGateway
The gateway addresses set manually for the EFI IPv4 network stack running on the communication device this EFI IPv4 Configuration II Protocol manages. It is not configurable when the policy is *Ip4Config2PolicyDhcp*.

The gateway addresses must be unicast IPv4 addresses. The corresponding *Data* is a pointer to an array of *EFI_IPv4_ADDRESS* instances. When *DataSize* is 0 and *Data* is *NULL,* the existing configuration is cleared from the EFI IPv4 Configuration II Protocol instance.

Ip4Config2DataTypeDnsServer
The DNS server list for the EFI IPv4 network stack running on the communication device this EFI IPv4 Configuration II Protocol manages. It is not configurable when the policy is *Ip4Config2PolicyDhcp*.The DNS server addresses must be unicast IPv4 addresses. The corresponding *Data* is a pointer to an array of *EFI_IPv4_ADDRESS* instances. When *DataSize* is 0 and *Data* is *NULL,* the existing configuration is cleared from the EFI IPv4 Configuration II Protocol instance.

```
//*******************************************************
// EFI_IP4_CONFIG2_INTERFACE_INFO related definitions
//*******************************************************
#define EFI_IP4_CONFIG2_INTERFACE_INFO_NAME_SIZE 32

//*******************************************************
// EFI_IP4_CONFIG2_INTERFACE_INFO
//*******************************************************
typedef struct {
  CHAR16                          Name[EFI_IP4_CONFIG2_INTERFACE_INFO_NAME_SIZE];
  UINT8                           IfType;
  UINT32                          HwAddressSize;
  EFI_MAC_ADDRESS                 HwAddress;
  EFI_IPv4_ADDRESS                StationAddress;
  EFI_IPv4_ADDRESS                SubnetMask;
  UINT32                          RouteTableSize;
  EFI_IP4_ROUTE_TABLE             *RouteTable OPTIONAL;
}   EFI_IP4_CONFIG2_INTERFACE_INFO;
```

Name
The name of the interface. It is a NULL-terminated Unicode string.

IfType
The interface type of the network interface. See RFC 1700, section “Number Hardware Type”.

HwAddressSize
The size, in bytes, of the network interface’s hardware address.

HwAddress
The hardware address for the network interface.

StationAddress
The station IPv4 address of this EFI IPv4 network stack.

SubnetMask
The subnet address mask that is associated with the station address.

RouteTableSize
Size of the following *RouteTable,* in bytes. May be zero.

RouteTable
The route table of the IPv4 network stack runs on this interface. Set to NULL if *RouteTableSize* is zero. Type *EFI_IP4_ROUTE_TABLE* is defined in *EFI_IP4_PROTOCOL.GetModeData()*.

The *EFI_IP4_CONFIG2_INTERFACE_INFO* structure describes the operational state of the interface this EFI IPv4 Configuration II Protocol instance manages. This type of data is read-only. When reading, the caller allocated buffer is used to return all of the data, i.e., the first part of the buffer is *EFI_IP4_CONFIG2_INTERFACE_INFO* and the

followings are the route table if present. The caller should NOT free the buffer pointed to by *RouteTable,* and the caller is only required to free the whole buffer if the data is not needed any more.

```
//*******************************************
// EFI_IP4_CONFIG2_POLICY
//*******************************************
typedef enum {
  Ip4Config2PolicyStatic,
  Ip4Config2PolicyDhcp,
  Ip4Config2PolicyMax
}   EFI_IP4_CONFIG2_POLICY;
```

Ip4Config2PolicyStatic
Under this policy, the *Ip4Config2DataTypeManualAddress, Ip4Config2DataTypeGateway* and *Ip4Config2DataTypeDnsServer* configuration data are required to be set manually. The EFI IPv4 Protocol will get all required configuration such as IPv4 address, subnet mask and gateway settings from the EFI IPv4 Configuration II protocol.

Ip4Config2PolicyDhcp
Under this policy, the *Ip4Config2DataTypeManualAddress, Ip4Config2DataTypeGateway* and *Ip4Config2DataTypeDnsServer* configuration data are not allowed to set via *SetData()*. All of these configurations are retrieved from DHCP server or other auto-configuration mechanism.

The *EFI_IP4_CONFIG2_POLICY* defines the general configuration policy the EFI IPv4 Configuration II Protocol supports. The default policy for a newly detected communication device is beyond the scope of this document. An implementation might leave it to platform to choose the default policy.

The configuration data of type *Ip4Config2DataTypeManualAddress, Ip4Config2DataTypeGateway* and *Ip4Config2DataTypeDnsServer* will be flushed if the policy is changed from *Ip4Config2PolicyStatic* to *Ip4Config2PolicyDhcp*.

```
//*******************************************
// EFI_IP4_CONFIG2_MANUAL_ADDRESS
//*******************************************
typedef struct {
  EFI_IPv4_ADDRESS       Address;
  EFI_IPv4_ADDRESS       SubnetMask;
}   EFI_IP4_CONFIG2_MANUAL_ADDRESS;
```

Address
The IPv4 unicast address.

SubnetMask
The subnet mask.

The *EFI_IP4_CONFIG2_MANUAL_ADDRESS* structure is used to set the station address information for the EFI IPv4 network stack manually when the policy is *Ip4Config2PolicyStatic*.

The *EFI_IP4_CONFIG2_DATA_TYPE* includes current supported data types; this specification allows future extension to support more data types.

Status Codes Returned

EFI_SUCCESS	The specified configuration data for the EFI IPv4 network stack is set successfully.

continues on next page

Table 28.28 – continued from previous page

EFI_INVALID_PARAMETER	One or more of the following are **TRUE**: • *This* is **NULL.** • One or more fields in *Data* and *DataSize* do not match the requirement of the data type indicated by *DataType*.
EFI_WRITE_PROTECTED	The specified configuration data is read-only or the specified configuration data can not be set under the current policy.
EFI_ACCESS_DENIED	Another set operation on the specified configuration data is already in process.
EFI_NOT_READY	An asynchronous process is invoked to set the specified configuration data and the process is not finished yet.
EFI_BAD_BUFFER_SIZE	The *DataSize* does not match the size of the type indicated by *DataType*.
EFI_UNSUPPORTED	This *DataType* is not supported.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.
EFI_DEVICE_ERROR	An unexpected system error or network error occurred.

28.4.3 EFI_IP4_CONFIG2_PROTOCOL.GetData()

Summary

Get the configuration data for the EFI IPv4 network stack running on the communication device this EFI IPv4 Configuration II Protocol instance manages.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP4_CONFIG2_GET_DATA) (
  IN EFI_IP4_CONFIG2_PROTOCOL    *This,
  IN EFI_IP4_CONFIG2_DATA_TYPE   DataType,
  IN OUT UINTN                   *DataSize,
  IN VOID                        *Data OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_IP4_CONFIG2_PROTOCOL* instance.

DataType
The type of data to get. Type *EFI_IP4_CONFIG2_DATA_TYPE* is defined in EFI_IP4_CONFIG2_PROTOCOL.SetData().

DataSize
On input, in bytes, the size of *Data*. On output, in bytes, the size of buffer required to store the specified configuration data.

Data
The data buffer in which the configuration data is returned. The type of the data buffer is associated with the *DataType*. Ignored if *DataSize* is 0. The various types are defined in *EFI_IP4_CONFIG2_PROTOCOL.SetData()*.

Description

This function returns the configuration data of type *DataType* for the EFI IPv4 network stack running on the communication device this EFI IPv4 Configuration II Protocol instance manages.

The caller is responsible for allocating the buffer used to return the specified configuration data and the required size will be returned to the caller if the size of the buffer is too small.

EFI_NOT_READY is returned if the specified configuration data is not ready due to an already in progress asynchronous configuration process. The caller can call *RegisterDataNotify()* to register an event on the specified configuration data. Once the asynchronous configuration process is finished, the event will be signaled and a subsequent *GetData()* call will return the specified configuration data.

Status Codes Returned

EFI_SUCCESS	The specified configuration data is got successfully.
EFI_INVALID_PARAMETER*	One or more of the followings are **TRUE:** • This is **NULL**. • DataSize is **NULL**. • *Data* is **NULL** if *DataSize* is not zero.
EFI_BUFFER_TOO_SMALL	The size of Data is too small for the specified configuration data and the required size is returned in *DataSize*.
EFI_NOT_READY	The specified configuration data is not ready due to an already in progress asynchronous configuration process.
EFI_NOT_FOUND	The specified configuration data is not found.

28.4.4 EFI_IP4_CONFIG2_PROTOCOL.RegisterDataNotify ()

Summary

Register an event that is to be signaled whenever a configuration process on the specified configuration data is done.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP4_CONFIG2_REGISTER_NOTIFY) (
  IN EFI_IP4_CONFIG2_PROTOCOL       *This,
  IN EFI_IP4_CONFIG2_DATA_TYPE      DataType,
IN EFI_EVENT                        Event
);
```

Parameters

This
Pointer to the *EFI_IP4_CONFIG2_PROTOCOL* instance.

DataType
The type of data to unregister the event for. Type *EFI_IP4_CONFIG2_DATA_TYPE* is defined in EFI_IP4_CONFIG2_PROTOCOL.SetData().

Event
The event to register.

Description

This function registers an event that is to be signaled whenever a configuration process on the specified configuration data is done. An event can be registered for different *DataType* simultaneously and the caller is responsible for determining which type of configuration data causes the signaling of the event in such case.

Status Codes Returned

EFI_SUCCESS	The notification event for the specified configuration data is registered.
EFI_INVALID_PARAMETER	*This* is **NULL** or Event is **NULL**.
EFI_UNSUPPORTED	The configuration data type specified by *DataType* is not supported.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.
EFI_ACCESS_DENIED	The *Event* is already registered for the *DataType*.

28.4.5 EFI_IP4_CONFIG2_PROTOCOL.UnregisterDataNotify ()

Summary

Remove a previously registered event for the specified configuration data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP4_CONFIG2_UNREGISTER_NOTIFY) (
  IN EFI_IP4_CONFIG2_PROTOCOL          *This,
  IN EFI_IP4_CONFIG2_DATA_TYPE         DataType,
  IN EFI_EVENT                         Event
  );
```

Parameters

This
Pointer to the *EFI_IP4_CONFIG2_PROTOCOL* instance.

DataType
The type of data to remove the previously registered event for. Type *EFI_IP4_CONFIG2_DATA_TYPE* is defined in EFI_IP4_CONFIG2_PROTOCOL.SetData().

Event
The event to unregister.

Description

This function removes a previously registered event for the specified configuration data.

Status Codes Returned

EFI_SUCCESS	The event registered for the specified configuration data is removed.
EFI_INVALID_PARAMETER	*This* is **NULL** or *Event* is **NULL**.
EFI_NOT_FOUND	The *Event* has not been registered for the specified *DataType*.

28.5 EFI IPv6 Protocol

This section defines the EFI IPv6 (Internet Protocol version 6) Protocol interface. It is split into the following three main sections:

- EFI IPv6 Service Binding Protocol
- EFI IPv6 Variable
- EFI IPv6 Protocol

The EFI IPv6 Protocol provides basic network IPv6 packet I/O services, which includes support for Neighbor Discovery Protocol (ND), Multicast Listener Discovery Protocol (MLD), and a subset of the Internet Control Message Protocol (ICMPv6).

28.5.1 IPv6 Service Binding Protocol

28.5.2 EFI_IP6_SERVICE_BINDING_PROTOCOL

Summary

The EFI IPv6 Service Binding Protocol is used to locate communication devices that are supported by an EFI IPv6 Protocol driver and to create and destroy EFI IPv6 Protocol child instances of the IP6 driver that can use the underlying communications device.

GUID

```
#define EFI_IP6_SERVICE_BINDING_PROTOCOL _GUID \
    {0xec835dd3,0xfe0f,0x617b,\
  {0xa6,0x21,0xb3,0x50,0xc3,0xe1,0x33,0x88}}
```

Description

A network application that requires basic IPv6 I/O services can use one of the protocol handler services, such as *BS->LocateHandleBuffer()*, to search for devices that publish an EFI IPv6 Service Binding Protocol GUID. Each device with a published EFI IPv6 Service Binding Protocol GUID supports the EFI IPv6 Protocol and may be available for use.

After a successful call to the *EFI_IP6_SERVICE_BINDING_PROTOCOL.CreateChild()* function, the newly created child EFI IPv6 Protocol driver is in an un-configured state; it is not ready to send and receive data packets.

Before a network application terminates execution, every successful call to the *EFI_IP6_SERVICE_BINDING_PROTOCOL.CreateChild()* function must be matched with a call to the *EFI_IP6_SERVICE_BINDING_PROTOCOL.DestroyChild()* function.

28.5.3 IPv6 Protocol

28.5.4 EFI_IP6_PROTOCOL

Summary

The EFI IPv6 Protocol implements a simple packet-oriented interface that can be used by drivers, daemons, and applications to transmit and receive network packets.

GUID

```
#define EFI_IP6_PROTOCOL_GUID \
  {0x2c8759d5,0x5c2d,0x66ef,\
    {0x92,0x5f,0xb6,0x6c,0x10,0x19,0x57,0xe2}}
```

Protocol Interface Structure

```
typedef struct _EFI_IP6_PROTOCOL {
  EFI_IP6_GET_MODE_DATA    GetModeData;
  EFI_IP6_CONFIGURE        Configure;
  EFI_IP6_GROUPS           Groups;
  EFI_IP6_ROUTES           Routes;
  EFI_IP6_NEIGHBORS        Neighbors;
  EFI_IP6_TRANSMIT         Transmit;
  EFI_IP6_RECEIVE          Receive;
  EFI_IP6_CANCEL           Cancel;
  EFI_IP6_POLL             Poll;
}  EFI_IP6_PROTOCOL;
```

Parameters

GetModeData
Gets the current operational settings for this instance of the EFI IPv6 Protocol driver. See the *GetModeData()* function description.

Configure
Changes or resets the operational settings for the EFI IPv6 Protocol. See the *Configure()* function description.

Groups
Joins and leaves multicast groups. See the *Groups()* function description.

Routes
Adds and deletes routing table entries. See the *Routes()* function description.

Neighbors
Adds and deletes neighbor cache entries. See the *Neighbors()* function description.

Transmit
Places outgoing data packets into the transmit queue. See the *Transmit()* function description.

Receive
Places a receiving request into the receiving queue. See the *Receive()* function description.

Cancel
Aborts a pending transmit or receive request. See the *Cancel()* function description.

Poll
Polls for incoming data packets and processes outgoing data packets. See the *Poll()* function description.

Description

The *EFI_IP6_PROTOCOL* defines a set of simple IPv6, and ICMPv6 services that can be used by any network protocol driver, daemon, or application to transmit and receive IPv6 data packets.

NOTE: *Byte Order: All the IPv6 addresses that are described in* EFI_IP6_PROTOCOL *are stored in network byte order. Both incoming and outgoing IP packets are also in network byte order. All other parameters that are defined in functions or data structures are stored in host byte order.*

28.5.5 EFI_IP6_PROTOCOL.GetModeData()

Summary

Gets the current operational settings for this instance of the EFI IPv6 Protocol driver.

Prototype

```
EFI_STATUS
(EFIAPI *EFI_IP6_GET_MODE_DATA) (
  IN EFI_IP6_PROTOCOL                    *This,
  OUT EFI_IP6_MODE_DATA                  *Ip6ModeData OPTIONAL,
  OUT EFI_MANAGED_NETWORK_CONFIG_DATA    *MnpConfigData OPTIONAL,
  OUT EFI_SIMPLE_NETWORK_MODE            *SnpModeData OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_IP6_PROTOCOL* instance.

Ip6ModeData
Pointer to the EFI IPv6 Protocol mode data structure. Type *EFI_IP6_MODE_DATA* is defined in "Related Definitions" below.

MnpConfigData
Pointer to the managed network configuration data structure. Type *EFI_MANAGED_NETWORK_CONFIG_DATA* is defined in *EFI_MANAGED_NETWORK_PROTOCOL.GetModeData()*.

SnpData
Pointer to the simple network mode data structure. Type *EFI_SIMPLE_NETWORK_MODE* is defined in the *EFI_SIMPLE_NETWORK_PROTOCOL*.

Description

The *GetModeData()* function returns the current operational mode data for this driver instance. The data fields in *EFI_IP6_MODE_DATA* are read only. This function is used optionally to retrieve the operational mode data of underlying networks or drivers.

Related Definition

```
//**************************************
// EFI_IP6_MODE_DATA
//**************************************
typedef struct {
  BOOLEAN                  IsStarted;
  UINT32                   MaxPacketSize;
  EFI_IP6_CONFIG_DATA      ConfigData;
  BOOLEAN                  IsConfigured;
```

(continues on next page)

(continued from previous page)

```
  UINT32                    AddressCount;
  EFI_IP6_ADDRESS_INFO      *AddressList;
  UINT32                    GroupCount;
  EFI_IPv6_ADDRESS          *GroupTable;
  UINT32                    RouteCount;
  EFI_IP6_ROUTE_TABLE       *RouteTable;
  UINT32                    NeighborCount;
  EFI_IP6_NEIGHBOR_CACHE    *NeighborCache;
  UINT32                    PrefixCount;
  EFI_IP6_ADDRESS_INFO      *PrefixTable;
  UINT32                    IcmpTypeCount;
  EFI_IP6_ICMP_TYPE         *IcmpTypeList;
}   EFI_IP6_MODE_DATA;
```

IsStarted
Set to **TRUE** after this EFI IPv6 Protocol instance is started. All other fields in this structure are undefined until this field is **TRUE**. Set to **FALSE** when the EFI IPv6 Protocol instance is stopped.

MaxPackeSize
The maximum packet size, in bytes, of the packet which the upper layer driver could feed.

ConfigData
Current configuration settings. Undefined until *IsStarted* is **TRUE**. Type *EFI_IP6_CONFIG_DATA* is defined below.

IsConfigured
Set to **TRUE** when the EFI IPv6 Protocol instance is configured. The instance is configured when it has a station address and corresponding prefix length. Set to **FALSE** when the EFI IPv6 Protocol instance is not configured.

AddressCount
Number of configured IPv6 addresses on this interface.

AddressList
List of currently configured IPv6 addresses and corresponding prefix lengths assigned to this interface. It is caller's responsibility to free this buffer. Type *EFI_IP6_ADDRESS_INFO* is defined below.

GroupCount
Number of joined multicast groups. Undefined until *IsConfigured* is **TRUE**.

GroupTable
List of joined multicast group addresses. It is caller's responsibility to free this buffer. Undefined until *IsConfigured* is **TRUE**.

RouteCount
Number of entries in the routing table. Undefined until *IsConfigured* is **TRUE**.

RouteTable
Routing table entries. It is caller's responsibility to free this buffer. Type *EFI_IP6_ROUTE_TABLE* is defined below.

NeighborCount
Number of entries in the neighbor cache. Undefined until *IsConfigured* is **TRUE**.

NeighborCache
Neighbor cache entries. It is caller's responsibility to free this buffer. Undefined until *IsConfigured* is **TRUE**. Type *EFI_IP6_NEIGHBOR_CACHE* is defined below.

PrefixCount
Number of entries in the prefix table. Undefined until *IsConfigured* is **TRUE**.

PrefixTable
On-link Prefix table entries. It is caller's responsibility to free this buffer. Undefined until *IsConfigured* is **TRUE**. Type *EFI_IP6_ADDRESS_INFO* is defined below.

IcmpTypeCount
Number of entries in the supported ICMP types list.

IcmpTypeList
Array of ICMP types and codes that are supported by this EFI IPv6 Protocol driver. It is caller's responsibility to free this buffer. Type *EFI_IP6_ICMP_TYPE* is defined below.

```
//*****************************************
// EFI_IP6_CONFIG_DATA
//*****************************************
typedef struct {
  UINT8              DefaultProtocol;
  BOOLEAN            AcceptAnyProtocol;
  BOOLEAN            AcceptIcmpErrors;
  BOOLEAN            AcceptPromiscuous;
  EFI_IPv6_ADDRESS   DestinationAddress;
  EFI_IPv6_ADDRESS   StationAddress;
  UINT8              TrafficClass;
  UINT8              HopLimit;
  UINT32             FlowLabel;
  UINT32             ReceiveTimeout;
  UINT32             TransmitTimeout;
} EFI_IP6_CONFIG_DATA;
```

DefaultProtocol
For the IPv6 packet to send and receive, this is the default value of the 'Next Header' field in the last IPv6 extension header or in the IPv6 header if there are no extension headers. Ignored when *AcceptPromiscuous* is **TRUE**. An updated list of protocol numbers can be found at "Links to UEFI-Related Documents" (*http://uefi.org/uefi)* under the heading "IANA Assigned Internet Protocol Numbers". The following values are illegal: 0 (IPv6 Hop-by-Hop Option), 1(ICMP), 2(IGMP), 41(IPv6), 43(Routing Header for IPv6), 44(Fragment Header for IPv6), 59(No Next Header for IPv6), 60(Destination Options for IPv6), 124(ISIS over IPv4).

AcceptAnyProtocol
Set to **TRUE** to receive all IPv6 packets that get through the receive filters. Set to **FALSE** to receive only the *DefaultProtocol* IPv6 packets that get through the receive filters. Ignored when *AcceptPromiscuous* is **TRUE**.

AcceptIcmpErrors
Set to **TRUE** to receive ICMP error report packets. Ignored when *AcceptPromiscuous* or *AcceptAnyProtocol* is **TRUE**.

AcceptPromiscuous
Set to **TRUE** to receive all IPv6 packets that are sent to any hardware address or any protocol address. Set to **FALSE** to stop receiving all promiscuous IPv6 packets.

DestinationAddress
The destination address of the packets that will be transmitted. Ignored if it is unspecified.

StationAddress
The station IPv6 address that will be assigned to this EFI IPv6 Protocol instance. This field can be set and changed only when the EFI IPv6 driver is transitioning from the stopped to the started states. If the StationAddress is specified, the EFI IPv6 Protocol driver will deliver only incoming IPv6 packets whose destination matches this IPv6 address exactly. The StationAddress is required to be one of currently configured IPv6 addresses. An address containing all zeroes is also accepted as a special case. Under this situation, the IPv6 driver is responsible for binding a source address to this EFI IPv6 protocol instance according to the source address

selection algorithm. Only incoming packets destined to the selected address will be delivered to the user. And the selected station address can be retrieved through later *GetModeData()* call. If no address is available for selecting, *EFI_NO_MAPPING* will be returned, and the station address will only be successfully bound to this EFI IPv6 protocol instance after *IP6ModeData.IsConfigured* changed to **TRUE**.

TrafficClass
TrafficClass field in transmitted IPv6 packets. Default value is zero.

HopLimit
HopLimit field in transmitted IPv6 packets.

FlowLabel
FlowLabel field in transmitted IPv6 packets. Default value is zero.

ReceiveTimeout
The timer timeout value (number of microseconds) for the receive timeout event to be associated with each assembled packet. Zero means do not drop assembled packets.

TransmitTimeout
The timer timeout value (number of microseconds) for the transmit timeout event to be associated with each outgoing packet. Zero means do not drop outgoing packets.

The EFI_IP6_CONFIG_DATA structure is used to report and change IPv6 session parameters.

```
//*******************************************
// EFI_IP6_ADDRESS_INFO
//*******************************************
typedef struct {
  EFI_IPv6_ADDRESS      Address;
  UINT8                 PrefixLength;
}   EFI_IP6_ADDRESS_INFO;
```

Address
The IPv6 address.

PrefixLength
The length of the prefix associated with the *Address*.

```
//*******************************************
// EFI_IP6_ROUTE_TABLE
//*******************************************
typedef struct {
  EFI_IPv6_ADDRESS      Gateway;
  EFI_IPv6_ADDRESS      Destination;
  UINT8                 PrefixLength;
}   EFI_IP6_ROUTE_TABLE;
```

Gateway
The IPv6 address of the gateway to be used as the next hop for packets to this prefix. If the IPv6 address is all zeros, then the prefix is on-link.

Destination
The destination prefix to be routed.

PrefixLength
The length of the prefix associated with the *Destination*.

EFI_IP6_ROUTE_TABLE is the entry structure that is used in routing tables.

```
//*******************************************
// EFI_IP6_NEIGHBOR_CACHE
//*******************************************
typedef struct {
  EFI_IPv6_ADDRESS         Neighbor;
  EFI_MAC_ADDRESS          LinkAddress;
  EFI_IP6_NEIGHBOR_STATE   State;
}   EFI_IP6_NEIGHBOR_CACHE;
```

Neighbor
The on-link unicast / anycast IP address of the neighbor.

LinkAddress
Link-layer address of the neighbor.

State
State of this neighbor cache entry.

EFI_IP6_NEIGHBOR_CACHE is the entry structure that is used in neighbor cache. It records a set of entries about individual neighbors to which traffic has been sent recently.

```
//*******************************************
// EFI_IP6_NEIGHBOR_STATE
//*******************************************
typedef enum {
  EfiNeighborInComplete,
  EfiNeighborReachable,
  EfiNeighborStale,
  EfiNeighborDelay,
  EfiNeighborProbe
}   EFI_IP6_NEIGHBOR_STATE;
```

Following is a description of the fields in the above enumeration.

EfiNeighborInComplete
Address resolution is being performed on this entry. Specially, Neighbor Solicitation has been sent to the solicited-node multicast address of the target, but corresponding Neighbor Advertisement has not been received.

EfiNeighborReachable
Positive confirmation was received that the forward path to the neighbor was functioning properly.

EfiNeighborStale
Reachable Time has elapsed since the last positive confirmation was received. In this state, the forward path to the neighbor was functioning properly.

EfiNeighborDelay
This state is an optimization that gives upper-layer protocols additional time to provide reachability confirmation.

EfiNeighborProbe
A reachability confirmation is actively sought by retransmitting Neighbor Solicitations every RetransTimer milliseconds until a reachability confirmation is received.

```
//*******************************************
// EFI_IP6_ICMP_TYPE
//*******************************************
typedef struct {
  UINT8       Type;
```

(continues on next page)

(continued from previous page)

```
  UINT8       Code;
}   EFI_IP6_ICMP_TYPE;
```

Type
The type of ICMP message. See "Links to UEFI-Related Documents" (*http://uefi.org/uefi)* under the heading "Internet Control Message Protocol Version 6 (ICMPv6) Parameters" for the complete list of ICMP message type.

Code
The code of the ICMP message, which further describes the different ICMP message formats under the same Type. See "Links to UEFI-Related Documents" (*http://uefi.org/uefi)* under the heading "Internet Control Message Protocol Version 6 (ICMPv6) Parameters" for details for code of ICMP message.

EFI_IP6_ICMP_TYPE is used to describe those ICMP messages that are supported by this EFI IPv6 Protocol driver.

```
//*******************************************************
// ICMPv6 type definitions for error messages
//*******************************************************
#define ICMP_V6_DEST_UNREACHABLE    0x1
#define ICMP_V6_PACKET_TOO_BIG      0x2
#define ICMP_V6_TIME_EXCEEDED       0x3
#define ICMP_V6_PARAMETER_PROBLEM   0x4
```

```
//*******************************************************
// ICMPv6 type definition for informational messages
//*******************************************************
#define ICMP_V6_ECHO_REQUEST        0x80
#define ICMP_V6_ECHO_REPLY          0x81
#define ICMP_V6_LISTENER_QUERY      0x82
#define ICMP_V6_LISTENER_REPORT     0x83
#define ICMP_V6_LISTENER_DONE       0x84
#define ICMP_V6_ROUTER_SOLICIT      0x85
#define ICMP_V6_ROUTER_ADVERTISE    0x86
#define ICMP_V6_NEIGHBOR_SOLICIT    0x87
#define ICMP_V6_NEIGHBOR_ADVERTISE  0x88
#define ICMP_V6_REDIRECT            0x89
#define ICMP_V6_LISTENER_REPORT_2   0x8F
```

```
//*******************************************************
// ICMPv6 code definitions for ICMP_V6_DEST_UNREACHABLE
//*******************************************************
#define ICMP_V6_NO_ROUTE_TO_DEST    0x0
#define ICMP_V6_COMM_PROHIBITED     0x1
#define ICMP_V6_BEYOND_SCOPE        0x2
#define ICMP_V6_ADDR_UNREACHABLE    0x3
#define ICMP_V6_PORT_UNREACHABLE    0x4
#define ICMP_V6_SOURCE_ADDR_FAILED  0x5
#define ICMP_V6_ROUTE_REJECTED      0x6
```

```
//*******************************************************
// ICMPv6 code definitions for ICMP_V6_TIME_EXCEEDED
//*******************************************************
```

(continues on next page)

(continued from previous page)

```
#define ICMP_V6_TIMEOUT_HOP_LIMIT   0x0
#define ICMP_V6_TIMEOUT_REASSEMBLE  0x1
```

```
//*******************************************************
// ICMPv6 code definitions for ICMP_V6_PARAMETER_PROBLEM
//*******************************************************
#define ICMP_V6_ERRONEOUS_HEADER      0x0
#define ICMP_V6_UNRECOGNIZE_NEXT_HDR  0x1
#define ICMP_V6_UNRECOGNIZE_OPTION    0x2
```

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_INVALID_PARAMETER	*This* is **NULL**
EFI_OUT_OF_RESOURCES	The required mode data could not be allocated.

28.5.6 EFI_IP6_PROTOCOL.Configure()

Summary

Assign IPv6 address and other configuration parameter to this EFI IPv6 Protocol driver instance.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP6_CONFIGURE) (
  IN EFI_IP6_PROTOCOL            *This,
  IN EFI_IP6_CONFIG_DATA         *Ip6ConfigData OPTIONAL
 );
```

Parameters

This
Pointer to the *EFI_IP6_PROTOCOL* instance.

Ip6ConfigData
Pointer to the EFI IPv6 Protocol configuration data structure. Type *EFI_IP6_CONFIG_DATA* is defined in *EFI_IP6_PROTOCOL.GetModeData()*.

Description

The *Configure()* function is used to set, change, or reset the operational parameters and filter settings for this EFI IPv6 Protocol instance. Until these parameters have been set, no network traffic can be sent or received by this instance. Once the parameters have been reset (by calling this function with *Ip6ConfigData* set to **NULL**), no more traffic can be sent or received until these parameters have been set again. Each EFI IPv6 Protocol instance can be started and stopped independently of each other by enabling or disabling their receive filter settings with the *Configure()* function.

If *Ip6ConfigData.StationAddress* is a valid non-zero IPv6 unicast address, it is required to be one of the currently configured IPv6 addresses list in the EFI IPv6 drivers, or else *EFI_INVALID_PARAMETER* will be returned. If *Ip6ConfigData.StationAddress* is unspecified, the IPv6 driver will bind a source address according to the source address selection algorithm. Clients could frequently call *GetModeData()* to check get currently configured IPv6 address list in the EFI IPv6 driver. If both *Ip6ConfigData.StationAddress* and *Ip6ConfigData.Destination* are unspecified, when

transmitting the packet afterwards, the source address filled in each outgoing IPv6 packet is decided based on the destination of this packet.

If operational parameters are reset or changed, any pending transmit and receive requests will be cancelled. Their completion token status will be set to *EFI_ABORTED* and their events will be signaled.

Status Codes Returned

EFI_SUCCESS	The driver instance was successfully opened.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE:** *This* is **NULL**. *Ip6ConfigData.StationAddress* is neither zero nor a unicast IPv6 address. *Ip6ConfigData.StationAddress* is neither zero nor one of the configured IP addresses in the EFI IPv6 driver. *Ip6ConfigData.DefaultProtocol* is illegal.
EFI_OUT_OF_RESOURCES	The EFI IPv6 Protocol driver instance data could not be allocated.
EFI_NO_MAPPING	The IPv6 driver was responsible for choosing a source address for this instance, but no source address was available for use.
EFI_ALREADY_STARTED	The interface is already open and must be stopped before the IPv6 address or prefix length can be changed.
EFI_DEVICE_ERROR	An unexpected system or network error occurred. The EFI IPv6 Protocol driver instance is not opened.
EFI_UNSUPPORTED	Default protocol specified through *Ip6ConfigData.DefaulProtocol* isn't supported.

28.5.7 EFI_IP6_PROTOCOL.Groups()

Summary

Joins and leaves multicast groups.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP6_GROUPS) (
  IN EFI_IP6_PROTOCOL      *This,
  IN BOOLEAN               JoinFlag,
  IN EFI_IPv6_ADDRESS      *GroupAddress OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_IP6_PROTOCOL* instance.

JoinFlag
Set to **TRUE** to join the multicast group session and **FALSE** to leave.

GroupAddress
Pointer to the IPv6 multicast address.

Description

The *Groups()* function is used to join and leave multicast group sessions. Joining a group will enable reception of matching multicast packets. Leaving a group will disable reception of matching multicast packets. Source-Specific Multicast isn't required to be supported.

If *JoinFlag* is **FALSE** and *GroupAddress* is **NULL,** all joined groups will be left.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_INVALID_PARAMETER	One or more of the following is **TRUE:** *This* is **NULL**. *JoinFlag* is **TRUE** and *GroupAddress* is **NULL** *GroupAddress* is not **NULL** and * *GroupAddress* is not a multicast IPv6 address. *GroupAddress* is not **NULL** and * *GroupAddress* is in the range of SSM destination address.
EFI_NOT_STARTED	This instance has not been started.
EFI_OUT_OF_RESOURCES	System resources could not be allocated.
EFI_UNSUPPORTED	This EFI IPv6 Protocol implementation does not support multicast groups.
EFI_ALREADY_STARTED	The group address is already in the group table (when *JoinFlag* is **TRUE**).
EFI_NOT_FOUND	The group address is not in the group table (when *JoinFlag* is **FALSE**).
EFI_DEVICE_ERROR	An unexpected system or network error occurred.

28.5.8 EFI_IP6_PROTOCOL.Routes()

Summary

Adds and deletes routing table entries.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP6_ROUTES) (
  IN EFI_IP6_PROTOCOL         *This,
  IN BOOLEAN                  DeleteRoute,
  IN EFI_IPv6_ADDRESS         *Destination OPTIONAL,
  IN UINT8                    PrefixLength,
  IN EFI_IPv6_ADDRESS         *GatewayAddress OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_IP6_PROTOCOL* instance.

DeleteRoute
Set to **TRUE** to delete this route from the routing table. Set to **FALSE** to add this route to the routing table. *Destination, PrefixLength* and *Gateway* are used as the key to each route entry.

Destination
The address prefix of the subnet that needs to be routed.

PrefixLength
The prefix length of *Destination*. Ignored if Destination is **NULL**.

GatewayAddress
The unicast gateway IPv6 address for this route.

Description

The *Routes()* function adds a route to or deletes a route from the routing table.

Routes are determined by comparing the leftmost *PrefixLength* bits of *Destination* with the destination IPv6 address arithmetically. The gateway address must be on the same subnet as the configured station address.

The default route is added with *Destination* and *PrefixLegth* both set to all zeros. The default route matches all destination IPv6 addresses that do not match any other routes.

All EFI IPv6 Protocol instances share a routing table.

NOTE: *There is no way to set up routes to other network interface cards because each network interface card has its own independent network stack that shares information only through the EFI IPv6 variable*.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_NOT_STARTED	The driver instance has not been started.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE:** *This* is **NULL**. When *DeleteRoute* is **TRUE**, both *Destination* and *GatewayAddress* are **NULL** When *DeleteRoute* is **FALSE,** either *Destination* or *GatewayAddress* is **NULL** * *GatewayAddress* is not a valid unicast IPv6 address. * *GatewayAddress* is one of the local configured IPv6 addresses.
EFI_OUT_OF_RESOURCES	Could not add the entry to the routing table.
EFI_NOT_FOUND	This route is not in the routing table (when *DeleteRoute* is **TRUE**).
EFI_ACCESS_DENIED	The route is already defined in the routing table (when *DeleteRoute* is **FALSE**).

28.5.9 EFI_IP6_PROTOCOL.Neighbors()

Summary

Add or delete Neighbor cache entries.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP6_NEIGHBORS) (
  IN EFI_IP6_PROTOCOL            *This,
  IN BOOLEAN                     DeleteFlag,
  IN EFI_IPv6_ADDRESS            *TargetIp6Address,
  IN EFI_MAC_ADDRESS             *TargetLinkAddress OPTIONAL
  IN UINT32                      Timeout,
  IN BOOLEAN                     Override
  );
```

Parameters

This
Pointer to the *EFI_IP6_PROTOCOL* instance.

DeleteFlag
Set to **TRUE** to delete the specified cache entry, set to **FALSE** to add (or update, if it already exists and *Override* is **TRUE**) the specified cache entry. *TargetIp6Address* is used as the key to find the requested cache entry.

TargetIp6Address
Pointer to Target IPv6 address.

TargetLinkAddress
Pointer to link-layer address of the target. Ignored if **NULL**.

Timeout
Time in 100-ns units that this entry will remain in the neighbor cache, it will be deleted after Timeout. A value of zero means that the entry is permanent. A non-zero value means that the entry is dynamic.

Override
If *TRUE,* the cached link-layer address of the matching entry will be overridden and updated; if *FALSE, EFI_ACCESS_DENIED* will be returned if a corresponding cache entry already existed.

Description

The *Neighbors()* function is used to add, update, or delete an entry from neighbor cache.

IPv6 neighbor cache entries are typically inserted and updated by the network protocol driver as network traffic is processed. Most neighbor cache entries will time out and be deleted if the network traffic stops. Neighbor cache entries that were inserted by *Neighbors()* may be static (will not timeout) or dynamic (will time out).

The implementation should follow the neighbor cache timeout mechanism which is defined in RFC4861. The default neighbor cache timeout value should be tuned for the expected network environment.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_NOT_STARTED	The driver instance has not been started.

continues on next page

Table 28.36 – continued from previous page

EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE:** This is **NULL**. *TargetIpAddress* is **NULL**. * *TargetLinkAddress* is invalid when not **NULL.** * *TargetIpAddress* is not a valid unicast IPv6 address. * *TargetIpAddress* is one of the local configured IPv6 addresses.
EFI_OUT_OF_RESOURCES	Could not add the entry to the neighbor cache.
EFI_NOT_FOUND	This entry is not in the neighbor cache (when *DeleteFlag* is **TRUE** or when *DeleteFlag* is **FALSE** while *TargetLinkAddress* is **NULL.**).
EFI_ACCESS_DENIED	The to-be-added entry is already defined in the neighbor cache, and that entry is tagged as un-overridden (when *DeleteFlag* is **FALSE**).

28.5.10 EFI_IP6_PROTOCOL.Transmit()

Summary

Places outgoing data packets into the transmit queue.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP6_TRANSMIT) (
  IN EFI_IP6_PROTOCOL            *This,
  IN EFI_IP6_COMPLETION_TOKEN    *Token
  );
```

Parameters

This
Pointer to the *EFI_IP6_PROTOCOL* instance.

Token
Pointer to the transmit token. Type *EFI_IP6_COMPLETION_TOKEN* is defined in "Related Definitions" below.

Description

The *Transmit( )* function places a sending request in the transmit queue of this EFI IPv6 Protocol instance. Whenever the packet in the token is sent out or some errors occur, the event in the token will be signaled and the status is updated.

Related Definition

```
//*************************************
// EFI_IP6_COMPLETION_TOKEN
//*************************************
typedef struct {
  EFI_EVENT                  Event;
  EFI_STATUS                 Status;
  union {
    EFI_IP6_RECEIVE_DATA     *RxData;
    EFI_IP6_TRANSMIT_DATA    *TxData;
```

(continues on next page)

(continued from previous page)

```
  }             Packet;
} EFI_IP6_COMPLETION_TOKEN;
```

Event
This *Event* will be signaled after the Status field is updated by the EFI IPv6 Protocol driver. The type of *Event* must be *EFI_NOTIFY_SIGNAL*.

Status
Will be set to one of the following values:

EFI_SUCCESS: The receive or transmit completed successfully.

EFI_ABORTED: The receive or transmit was aborted.

EFI_TIMEOUT: The transmit timeout expired.

EFI_ICMP_ERROR: An ICMP error packet was received.

EFI_DEVICE_ERROR: An unexpected system or network error occurred.

EFI_SECURITY_VIOLATION: The transmit or receive was failed because of an IPsec policy check.

RxData
When the Token is used for receiving, *RxData* is a pointer to the *EFI_IP6_RECEIVE_DATA*. Type *EFI_IP6_RECEIVE_DATA* is defined below.

TxData
When the Token is used for transmitting, *TxData* is a pointer to the *EFI_IP6_TRANSMIT_DATA*. Type *EFI_IP6_TRANSMIT_DATA* is defined below.

EFI_IP6_COMPLETION_TOKEN structures are used for both transmit and receive operations.

When the structure is used for transmitting, the *Event* and *TxData* fields must be filled in by the EFI IPv6 Protocol client. After the transmit operation completes, the EFI IPv6 Protocol driver updates the *Status* field and the *Event* is signaled.

When the structure is used for receiving, only the *Event* field must be filled in by the EFI IPv6 Protocol client. After a packet is received, the EFI IPv6 Protocol driver fills in the RxData and *Status* fields and the Event is signaled

```
//*******************************************
// EFI_IP6_RECEIVE_DATA
//*******************************************
typedef struct _EFI_IP6_RECEIVE_DATA {
  EFI_TIME                TimeStamp;
  EFI_EVENT               RecycleSignal;
  UINT32                  HeaderLength;
  EFI_IP6_HEADER          *Header;
  UINT32                  DataLength;
```

(continues on next page)

(continued from previous page)

```
  UINT32                   FragmentCount;
  EFI_IP6_FRAGMENT_DATA    FragmentTable[1];
}   EFI_IP6_RECEIVE_DATA;
```

TimeStamp
Time when the EFI IPv6 Protocol driver accepted the packet. *TimeStamp is zero filled if timestamps are disabled or unsupported.*

RecycleSignal
After this event is signaled, the receive data structure is released and must not be referenced.

HeaderLength
Length of the IPv6 packet headers, including both the IPv6 header and any extension headers.

Header
Pointer to the IPv6 packet header. If the IPv6 packet was fragmented, this argument is a pointer to the header in the first fragment. Type *EFI_IP6_HEADER* is defined below.

DataLength
Sum of the lengths of IPv6 packet buffers in FragmentTable. May be zero.

FragmentCount
Number of IPv6 payload fragments. May be zero.

FragmentTable
Array of payload fragment lengths and buffer pointers. Type *EFI_IP6_FRAGMENT_DATA* is defined below.

The EFI IPv6 Protocol receive data structure is filled in when IPv6 packets have been assembled. In the case of IPv6 packet assembly, the individual packet fragments are only verified and are not reorganized into a single linear buffer.

The *FragmentTable* contains a sorted list of zero or more packet fragment descriptors. The referenced packet fragments may not be in contiguous memory locations.

```
//*****************************************
// EFI_IP6_HEADER
//*****************************************
#pragma pack(1)
typedef struct _EFI_IP6_HEADER {
  UINT8                TrafficClassH:4;
  UINT8                Version:4;
  UINT8                FlowLabelH:4;
  UINT8                TrafficClassL:4;
  UINT16               FlowLabelL;
  UINT16               PayloadLength;
  UINT8                NextHeader;
  UINT8                HopLimit;
  EFI_IPv6_ADDRESS     SourceAddress;
  EFI_IPv6_ADDRESS     DestinationAddress;
} EFI_IP6_HEADER;
#pragma pack
```

The fields in the IPv6 header structure are defined in the Internet Protocol version6 specification, which can be found at “Links to UEFI-Related Documents” (*http://uefi.org/uefi)* under the heading “Internet Protocol version 6 Specification”.

```
//*******************************************
// EFI_IP6_FRAGMENT_DATA
//*******************************************
typedef struct _EFI_IP6_FRAGMENT_DATA {
  UINT32                    FragmentLength;
  VOID                      *FragmentBuffer;
}   EFI_IP6_FRAGMENT_DATA;
```

FragmentLength
Length of fragment data. This field may not be set to zero.

FragmentBuffer
Pointer to fragment data. This field may not be set to **NULL**.

The *EFI_IP6_FRAGMENT_DATA* structure describes the location and length of the IPv6 packet fragment to transmit or that has been received.

```
//*******************************************
// EFI_IP6_TRANSMIT_DATA
//*******************************************
typedef struct _EFI_IP6_TRANSMIT_DATA {
  EFI_IPv6_ADDRESS            DestinationAddress;
  EFI_IP6_OVERRIDE_DATA       *OverrideData;
  UINT32                      ExtHdrsLength;
  VOID                        *ExtHdrs;
  UINT8                       NextHeader;
  UINT32                      DataLength;
  UINT32                      FragmentCount
  EFI_IP6_FRAGMENT_DATA       FragmentTable[1];
}   EFI_IP6_TRANSMIT_DATA;
```

DestinationAddress
The destination IPv6 address. If it is unspecified, *ConfigData.DestinationAddress* will be used instead.

OverrideData
If not *NULL,* the IPv6 transmission control override data. Type *EFI_IP6_OVERRIDE_DATA* is defined below.

ExtHdrsLength
Total length in byte of the IPv6 extension headers specified in *ExtHdrs*

ExtHdrs
Pointer to the IPv6 extension headers. The IP layer will append the required extension headers if they are not specified by ExtHdrs. Ignored if *ExtHdrsLength* is zero.

NextHeader
The protocol of first extension header in *ExtHdrs*. Ignored if *ExtHdrsLength* is zero.

DataLength
Total length in bytes of the *FragmentTable* data to transmit.

FragmentCount
Number of entries in the fragment data table.

FragmentTable
Start of the fragment data table. Type *EFI_IP6_FRAGMENT_DATA* is defined above.

The *EFI_IP6_TRANSMIT_DATA* structure describes a possibly fragmented packet to be transmitted.

```
//*******************************************
// EFI_IP6_OVERRIDE_DATA
//*******************************************
typedef struct _EFI_IP6_OVERRIDE_DATA {
  UINT8        Protocol;
  UINT8        HopLimit;
  UINT32       FlowLabel;
}   EFI_IP6_OVERRIDE_DATA;
```

Protocol
Protocol type override.

HopLimit
Hop-Limit override.

FlowLabel
Flow-Label override.

The information and flags in the override data structure will override default parameters or settings for one *Transmit()* function call.

Status Codes Returned

EFI_SUCCESS	The data has been queued for transmission.
EFI_NOT_STARTED	This instance has not been started.
EFI_NO_MAPPING	The IPv6 driver was responsible for choosing a source address for this transmission, but no source address was available for use.
EFI_INVALID_PARAMETER	One or more of the following is **TRUE:** • *This* is **NULL**. • *Token* is **NULL**. • *Token.Event* is **NULL** • *Token.Packet.TxData* is **NULL**. • *Token.Packet.ExtHdrsLength* is not zero and *Token.Packet.ExtHdrs* is **NULL**. • *Token.Packet.FragmentCount* is zero. • One or more of the *Token.Packet.TxDat FragmentTable[].FragmentLength* fields is zero. • One or more of the *Token.Packet.TxDat FragmentTable[].FragmentBuffer* fields is **NULL**. • *Token.Packet.TxData.DataLength* is zero or not equal to the sum of fragment lengths. • *Token.Packet.TxData.DestinationAddress* is non-zero when *DestinationAddress* is configured as non-zero when doing *Configure()* for this EFI IPv6 protocol instance. • *Token. Packet.TxData.DestinationAddress* is unspecified when *DestinationAddress* is unspecified when doing *Configure()* for this EFI IPv6 protocol instance.
EFI_ACCESS_DENIED	The transmit completion token with the same *Token.Event* was already in the transmit queue.

continues on next page

Table 28.37 – continued from previous page

EFI_NOT_READY	The completion token could not be queued because the transmit queue is full.
EFI_NOT_FOUND	No route was found to destination address.
EFI_OUT_OF_RESOURCES	Could not queue the transmit data.
EFI_BUFFER_TOO_SMALL	*Token.Packet.TxData.DataLength* is too short to transmit.
EFI_BAD_BUFFER_SIZE	If *Token.Packet.TxData.DataLength* is beyond the maximum that which can be described through the Fragment Offset field in Fragment header when performing fragmentation.
EFI_DEVICE_ERROR	An unexpected system or network error occurred.
EFI_NO_MEDIA	There was a media error.

28.5.11 EFI_IP6_PROTOCOL.Receive()

Summary

Places a receiving request into the receiving queue.

Prototype

```
typedef
  EFI_STATUS
  (EFIAPI *EFI_IP6_RECEIVE) (
    IN EFI_IP6_PROTOCOL            *This,
    IN EFI_IP6_COMPLETION_TOKEN    *Token
    );
```

Parameters

This
Pointer to the *EFI_IP6_PROTOCOL* instance.

Token
Pointer to a token that is associated with the receive data descriptor. Type *EFI_IP6_COMPLETION_TOKEN* is defined in "Related Definitions" of above *Transmit()*.

Description

The *Receive()* function places a completion token into the receive packet queue. This function is always asynchronous.

The *Token.Event* field in the completion token must be filled in by the caller and cannot be **NULL**. When the receive operation completes, the EFI IPv6 Protocol driver updates the *Token.Status* and *Token.Packet.RxData* fields and the *Token.Event* is signaled.

Status Codes Returned

EFI_SUCCESS	The receive completion token was cached.
EFI_NOT_STARTED	This EFI IPv6 Protocol instance has not been started.
EFI_NO_MAPPING	When IP6 driver responsible for binding source address to this instance, while no source address is available for use.

continues on next page

Table 28.38 – continued from previous page

EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE:** *This* is **NULL**. *Token* is **NULL**. *Token.Event* is **NULL**.
EFI_OUT_OF_RESOURCES	The receive completion token could not be queued due to a lack of system resources (usually memory).
EFI_DEVICE_ERROR	An unexpected system or network error occurred. The EFI IPv6 Protocol instance has been reset to startup defaults.
EFI_ACCESS_DENIED	The receive completion token with the same *Token.Event* was already in the receive queue.
EFI_NOT_READY	The receive request could not be queued because the receive queue is full.
EFI_DEVICE_ERROR	An unexpected system or network error occurred.
EFI_NO_MEDIA	There was a media error.

28.5.12 EFI_IP6_PROTOCOL.Cancel()

Summary

Abort an asynchronous transmits or receive request.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP6_CANCEL)(
  IN EFI_IP6_PROTOCOL            *This,
  IN EFI_IP6_COMPLETION_TOKEN    *Token OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_IP6_PROTOCOL* instance.

Token
Pointer to a token that has been issued by *EFI_IP6_PROTOCOL.Transmit()* or *EFI_IP6_PROTOCOL.Receive()*. If *NULL,* all pending tokens are aborted. Type *EFI_IP6_COMPLETION_TOKEN* is defined in *EFI_IP6_PROTOCOL.Transmit()*.

Description

The *Cancel()* function is used to abort a pending transmit or receive request. If the token is in the transmit or receive request queues, after calling this function, Token->Status will be set to *EFI_ABORTED* and then Token->Event will be signaled. If the token is not in one of the queues, which usually means the asynchronous operation has completed, this function will not signal the token and *EFI_NOT_FOUND* is returned.

Status Codes Returned

EFI_SUCCESS	The asynchronous I/O request was aborted and *Token->Event* was signaled. When *Token* is *NULL,* all pending requests were aborted and their events were signaled.
EFI_INVALID_PARAMETER	*This* is **NULL**.
EFI_NOT_STARTED	This instance has not been started.
EFI_NOT_FOUND	When *Token* is not **NULL**, the asynchronous I/O request was not found in the transmit or receive queue. It has either completed or was not issued by *Transmit()* and *Receive()*.
EFI_DEVICE_ERROR	An unexpected system or network error occurred.

28.5.13 EFI_IP6_PROTOCOL.Poll()

Summary

Polls for incoming data packets and processes outgoing data packets.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP6_POLL) (
  IN EFI_IP6_PROTOCOL      *This
  );
```

Description

The *Poll()* function polls for incoming data packets and processes outgoing data packets. Network drivers and applications can call the *EFI_IP6_PROTOCOL.Poll()* function to increase the rate that data packets are moved between the communications device and the transmit and receive queues.

In some systems the periodic timer event may not poll the underlying communications device fast enough to transmit and/or receive all data packets without missing incoming packets or dropping outgoing packets. Drivers and applications that are experiencing packet loss should try calling the *EFI_IP6_PROTOCOL.Poll()* function more often.

Status Codes Returned

EFI_SUCCESS	Incoming or outgoing data was processed.
EFI_NOT_STARTED	This EFI IPv6 Protocol instance has not been started.
EFI_INVALID_PARAMETER	*This* is **NULL**.
EFI_DEVICE_ERROR	An unexpected system or network error occurred.
EFI_NOT_READY	No incoming or outgoing data is processed.
EFI_TIMEOUT	Data was dropped out of the transmit and/or receive queue. Consider increasing the polling rate.

28.6 EFI IPv6 Configuration Protocol

This section provides a detailed description of the EFI IPv6 Configuration Protocol.

28.6.1 EFI_IP6_CONFIG_PROTOCOL

Summary

The *EFI_IP6_CONFIG_PROTOCOL* provides the mechanism to set and get various types of configurations for the EFI IPv6 network stack.

GUID

```
#define EFI_IP6_CONFIG_PROTOCOL_GUID \
  {0x937fe521,0x95ae,0x4d1a,\
    {0x89,0x29,0x48,0xbc,0xd9,0x0a,0xd3,0x1a}
```

Protocol Interface Structure

```
typedef struct _EFI_IP6_CONFIG_PROTOCOL {
  EFI_IP6_CONFIG_SET_DATA           SetData;
  EFI_IP6_CONFIG_GET_DATA           GetData;
  EFI_IP6_CONFIG_REGISTER_NOTIF     RegisterDataNotify;
  EFI_IP6_CONFIG_UNREGISTER_NOTIFY  UnregisterDataNotify;
}   EFI_IP6_CONFIG_PROTOCOL;
```

Parameters

SetData
Set the configuration for the EFI IPv6 network stack running on the communication device this EFI IPv6 Configuration Protocol instance manages. See the *SetData()* function description.

GetData
Get the configuration or register an event to monitor the change of the configuration for the EFI IPv6 network stack running on the communication device this EFI IPv6 Configuration Protocol instance manages. See the *GetData()* function description.

RegiseterDataNotify
Register an event that is to be signaled whenever a configuration process on the specified configuration data is done.

UnregisterDataNotify
Remove a previously registered event for the specified configuration data.

Description

The *EFI_IP6_CONFIG_PROTOCOL* is designed to be the central repository for the common configurations and the administrator configurable settings for the EFI IPv6 network stack.

An EFI IPv6 Configuration Protocol instance will be installed on each communication device that the EFI IPv6 network stack runs on.

28.6.2 EFI_IP6_CONFIG_PROTOCOL.SetData()

Summary

Set the configuration for the EFI IPv6 network stack running on the communication device this EFI IPv6 Configuration Protocol instance manages.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP6_CONFIG_SET_DATA) (
  IN EFI_IP6_CONFIG_PROTOCOL        *This,
  IN EFI_IP6_CONFIG_DATA_TYPE       DataType,
  IN UINTN                          DataSize,
  IN VOID                           *Data
  );
```

Parameters

This
Pointer to the *EFI_IP6_CONFIG_PROTOCOL* instance.

DataType
The type of data to set. Type *EFI_IP6_CONFIG_DATA_TYPE* is defined in "Related Definitions" below.

DataSize
Size of the buffer pointed to by *Data* in bytes.

Data
The data buffer to set. The type of the data buffer is associated with the *DataType*. The various types are defined in "Related Definitions" below.

Description

This function is used to set the configuration data of type DataType for the EFI IPv6 network stack running on the communication device this EFI IPv6 Configuration Protocol instance manages.

The *DataSize* is used to calculate the count of structure instances in the Data for some *DataType* that multiple structure instances are allowed.

This function is always non-blocking. When setting some type of configuration data, an asynchronous process is invoked to check the correctness of the data, such as doing Duplicate Address Detection on the manually set local IPv6 addresses. *EFI_NOT_READY* is returned immediately to indicate that such an asynchronous process is invoked and the process is not finished yet. The caller willing to get the result of the asynchronous process is required to call *RegisterDataNotify()* to register an event on the specified configuration data. Once the event is signaled, the caller can call *GetData()* to get back the configuration data in order to know the result. For other types of configuration data that do not require an asynchronous configuration process, the result of the operation is immediately returned.

Related Definition

```
//*************************************************
// EFI_IP6_CONFIG_DATA_TYPE
//*************************************************
typedef enum {
  Ip6ConfigDataTypeInterfaceInfo,
  Ip6ConfigDataTypeAltInterfaceId,
  Ip6ConfigDataTypePolicy,
  Ip6ConfigDataTypeDupAddrDetectTransmits,
```

(continues on next page)

(continued from previous page)

```
  Ip6ConfigDataTypeManualAddress,
  Ip6ConfigDataTypeGateway,
  Ip6ConfigDataTypeDnsServer,
  Ip6ConfigDataTypeMaximum
}   EFI_IP6_CONFIG_DATA_TYPE;
```

Ip6ConfigDataTypeInterfaceInfo
The interface information of the communication device this EFI IPv6 Configuration Protocol instance manages. This type of data is read only. The corresponding *Data* is of type *EFI_IP6_CONFIG_INTERFACE_INFO*.

Ip6ConfigDataTypeAltInterfaceId
The alternative interface ID for the communication device this EFI IPv6 Configuration Protocol instance manages if the link local IPv6 address generated from the interfaced ID based on the default source the EFI IPv6 Protocol uses is a duplicate address. The length of the interface ID is 64-bit. The corresponding *Data* is of type *EFI_IP6_CONFIG_INTERFACE_ID*.

Ip6ConfigDataTypePolicy
The general configuration policy for the EFI IPv6 network stack running on the communication device this EFI IPv6 Configuration Protocol instance manages. The policy will affect other configuration settings. The corresponding *Data* is of type *EFI_IP6_CONFIG_POLICY*.

Ip6ConfigDataTypeDupAddrDetectTransmits
The number of consecutive Neighbor Solicitation messages sent while performing Duplicate Address Detection on a tentative address. A value of zero indicates that Duplicate Address Detection will not be performed on tentative addresses. The corresponding *Data* is of type *EFI_IP6_CONFIG_DUP_ADDR_DETECT_TRANSMITS*.

Ip6ConfigDataTypeManualAddress
The station addresses set manually for the EFI IPv6 network stack. It is only configurable when the policy is *Ip6ConfigPolicyManual*. The corresponding Data is a pointer to an array of *EFI_IPv6_ADDRESS* instances. When *DataSize* is 0 and *Data* is *NULL,* the existing configuration is cleared from the EFI IPv6 Configuration Protocol instance.

Ip6ConfigDataTypeGateway
The gateway addresses set manually for the EFI IPv6 network stack running on the communication device this EFI IPv6 Configuration Protocol manages. It is not configurable when the policy is *Ip6ConfigPolicyAutomatic*. The gateway addresses must be unicast IPv6 addresses. The corresponding Data is a pointer to an array of *EFI_IPv6_ADDRESS* instances. When *DataSize* is 0 and *Data* is *NULL,* the existing configuration is cleared from the EFI IPv6 Configuration Protocol instance.

Ip6ConfigDataTypeDnsServer
The DNS server list for the EFI IPv6 network stack running on the communication device this EFI IPv6 Configuration Protocol manages. It is not configurable when the policy is *Ip6ConfigPolicyAutomatic*.The DNS server addresses must be unicast IPv6 addresses. The corresponding Data is a pointer to an array of *EFI_IPv6_ADDRESS* instances. When *DataSize* is 0 and *Data* is *NULL,* the existing configuration is cleared from the EFI IPv6 Configuration Protocol instance.

```
//*******************************************************
// EFI_IP6_CONFIG_INTERFACE_INFO
//*******************************************************
typedef struct {
  CHAR16                Name[32];
  UINT8                 IfType;
  UINT32                HwAddressSize;
  EFI_MAC_ADDRESS       HwAddress;
  UINT32                AddressInfoCount;
```

(continues on next page)

(continued from previous page)

```
  EFI_IP6_ADDRESS_INFO  *AddressInfo;
  UINT32                RouteCount;
  EFI_IP6_ROUTE_TABLE   *RouteTable;
}   EFI_IP6_CONFIG_INTERFACE_INFO;
```

Name
The name of the interface. It is a NULL-terminated string.

IfType
The interface type of the network interface. See RFC 3232, section "Number Hardware Type".

HwAddressSize
The size, in bytes, of the network interface's hardware address.

HwAddress
The hardware address for the network interface.

AddressInfoCount
Number of *EFI_IP6_ADDRESS_INFO* structures pointed to by AddressInfo.

AddressInfo
Pointer to an array of *EFI_IP6_ADDRESS_INFO* instances which contain the local IPv6 addresses and the corresponding prefix length information. Set to **NULL** if *AddressInfoCount* is zero. Type *EFI_IP6_ADDRESS_INFO* is defined in *EFI_IP6_PROTOCOL.GetModeData()*.

RouteCount
Number of route table entries in the following *RouteTable*.

RouteTable
The route table of the IPv6 network stack runs on this interface. Set to **NULL** if *RouteCount* is zero. Type *EFI_IP6_ROUTE_TABLE* is defined in *EFI_IP6_PROTOCOL.GetModeData()*.

The EFI_IP6_CONFIG_INTERFACE_INFO structure describes the operational state of the interface this EFI IPv6 Configuration Protocol instance manages. This type of data is read-only. When reading, the caller allocated buffer is used to return all of the data, i.e., the first part of the buffer is EFI_IP6_CONFIG_INTERFACE_INFO and the followings are the array of EFI_IP6_ADDRESS_INFO and the route table if present. The caller should NOT free the buffer pointed to by AddressInfo or RouteTable, and the caller is only required to free the whole buffer if the data is not needed any more.

```
//*************************************************
// EFI_IP6_CONFIG_INTERFACE_ID
//*************************************************
typedef struct {
  UINT8           Id[8];
}   EFI_IP6_CONFIG_INTERFACE_ID;
```

The *EFI_IP6_CONFIG_INTERFACE_ID* structure describes the 64-bit interface ID.

```
//*************************************************
// EFI_IP6_CONFIG_POLICY
//*************************************************
typedef enum {
  Ip6ConfigPolicyManual,
  Ip6ConfigPolicyAutomatic
}   EFI_IP6_CONFIG_POLICY;
```

Ip6ConfigPolicyManual
Under this policy, the *IpI6ConfigDataTypeManualAddress, Ip6ConfigDataTypeGateway* and *Ip6ConfigDataTypeDnsServer* configuration data are required to be set manually. The EFI IPv6 Protocol will get all required configuration such as address, prefix and gateway settings from the EFI IPv6 Configuration protocol.

Ip6ConfigPolicyAutomatic
Under this policy, the *IpI6ConfigDataTypeManualAddress, Ip6ConfigDataTypeGateway* and *Ip6ConfigDataTypeDnsServer* configuration data are not allowed to set via *SetData()*. All of these configurations are retrieved from some auto configuration mechanism. The EFI IPv6 Protocol will use the IPv6 stateless address autoconfiguration mechanism and/or the IPv6 stateful address autoconfiguration mechanism described in the related RFCs to get address and other configuration information.

The *EFI_IP6_CONFIG_POLICY* defines the general configuration policy the EFI IPv6 Configuration Protocol supports. The default policy for a newly detected communication device is beyond the scope of this document. An implementation might leave it to platform to choose the default policy.

The configuration data of type *IpI6ConfigDataTypeManualAddress, Ip6ConfigDataTypeGateway* and *Ip6ConfigDataTypeDnsServer* will be flushed if the policy is changed from *Ip6ConfigPolicyManual* to *Ip6ConfigPolicyAutomatic*.

```
//*******************************************************
// EFI_IP6_CONFIG_DUP_ADDR_DETECT_TRANSMITS
//*******************************************************
typedef struct {
  UINT32             DupAddrDetectTransmits;
}   EFI_IP6_CONFIG_DUP_ADDR_DETECT_TRANSMITS;
```

The *EFI_IP6_CONFIG_DUP_ADDR_DETECT_TRANSMITS* structure describes the number of consecutive Neighbor Solicitation messages sent while performing Duplicate Address Detection on a tentative address. The default value for a newly detected communication device is 1.

```
//*******************************************************
// EFI_IP6_CONFIG_MANUAL_ADDRESS
//*******************************************************
typedef struct {
  EFI_IPv6_ADDRESS      Address;
  BOOLEAN               IsAnycast;
  UINT8                 PrefixLength;
}   EFI_IP6_CONFIG_MANUAL_ADDRESS;
```

Address
The IPv6 unicast address.

IsAnycast
Set to **TRUE** if *Address* is anycast.

PrefixLength
The length, in bits, of the prefix associated with this *Address*.

The EFI_IP6_CONFIG_MANUAL_ADDRESS structure is used to set the station address information for the EFI IPv6 network stack manually when the policy is Ip6ConfigPolicyManual.

Status Codes Returned

EFI_SUCCESS	The specified configuration data for the EFI IPv6 network stack is set successfully.
EFI_INVALID_PARAMETER	One or more of the following are **TRUE:** • *This* is **NULL**. • One or more fields in *Data* and *DataSize* do not match the requirement of the data type indicated by *DataType*.
EFI_WRITE_PROTECTED	The specified configuration data is read-only or the specified configuration data can not be set under the current policy.
EFI_ACCESS_DENIED	Another set operation on the specified configuration data is already in process.
EFI_NOT_READY	An asynchronous process is invoked to set the specified configuration data and the process is not finished yet.
EFI_BAD_BUFFER_SIZE	The *DataSize* does not match the size of the type indicated by *DataType*.
EFI_UNSUPPORTED	This *DataType* is not supported.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.
EFI_DEVICE_ERROR	An unexpected system error or network error occurred.

28.6.3 EFI_IP6_CONFIG_PROTOCOL.GetData()

Summary

Get the configuration data for the EFI IPv6 network stack running on the communication device this EFI IPv6 Configuration Protocol instance manages.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP6_CONFIG_GET_DATA) (
  IN EFI_IP6_CONFIG_PROTOCOL        *This,
  IN EFI_IP6_CONFIG_DATA_TYPE       DataType,
  IN OUT UINTN                      *DataSize,
  IN VOID                           *Data OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_IP6_CONFIG_PROTOCOL* instance.

DataType
The type of data to get. Type *EFI_IP6_CONFIG_DATA_TYPE* is defined in *EFI_IP6_CONFIG_PROTOCOL.SetData()*.

DataSize
On input, in bytes, the size of *Data*. On output, in bytes, the size of buffer required to store the specified configuration data.

Data
The data buffer in which the configuration data is returned. The type of the data buffer is associated with the *DataType*. Ignored if *DataSize* is 0. The various types are defined in *EFI_IP6_CONFIG_PROTOCOL.SetData()*.

Description

This function returns the configuration data of type *DataType* for the EFI IPv6 network stack running on the communication device this EFI IPv6 Configuration Protocol instance manages.

The caller is responsible for allocating the buffer used to return the specified configuration data and the required size will be returned to the caller if the size of the buffer is too small.

EFI_NOT_READY is returned if the specified configuration data is not ready due to an already in progress asynchronous configuration process. The caller can call *RegisterDataNotify()* to register an event on the specified configuration data. Once the asynchronous configuration process is finished, the event will be signaled and a subsequent *GetData()* call will return the specified configuration data.

Status Codes Returned

EFI_SUCCESS	The specified configuration data is got successfully.
EFI_INVALID_PARAMETER	One or more of the followings are **TRUE:** • *This* is **NULL**. • *DataSize* is **NULL**. • *Data* is **NULL** if * *DataSize* is not zero.
EFI_BUFFER_TOO_SMALL	The size of Data is too small for the specified configuration data and the required size is returned in *DataSize*.
EFI_NOT_READY	The specified configuration data is not ready due to an already in progress asynchronous configuration process.
EFI_NOT_FOUND	The specified configuration data is not found.

28.6.4 EFI_IP6_CONFIG_PROTOCOL.RegisterDataNotify ()

Summary

Register an event that is to be signaled whenever a configuration process on the specified configuration data is done.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP6_CONFIG_REGISTER_NOTIFY) (
  IN EFI_IP6_CONFIG_PROTOCOL       *This,
  IN EFI_IP6_CONFIG_DATA_TYPE      DataType,
  IN EFI_EVENT                     Event
  );
```

Parameters

This
Pointer to the *EFI_IP6_CONFIG_PROTOCOL* instance.

DataType
The type of data to unregister the event for. Type *EFI_IP6_CONFIG_DATA_TYPE* is defined in *EFI_IP6_CONFIG_PROTOCOL.SetData()*.

Event
The event to register.

Description

This function registers an event that is to be signaled whenever a configuration process on the specified configuration data is done. An event can be registered for different DataType simultaneously and the caller is responsible for determining which type of configuration data causes the signaling of the event in such case.

Status Codes Returned

EFI_SUCCESS	The notification event for the specified configuration data is registered.
EFI_INVALID_PARAMETER	This is **NULL** or Event is **NULL**.
EFI_UNSUPPORTED	The configuration data type specified by DataType is not supported.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.
EFI_ACCESS_DENIED	The Event is already registered for the DataType.

28.6.5 EFI_IP6_CONFIG_PROTOCOL.UnregisterDataNotify()

Summary

Remove a previously registered event for the specified configuration data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IP6_CONFIG_UNREGISTER_NOTIFY) (
  IN EFI_IP6_CONFIG_PROTOCOL              *This,
  IN EFI_IP6_CONFIG_DATA_TYPE             DataType,
  IN EFI_EVENT                            Event
  );
```

Parameters

This
Pointer to the EFI_IP6_CONFIG_PROTOCOL instance.

DataType
The type of data to remove the previously registered event for. Type *EFI_IP6_CONFIG_DATA_TYPE* is defined in *EFI_IP6_CONFIG_PROTOCOL.SetD* ata().

Event
The event to unregister.

Description

This function removes a previously registered event for the specified configuration data.

Status Codes Returned

EFI_SUCCESS	The event registered for the specified configuration data is removed.
EFI_INVALID_PARAMETER	This is **NULL** or Event is **NULL**
EFI_NOT_FOUND	The event has not been registered for the specified DataType.

28.7 IPsec

28.7.1 IPsec Overview

IPsec is a framework of open standards that provides data confidentiality, data integrity, data authentication and replay protection between participating peers. A set of security services is provided by IPsec for traffic at the IP layer, in both the IPv4 and IPv6 environment. To the stronger, IPV6 requires IPSec support.

IPsec is documented in a series of Internet RFCs. The overall IPsec architecture and implementation are guided by "Security Architecture for the Internet Protocol", RFC 4301.

Two different security protocols - Authentication Header (AH, described in RFC 4302) and Encapsulated Security Payload (ESP, described in RFC 4303) - are used to provide package-level security for IP datagram.

This section attempts to capture the generic configuration for an IPsec implementation in an EFI environment.

28.7.2 EFI IPsec Configuration Protocol

This section provides a detailed description of the EFI IPsec Configuration Protocol. This protocol sets and obtains the IPsec configuration information.

28.7.3 EFI_IPSEC_CONFIG_PROTOCOL

Summary

The *EFI_IPSEC_CONFIG_PROTOCOL* provides the mechanism to set and retrieve security and policy related information for the EFI IPsec protocol driver.

GUID

```
#define EFI_IPSEC_CONFIG_PROTOCOL_GUID \
  {0xce5e5929,0xc7a3,0x4602,\
    {0xad,0x9e,0xc9,0xda,0xf9,0x4e,0xbf,0xcf}}
```

Protocol Interface Structure

```
typedef struct _EFI_IPSEC_CONFIG_PROTOCOL {
  EFI_IPSEC_CONFIG_SET_DATA               SetData;
  EFI_IPSEC_CONFIG_GET_DATA               GetData;
  EFI_IPSEC_CONFIG_GET_NEXT_SELECTOR      GetNextSelector;
  EFI_IPSEC_CONFIG_REGISTER_NOTIFY        RegisterDataNotify;
  EFI_IPSEC_CONFIG_UNREGISTER_NOTIFY      UnregisterDataNotify;
}   EFI_IPSEC_CONFIG_PROTOCOL;
```

Parameters

SetData
Set the configuration and control information for the EFI IPsec protocol driver. See the *SetData()* function description.

GetData
Look up and retrieve the IPsec configuration data. See the *GetData()* function description.

GetNextSelector
Enumerates the current IPsec configuration data entry selector. See the *GetNextSelector()* function description.

RegiseterNotify
Register an event that is to be signaled whenever a configuration process on the specified IPsec configuration data is done.

UnregisterNotify
Remove a registered event for the specified IPsec configuration data.

Description

The *EFI_IPSEC_CONFIG_PROTOCOL* provides the ability to set and lookup the IPsec SAD (Security Association Database), SPD (Security Policy Database) data entry and configure the security association management protocol such as IKEv2. This protocol is used as the central repository of any policy-specific configuration for EFI IPsec driver.

EFI_IPSEC_CONFIG_PROTOCOL can be bound to both IPv4 and IPv6 stack. User can use this protocol for IPsec configuration in both IPv4 and IPv6 environment.

28.7.4 EFI_IPSEC_CONFIG_PROTOCOL.SetData()

Summary

Set the security association, security policy and peer authorization configuration information for the EFI IPsec driver.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IPSEC_CONFIG_SET_DATA) (
  IN EFI_IPSEC_CONFIG_PROTOCOL         *This,
  IN EFI_IPSEC_CONFIG_DATA_TYPE        DataType,
  IN EFI_IPSEC_CONFIG_SELECTOR         *Selector
  IN VOID                              *Data
  IN EFI_IPSEC_CONFIG_SELECTOR         *InsertBefore OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_IPSEC_CONFIG_PROTOCOL* instance.

InsertBefore
Pointer to one entry selector which describes the expected position the new data entry will be added. If *InsertBefore* is *NULL,* the new entry will be appended the end of database.

DataType
The type of data to be set. Type *EFI_IPSEC_CONFIG_DATA_TYPE* is defined in "Related Definitions" below.

Selector
Pointer to an entry selector on operated configuration data specified by *DataType*. A **NULL** *Selector* causes the entire specified-type configuration information to be flushed.

Data
The data buffer to be set. The structure of the data buffer is associated with the *DataType*. The various types are defined in "Related Definitions" below.

Description

This function is used to set the IPsec configuration information of type *DataType* for the EFI IPsec driver.

The IPsec configuration data has a unique selector/identifier separately to identify a data entry. The selector structure depends on *DataType's* definition.

Using *SetData()* with a *Data* of **NULL** causes the IPsec configuration data entry identified by *DataType* and Selector to be deleted.

Related Definition

```
//*******************************************************
// EFI_IPSEC_CONFIG_DATA_TYPE
//*******************************************************
typedef enum {
  IPsecConfigDataTypeSpd,
  IPsecConfigDataTypeSad,
  IPsecConfigDataTypePad,
  IPsecConfigDataTypeMaximum
}   EFI_IPSEC_CONFIG_DATA_TYPE;
```

IPsecConfigDataTypeSpd
The IPsec Security Policy Database (aka SPD) setting. In IPsec, an essential element of Security Association (SA) processing is underlying SPD that specifies what services are to be offered to IP datagram and in what fashion. The SPD must be consulted during the processing of all traffic (inbound and outbound), including traffic not protected by IPsec, that traverses the IPsec boundary. With this *DataType, SetData()* function is to set the SPD entry information, which may add one new entry, delete one existed entry or flush the whole database according to the parameter values. The corresponding Data is of type *EFI_IPSEC_SPD_DATA*.

IPsecConfigDataTypeSad
The IPsec Security Association Database (aka SAD) setting. A SA is a simplex connection that affords security services to the traffic carried by it. Security services are afforded to an SA by the use of AH, or ESP, but not both. The corresponding Data is of type *EFI_IPSEC_SA_DATA2* or *EFI_IPSEC_SAD_DATA*. Compared with *EFI_IPSEC_SA_DATA,* the *EFI_IPSEC_SA_DATA2* contains the extra Tunnel Source Address and Tunnel Destination Address thus it is recommended to be use if the implementation supports tunnel mode.

IPsecConfigDataTypePad
The IPsec Peer Authorization Database (aka PAD) setting, which provides the link between the SPD and a security association management protocol. The PAD entry specifies the authentication protocol (e.g. IKEv1, IKEv2) method used and the authentication data. The corresponding *Data* is of type *EFI_IPSEC_PAD_DATA*.

```
//*******************************************************
// EFI_IPSEC_CONFIG_SELECTOR
//*******************************************************
typedef union {
  EFI_IPSEC_SPD_SELECTOR             SpdSelector;
  EFI_IPSEC_SA_ID                    SaId;
  EFI_IPSEC_PAD_ID                   PadId;
}   EFI_IPSEC_CONFIG_SELECTOR;
```

The *EFI_IPSEC_CONFIG_SELECTOR* describes the expected IPsec configuration data selector of type *EFI_IPSEC_CONFIG_DATA_TYPE*.

```
//*******************************************************
// EFI_IPSEC_SPD_SELECTOR
//*******************************************************
typedef struct _EFI_IPSEC_SPD_SELECTOR {
  UINT32                             LocalAddressCount;
  EFI_IP_ADDRESS_INFO                *LocalAddress;
  UINT32                             RemoteAddressCount;
  EFI_IP_ADDRESS_INFO                *RemoteAddress;
```

(continues on next page)

(continued from previous page)

```
  UINT16                              NextLayerProtocol;

  // Several additional selectors depend on the ProtoFamily
  UINT16                               LocalPort;
  UINT16                               LocalPortRange;
  UINT16                               RemotePort;
  UINT16                               RemotePortRange;
}   EFI_IPSEC_SPD_SELECTOR;
```

LocalAddressCount
Specifies the actual number of entries in *LocalAddress*.

LocalAddress
A list of ranges of IPv4 or IPv6 addresses, which refers to the addresses being protected by IPsec policy.

RemoteAddressCount
Specifies the actual number of entries in *RemoteAddress*.

RemoteAddress
A list of ranges of IPv4 or IPv6 addresses, which are peer entities to *LocalAddress*.

NextLayerProtocol
Next layer protocol. Obtained from the IPv4 Protocol or the IPv6 Next Header fields. The next layer protocol is whatever comes after any IP extension headers that are present. A zero value is a wildcard that matches any value in *NextLayerProtocol* field.

LocalPort
Local Port if the Next Layer Protocol uses two ports (as do TCP, UDP, and others). A zero value is a wildcard that matches any value in *LocalPort* field.

LocalPortRange
A designed port range size. The start port is *LocalPort,* and the total number of ports is described by *LocalPortRange*. This field is ignored if *NextLayerProtocol* does not use ports.

RemotePort
Remote Port if the Next Layer Protocol uses two ports. A zero value is a wildcard that matches any value in RemotePort field.

RemotePortRange
A designed port range size. The start port is *RemotePort,* and the total number of ports is described by *RemotePortRange*. This field is ignored if *NextLayerProtocol* does not use ports.

NOTE: *The* LocalPort *and* RemotePort *selectors have different meaning depending on the* NextLayerProtocol *field. for example, if* NextLayerProtocol *value is ICMP*, LocalPort *and* RemotePort *describe the ICMP message type and code. This is described in section 4.4.1.1 of RFC 4301).*

```
//*****************************************************
// EFI_IP_ADDRESS_INFO
//*****************************************************
typedef struct _EFI_IP_ADDRESS_INFO {
  EFI_IP_ADDRESS                      Address;
  UINT8                               PrefixLength;
}   EFI_IP_ADDRESS_INFO;
```

Address
The IPv4 or IPv6 address.

PrefixLength
The length of the prefix associated with the *Address*.

```
#define MAX_PEERID_LEN 128
//*******************************************************
// EFI_IPSEC_SPD_DATA
//*******************************************************
typedef struct _EFI_IPSEC_SPD_DATA {
  UINT8                                 *Name[MAX_PEERID_LEN];
  UINT32                                PackageFlag;
  EFI_IPSEC_TRAFFIC_DIR                 TrafficDirection;
  EFI_IPSEC_ACTION                      Action;
  EFI_IPSEC_PROCESS_POLICY              *ProcessingPolicy;
  UINTN                                 SaIdCount;
  EFI_IPSEC_SA_ID                       *SaId[1];
}   EFI_IPSEC_SPD_DATA;
```

Name
A null-terminated ASCII name string which is used as a symbolic identifier for an IPsec Local or Remote address. The *Name* is optional, and can be **NULL**.

PackageFlag
Bit-mapped list describing Populate from Packet flags. When creating a SA, if *PackageFlag* bit is set to **TRUE**, instantiate the selector from the corresponding field in the package that triggered the creation of the SA, else from the value(s) in the corresponding SPD entry. The *PackageFlag* bit setting for corresponding selector field of *EFI_IPSEC_SPD_SELECTOR:*

Bit 0: **EFI_IPSEC_SPD_SELECTOR**. *LocalAddress*
Bit 1: **EFI_IPSEC_SPD_SELECTOR**. *RemoteAddress*
Bit 2: **EFI_IPSEC_SPD_SELECTOR**. *NextLayerProtocol*
Bit 3: **EFI_IPSEC_SPD_SELECTOR**. *LocalPort*
Bit 4: **EFI_IPSEC_SPD_SELECTOR**. *RemotePort*
Others: Reserved.

TrafficDirection
The traffic direction of data gram.

Action
Processing choices to indicate which action is required by this policy.

ProcessingPolicy
The policy and rule information for a SPD entry. The type *EFI_IPSEC_PROCESSPOLICY* is defined in below.

SaIdCount
Specifies the actual number of entries in SaId list.

SaId
Pointer to the SAD entry used for the traffic processing. The existed SAD entry links indicate this is the manual key case.

```
//*******************************************************
// EFI_IPSEC_TRAFFIC_DIR
//*******************************************************
```

(continues on next page)

(continued from previous page)

```
typedef enum {
  EfiIPsecInBound,
  EfiIPsecOutBound
} EFI_IPSEC_TRAFFIC_DIR;
```

The *EFI_IPSEC_TRAFFIC_DIR* represents the directionality in an SPD entry. The *EfiIPsecInBound* refers to traffic entering an IPsec implementation via the unprotected interface or emitted by the implementation on the unprotected side of the boundary and directed towards the protected interface. The *EfiIPsecOutBound* refers to traffic entering the implementation via the protected interface, or emitted by the implementation on the protected side of the boundary and directed toward the unprotected interface.

```
//*******************************************************
// EFI_IPSEC_ACTION
//*******************************************************
typedef enum {
  EfiIPsecActionDiscard,
  EfiIPsecActionBypass,
  EfiIPsecActionProtect
}   EFI_IPSEC_ACTION;
```

For any inbound or outbound datagram, EFI_IPSEC_ACTION represents three possible processing choices:

EfiIPsecActionDiscard
Refers to traffic that is not allowed to traverse the IPsec boundary (in the direction specified by *EFI_IPSEC_TRAFFIC_DIR* ;

EfiIPsecActionByPass
Refers to traffic that is allowed to cross the IPsec boundary without protection.

EfiIPsecActionProtect
Refers to traffic that is afforded IPsec protection, and for such traffic the SPD must specify the security protocols to be employed, their mode, security service options, and the cryptographic algorithms to be used.

```
//*********************************************************
// EFI_IPSEC_PROCESS_POLICY
//*********************************************************
typedef struct _EFI_IPSEC_PROCESS_POLICY {
  BOOLEAN                     ExtSeqNum;
  BOOLEAN                     SeqOverflow;
  BOOLEAN                     FragCheck;
  EFI_IPSEC_SA_LIFETIME       SaLifetime;
  EFI_IPSEC_MODE              Mode;
  EFI_IPSEC_TUNNEL_OPTION     *TunnelOption;
  EFI_IPSEC_PROTOCOL_TYPE     Proto;
  UINT8                       AuthAlgoId;
  UINT8                       EncAlgoId;
} EFI_IPSEC_PROCESS_POLICY;
```

If required action of an SPD entry is *EfiIPsecActionProtect,* the *EFI_IPSEC_PROCESS_POLICY* structure describes a policy list for traffic processing.

ExtSeqNum
Extended Sequence Number. Is this SA using extended sequence numbers. 64-bit counter is used if **TRUE**.

SeqOverflow
A flag indicating whether overflow of the sequence number counter should generate an auditable event and

prevent transmission of additional packets on the SA, or whether rollover is permitted.

FragCheck
Is this SA using stateful fragment checking. **TRUE** represents stateful fragment checking.

SaLifetime
A time interval after which a SA must be replaced with a new SA (and new SPI) or terminated. The type *EFI_IPSEC_SA_LIFETIME* is defined in below.

Mode
IPsec mode: tunnel or transport

TunnelOption
Tunnel Option. *TunnelOption* is ignored if Mode is *EfiIPsecTransport*. The type *EFI_IPSEC_TUNNEL_OPTION* is defined in below

Proto
IPsec protocol: AH or ESP

AuthAlgoId
Cryptographic algorithm type used for authentication

EncAlgoId
Cryptographic algorithm type used for encryption. *EncAlgo* is **NULL** when IPsec protocol is AH. For ESP protocol, *EncAlgo* can also be used to describe the algorithm if a combined mode algorithm is used.

```
//*******************************************************
// EFI_IPSEC_SA_LIFETIME
//*******************************************************
typedef struct _EFI_IPSEC_SA_LIFETIME {
  UINT64                    ByteCount;
  UINT64                    SoftLifetime;
  UINT64                    HardLifetime
}   EFI_IPSEC_SA_LIFETIME;
```

EFI_IPSEC_SA_LIFETIME defines the lifetime of an SA, which represents when a SA must be replaced or terminated. A value of all 0 for each field removes the limitation of a SA lifetime.

ByteCount
The number of bytes to which the IPsec cryptographic algorithm can be applied. For ESP, this is the encryption algorithm and for AH, this is the authentication algorithm. The *ByteCount* includes pad bytes for cryptographic operations.

SoftLifetime
A time interval in second that warns the implementation to initiate action such as setting up a replacement SA.

HardLifetime
A time interval in second when the current SA ends and is destroyed.

```
//*******************************************************
// EFI_IPSEC_MODE
//*******************************************************
typedef enum {
  EfiIPsecTransport,
  EfiIPsecTunnel
}   EFI_IPSEC_MODE;
```

There are two modes of IPsec operation: transport mode and tunnel mode. In *EfiIPsecTransport* mode, AH and ESP provide protection primarily for next layer protocols; In *EfiIPsecTunnel* mode, AH and ESP are applied to tunneled IP

packets.

```
typedef enum {
  EfiIPsecTunnelClearDf,
  EfiIPsecTunnelSetDf,
  EfiIPsecTunnelCopyDf
}   EFI_IPSEC_TUNNEL_DF_OPTION;
```

The option of copying the DF bit from an outbound package to the tunnel mode header that it emits, when traffic is carried via a tunnel mode SA. This applies to SAs where both inner and outer headers are IPv4. The value can be:

EfiIPsecTunnelClearDf:
Clear DF bit from inner header

EfiIPsecTunnelSetDf:
Set DF bit from inner header

EfiIPsecTunnelCopyDf:
Copy DF bit from inner header

```
//*******************************************************
// EFI_IPSEC_TUNNEL_OPTION
//*******************************************************
typedef struct _EFI_IPSEC_TUNNEL_OPTION {
  EFI_IP_ADDRESS              LocalTunnelAddress;
  EFI_IP_ADDRESS              RemoteTunnelAddress;
  EFI_IPSEC_TUNNEL_DF_OPTION  DF;
}   EFI_IPSEC_TUNNEL_OPTION;
```

LocalTunnelAddress
Local tunnel address when IPsec mode is *EfiIPsecTunnel*

RemoteTunnelAddress
Remote tunnel address when IPsec mode is *EfiIPsecTunnel*

DF
The option of copying the DF bit from an outbound package to the tunnel mode header that it emits, when traffic is carried via a tunnel mode SA.

```
//*******************************************************
// EFI_IPSEC_PROTOCOL_TYPE
//*******************************************************
typedef enum {
  EfiIPsecAH,
  EfiIPsecESP
}   EFI_IPSEC_PROTOCOL_TYPE;
```

IPsec protocols definition. *EfiIPsecAH* is the IP Authentication Header protocol which is specified in RFC 4302. *EfiIPsecESP* is the IP Encapsulating Security Payload which is specified in RFC 4303.

```
//*******************************************************
// EFI_IPSEC_SA_ID
//*******************************************************
typedef struct _EFI_IPSEC_SA_ID {
  UINT32                      Spi;
  EFI_IPSEC_PROTOCOL_TYPE     Proto;
```

(continues on next page)

(continued from previous page)

```
  EFI_IP_ADDRESS                DestAddress;
}   EFI_IPSEC_SA_ID;
```

A triplet to identify an SA, consisting of the following members:

Spi
Security Parameter Index (aka SPI). An arbitrary 32-bit value that is used by a receiver to identity the SA to which an incoming package should be bound.

Proto
IPsec protocol: AH or ESP

DestAddress
Destination IP address.

```
//*******************************************************
// EFI_IPSEC_SA_DATA
//*******************************************************
typedef struct _EFI_IPSEC_SA_DATA {
  EFI_IPSEC_MODE                Mode;
  UINT64                        SNCount;
  UINT8                         AntiReplayWindows;
  EFI_IPSEC_ALGO_INFO           AlgoInfo;
  EFI_IPSEC_SA_LIFETIME         SaLifetime;
  UINT32                        PathMTU;
  EFI_IPSEC_SPD_SELECTOR        *SpdSelector;
  BOOLEAN                       ManualSet
}   EFI_IPSEC_SA_DATA;
```

The data items defined in one SAD entry:

Mode
IPsec mode: tunnel or transport

SNCount
Sequence Number Counter. A 64-bit counter used to generate the sequence number field in AH or ESP headers.

ReplayWindows
Anti-Replay Window. A 64-bit counter and a bit-map used to determine whether an inbound AH or ESP packet is a replay.

AlgoInfo
AH/ESP cryptographic algorithm, key and parameters.

SaLifeTime
Lifetime of this SA.

PathMTU
Any observed path MTU and aging variables. The Path MTU processing is defined in section 8 of RFC 4301.

SpdSelector
Link to one SPD entry.

ManualSet
Indication of whether it's manually set or negotiated automatically. If *ManualSet* is *FALSE,* the corresponding SA entry is inserted through IKE protocol negotiation.

```
//*******************************************************
// EFI_IPSEC_SA_DATA2
//*******************************************************
typedef struct _EFI_IPSEC_SA_DATA2 {
  EFI_IPSEC_MODE                Mode;
  UINT64                        SNCount;
  UINT8                         AntiReplayWindows;
  EFI_IPSEC_ALGO_INFO           AlgoInfo;
  EFI_IPSEC_SA_LIFETIME         SaLifetime;
  UINT32                        PathMTU;
  EFI_IPSEC_SPD_SELECTOR        *SpdSelector;
  BOOLEAN                       ManualSet;
  EFI_IP_ADDRESS                TunnelSourceAddress;
  EFI_IP_ADDRESS                TunnelDestinationAddress
}   EFI_IPSEC_SA_DATA2;
```

The data items defined in one SAD entry:

Mode
IPsec mode: tunnel or transport

SNCount
Sequence Number Counter. A 64-bit counter used to generate the sequence number field in AH or ESP headers.

ReplayWindows
Anti-Replay Window. A 64-bit counter and a bit-map used to determine whether an inbound AH or ESP packet is a replay.

AlgoInfo
AH/ESP cryptographic algorithm, key and parameters.

SaLifeTime
Lifetime of this SA.

PathMTU
Any observed path MTU and aging variables. The Path MTU processing is defined in section 8 of RFC 4301.

SpdSelector
Link to one SPD entry.

ManualSet
Indication of whether it's manually set or negotiated automatically. If ManualSet is **FALSE**, the corresponding SA entry is inserted through IKE protocol negotiation

TunnelSourceAddress
The tunnel header IP source address.

TunnelDestinationAddress
The tunnel header IP destination address.

```
//*******************************************************
// EFI_IPSEC_ALGO_INFO
//*******************************************************
typedef union {
  EFI_IPSEC_AH_ALGO_INFO              AhAlgoInfo;
  EFI_IPSEC_ESP_ALGO_INFO             EspAlgoInfo;
}   EFI_IPSEC_ALGO_INFO;
```

(continues on next page)

(continued from previous page)

```
//*******************************************************
// EFI_IPSEC_AH_ALGO_INFO
//*******************************************************
typedef struct _EFI_IPSEC_AH_ALGO_INFO {
  UINT8                 AuthAlgoId;
  UINTN                 KeyLength;
  VOID                  *Key;
}   EFI_IPSEC_AH_ALGO_INFO;
```

The security algorithm selection for IPsec AH authentication. The required authentication algorithm is specified in RFC 4305.

```
//*******************************************************
// EFI_IPSEC_ESP_ALGO_INFO
//*******************************************************
typedef struct _EFI_IPSEC_ESP_ALGO_INFO {
  UINT8                 EncAlgoId;
  UINTN                 EncKeyLength;
  VOID                  *EncKey;
  UINT8                 AuthAlgoId;
  UINTN                 AuthKeyLength;
  VOID                  *AuthKey;
}   EFI_IPSEC_ESP_ALGO_INFO;
```

The security algorithm selection for IPsec ESP encryption and authentication. The required authentication algorithm is specified in RFC 4305. *EncAlgoId* fields can also specify an ESP combined mode algorithm (e.g. AES with CCM mode, specified in RFC 4309), which provides both confidentiality and authentication services.

```
//*******************************************************
// EFI_IPSEC_PAD_ID
//*******************************************************
typedef struct _EFI_IPSEC_PAD_ID {
  BOOLEAN                 PeerIdValid;
  union {
    EFI_IP_ADDRESS_INFO   IpAddress;
    UINT8                 PeerId [MAX_PEERID_LEN];
  } Id;
} EFI_IPSEC_PAD_ID;
```

The entry selector for IPsec PAD that represents how to authenticate each peer. *EFI_IPSEC_PAD_ID* specifies the identifier for PAD entry, which is also used for SPD lookup.

IpAddress
Pointer to the IPv4 or IPv6 address range.

PeerId
Pointer to a null-terminated ASCII string representing the symbolic names. A PeerId can be a DNS name, Distinguished Name, RFC 822 email address or Key ID (specified in section 4.4.3.1 of RFC 4301)

```
//*******************************************************
// EFI_IPSEC_PAD_DATA
//*******************************************************
typedef struct _EFI_IPSEC_PAD_DATA {
```

(continues on next page)

(continued from previous page)

```
  EFI_IPSEC_AUTH_PROTOCOL_TYPE      AuthProtocol;
  EFI_IPSEC_AUTH_METHOD             AuthMethod;
  BOOLEAN                           IkeIdFlag;
  UINTN                             AuthDataSize;
  VOID                              *AuthData;
  UINTN                             RevocationDataSize;
  VOID                              *RevocationData;
}   EFI_IPSEC_PAD_DATA;
```

The data items defined in one PAD entry:

AuthProtocol
Authentication Protocol for IPsec security association management

AuthMethod
Authentication method used.

IkeIdFlag
The IKE ID payload will be used as a symbolic name for SPD lookup if *IkeIdFlag* is **TRUE**. Otherwise, the remote IP address provided in traffic selector payloads will be used.

AuthDataSize
The size of Authentication data buffer, in bytes.

AuthData
Buffer for Authentication data, (e.g., the pre-shared secret or the trust anchor relative to which the peer's certificate will be validated).

RevocationDataSize
The size of *RevocationData,* in bytes.

RevocationData
Pointer to CRL or OCSP data, if certificates are used for authentication method.

```
//*******************************************************
// EFI_IPSEC_AUTH_PROTOCOL
//*******************************************************
typedef enum {
  EfiIPsecAuthProtocolIKEv1,
  EfiIPsecAuthProtocolIKEv2,
  EfiIPsecAuthProtocolMaximum
}   EFI_IPSEC_AUTH_PROTOCOL_TYPE;
```

EFI_IPSEC_AUTH_PROTOCOL_TYPE defines the possible authentication protocol for IPsec security association management.

```
//*******************************************************
// EFI_IPSEC_AUTH_METHOD
//*******************************************************
typedef enum {
  EfiIPsecAuthMethodPreSharedSecret,
  EfiIPsecAuthMethodCertificates,
  EfiIPsecAuthMethodMaximum
}   EFI_IPSEC_AUTH_METHOD;
```

EfiIPsecAuthMethodPreSharedScret
Using Pre-shared Keys for manual security associations.

EfiIPsecAuthMethodCertificates
IKE employs X.509 certificates for SA establishment.

Status Codes Returned

EFI_SUCCESS	The specified configuration entry data is set successfully.
EFI_INVALID_PARAMETER	One or more of the following are **TRUE:** • *This* is **NULL**.
EFI_UNSUPPORTED	The specified *DataType* is not supported.
EFI_OUT_OF_RESOURCES	The required system resource could not be allocated.

28.7.5 EFI_IPSEC_CONFIG_PROTOCOL.GetData()

Summary

Return the configuration value for the EFI IPsec driver.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IPSEC_CONFIG_GET_DATA) (
  IN EFI_IPSEC_CONFIG_PROTOCOL         *This,
  IN EFI_IPSEC_CONFIG_DATA_TYPE        DataType,
  IN EFI_IPSEC_CONFIG_SELECTOR         *Selector,
  IN OUT UINTN                         *DataSize,
  OUT VOID                             *Data
);
```

Parameters

This
Pointer to the *EFI_IPSEC_CONFIG_PROTOCOL* instance.

DataType
The type of data to retrieve. Type

EFI_IPSEC_CONFIG_DATA_TYPE is defined i
EFI_IPSEC_CONFIG_PROTOCOL.SetData().

Selector
Pointer to an entry selector which is an identifier of the IPsec configuration data entry. Type

EFI_IPSEC_CONFIG_SELECTOR is defined in the
EFI_IPSEC_CONFIG_PROTOCOL.SetData() function description.

DataSize
On output the size of data returned in *Data*.

Data
The buffer to return the contents of the IPsec configuration data. The type of the data buffer is associated with the *DataType*.

Description

This function lookup the data entry from IPsec database or IKEv2 configuration information. The expected data type and unique identification are described in DataType and Selector parameters.

Status Codes Returned

EFI_SUCCESS	The specified configuration data is got successfully.
EFI_INVALID_PARAMETER	One or more of the followings are **TRUE:** • *This* is **NULL**. • *Selector* is **NULL**. • *DataSize* is **NULL**. • *Data* is **NULL**.
EFI_NOT_FOUND	The configuration data specified by *Selector* is not found.
EFI_UNSUPPORTED	The specified *DataType* is not supported.
EFI_BUFFER_TOO_SMALL	The *DataSize* is too small for the result. *DataSize* has been updated with the size needed to complete the request.

28.7.6 EFI_IPSEC_CONFIG_PROTOCOL.GetNextSelector()

Summary

Enumerates the current selector for IPsec configuration data entry.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IPSEC_CONFIG_GET_NEXT_SELECTOR) (
  IN EFI_IPSEC_CONFIG_PROTOCOL            *This,
  IN EFI_IPSEC_CONFIG_DATA_TYPE           DataType,
  IN OUT UINTN                            *SelectorSize,
  IN OUT EFI_IPSEC_CONFIG_SELECTOR        *Selector,
  );
```

Parameters

This
Pointer to the *EFI_IPSEC_CONFIG_PROTOCOL* instance.

DataType
The type of IPsec configuration data to retrieve. Type *EFI_IPSEC_CONFIG_DATA_TYPE* is defined in *EFI_IPSEC_CONFIG_PROTOCOL.SetData()*.

SelectorSize
The size of the *Selector* buffer.

Selector
On input, supplies the pointer to last *Selector* that was returned by *GetNextSelector* (). On output, returns one

copy of the current entry *Selector* of a given *DataType*. Type *EFI_IPSEC_CONFIG_SELECTOR* is defined in the *EFI_IPSEC_CONFIG_PROTOCOL.SetData()* function description.

Description

This function is called multiple times to retrieve the entry *Selector* in IPsec configuration database. On each call to *GetNextSelector(),* the next entry *Selector* are retrieved into the output interface. If the entire IPsec configuration database has been iterated, the error *EFI_NOT_FOUND* is returned. If the *Selector* buffer is too small for the next Selector copy, an *EFI_BUFFER_TOO_SMALL* error is returned, and SelectorSize is updated to reflect the size of buffer needed.

On the initial call to *GetNextSelector()* to start the IPsec configuration database search, a pointer to the buffer with all zero value is passed in *Selector*. Calls to *SetData()* between calls to *GetNextSelector* may produce unpredictable results.

Status Codes Returned

EFI_SUCCESS	The specified configuration data is got successfully.
EFI_INVALID_PARAMETER	One or more of the followings are **TRUE:** • *This* is **NULL**. • *SelectorSize* is **NULL**. • *Selector* is **NULL**.
EFI_NOT_FOUND	The next configuration data entry was not found.
EFI_UNSUPPORTED	The specified *DataType* is not supported.
EFI_BUFFER_TOO_SMALL	The *SelectorSize* is too small for the result. This parameter has been updated with the size needed to complete the search request.

28.7.7 EFI_IPSEC_CONFIG_PROTOCOL.RegisterDataNotify ()

Summary

Register an event that is to be signaled whenever a configuration process on the specified IPsec configuration information is done.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IPSEC_CONFIG_REGISTER_NOTIFY) (
  IN EFI_IPSEC_CONFIG_PROTOCOL         *This,
  IN EFI_IPSEC_CONFIG_DATA_TYPE        DataType,
  IN EFI_EVENT                         Event
);
```

Parameters

This
Pointer to the *EFI_IPSEC_CONFIG_PROTOCOL* instance.

DataType
The type of data to be registered the event for. Type *EFI_IPSEC_CONFIG_DATA_TYPE* is defined in *EFI_IPSEC_CONFIG_PROTOCOL.SetData()* function description.

Event
The event to be registered.

Description

This function registers an event that is to be signaled whenever a configuration process on the specified IPsec configuration data is done (e.g. IPsec security policy database configuration is ready). An event can be registered for different *DataType* simultaneously and the caller is responsible for determining which type of configuration data causes the signaling of the event in such case.

Status Codes Returned

EFI_SUCCESS	The event is registered successfully.
EFI_INVALID_PARAMETER	*This* is **NULL** or *Event* is **NULL**.
EFI_ACCESS_DENIED	The *Event* is already registered for the *DataType*.
EFI_UNSUPPORTED	The notify registration unsupported or the specified *DataType* is not supported.

28.7.8 EFI_IPSEC_CONFIG_PROTOCOL.UnregisterDataNotify ()

Summary

Remove the specified event that is previously registered on the specified IPsec configuration data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IPSEC_CONFIG_UNREGISTER_NOTIFY) (
  IN EFI_IPSEC_CONFIG_PROTOCOL           *This,
  IN EFI_IPSEC_CONFIG_DATA_TYPE          DataType,
  IN EFI_EVENT                           Event
  );
```

Parameters

This
Pointer to the EFI_IPSEC_CONFIG_PROTOCOL instance.

DataType
The configuration data type to remove the registered event for. Type *EFI_IPSEC_CONFIG_DATA_TYPE* is defined in *EFI_IPSEC_CONFIG_PROTOCOL.SetData()* function description.

Event
The event to be unregistered.

Description

This function removes a previously registered event for the specified configuration data.

Status Codes Returned

EFI_SUCCESS	The event is removed successfully.
EFI_NOT_FOUND	The *Event* specified by *DataType* could not be found in the database.
EFI_INVALID_PARAMETER	*This* is **NULL** or *Event* is **NULL**.

continues on next page

Table 28.49 – continued from previous page

EFI_UNSUPPORTED	The notify registration unsupported or the specified *DataType* is not supported.

28.7.9 EFI IPsec Protocol

This section provides a detailed description of the *EFI_IPSEC_PROTOCOL*. This protocol handles IPsec-protected traffic.*

28.7.10 EFI_IPSEC_PROTOCOL

Summary

The *EFI_IPSEC_PROTOCOL* is used to abstract the ability to deal with the individual packets sent and received by the host and provide packet-level security for IP datagram.

GUID

```
#define EFI_IPSEC_PROTOCOL_GUID \
  {0xdfb386f7,0xe100,0x43ad,\
    {0x9c,0x9a,0xed,0x90,0xd0,0x8a,0x5e,0x12 }}
```

Protocol Interface Structure

```
typedef struct _EFI_IPSEC_PROTOCOL {
  EFI_IPSEC_PROCESS           Process;
  EFI_EVENT                   DisabledEvent;
  BOOLEAN                     DisabledFlag;
}   EFI_IPSEC_PROTOCOL;
```

Parameters

Process
Handle the IPsec message.

DisabledEvent
Event signaled when the interface is disabled.

DisabledFlag
State of the interface.

Description

The EFI_IPSEC_PROTOCOL provides the ability for securing IP communications by authenticating and/or encrypting each IP packet in a data stream.

EFI_IPSEC_PROTOCOL can be consumed by both the IPv4 and IPv6 stack. A user can employ this protocol for IPsec package handling in both IPv4 and IPv6 environment.

28.7.11 EFI_IPSEC_PROTOCOL.Process()

Summary

Handles IPsec packet processing for inbound and outbound IP packets.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IPSEC_PROCESS) (
  IN EFI_IPSEC_PROTOCOL          *This,
  IN EFI_HANDLE                  NicHandle,
  IN UINT8                       IpVer,
  IN OUT VOID                    *IpHead,
  IN UINT8                       *LastHead,
  IN VOID                        *OptionsBuffer,
  IN UINT32                      OptionsLength,
  IN OUT EFI_IPSEC_FRAGMENT_DATA **FragmentTable,
  IN UINT32                      *FragmentCount,
  IN EFI_IPSEC_TRAFFIC_DIR       TrafficDirection,
  OUT EFI_EVENT                  *RecycleSignal
)
```

Related Definition

```
//*******************************************************
// EFI_IPSEC_FRAGMENT_DATA
//*******************************************************
typedef struct _EFI_IPSEC_FRAGMENT_DATA {
  UINT32                FragmentLength;
  VOID                  *FragmentBuffer;
}   EFI_IPSEC_FRAGMENT_DATA;
```

EFI_IPSEC_FRAGMENT_DATA defines the instances of packet fragments.

This
Pointer to the *EFI_IPSEC_PROTOCOL* instance.

NicHandle
Instance of the network interface.

IpVer
IPV4 or IPV6.

IpHead
Pointer to the IP Header.

LastHead
The protocol of the next layer to be processed by IPsec.

OptionsBuffer
Pointer to the options buffer.

OptionsLength
Length of the options buffer.

FragmentTable
Pointer to a list of fragments.

FragmentCount
Number of fragments.

TrafficDirection
Traffic direction.

RecycleSignal
Event for recycling of resources.

Description

The *EFI_IPSEC_PROCESS* process routine handles each inbound or outbound packet. The behavior is that it can perform one of the following actions: bypass the packet, discard the packet, or protect the packet.

Status Codes Returned

EFI_SUCCESS	The packet was bypassed and all buffers remain the same.
EFI_SUCCESS	The packet was protected.
EFI_ACCESS_DENIED	The packet was discarded.

28.7.12 EFI IPsec2 Protocol

This section provides a detailed description of the *EFI_IPSEC2_PROTOCOL*. This protocol handles IPsec-protected traffic.

28.7.13 EFI_IPSEC2_PROTOCOL

Summary

The *EFI_IPSEC2_PROTOCOL* is used to abstract the ability to deal with the individual packets sent and received by the host and provide packet-level security for IP datagram..

GUID

```
#define EFI_IPSEC2_PROTOCOL_GUID \
  {0xa3979e64, 0xace8, 0x4ddc, \
    {0xbc, 0x07, 0x4d, 0x66, 0xb8, 0xfd, 0x09, 0x77}};
```

Protocol Interface Structure

```
typedef struct _EFI_IPSEC2_PROTOCOL {
  EFI_IPSEC_PROCESSEXT        ProcessExt;
  EFI_EVENT                   DisabledEvent;
  BOOLEAN                     DisabledFlag;
}   EFI_IPSEC2_PROTOCOL;
```

Parameters

ProcessExt
Handle the IPsec message with the extension header processing support.

DisabledEvent
Event signaled when the interface is disabled.

DisabledFlag
State of the interface.

Description

The *EFI_IPSEC2_PROTOCOL* provides the ability for securing IP communications by authenticating and/or encrypting each IP packet in a data stream.

EFI_IPSEC2_PROTOCOL can be consumed by both the IPv4 and IPv6 stack. A user can employ this protocol for IPsec package handling in both IPv4 and IPv6 environment.

28.7.14 EFI_IPSEC2_PROTOCOL.ProcessExt()

Summary

Handles IPsec processing for both inbound and outbound IP packets. Compare with *Process()* in *EFI_IPSEC_PROTOCOL,* this interface has the capability to process Option(Extension Header).

Prototype

```
Typedef
EFI_STATUS
(EFIAPI *EFI_IPSEC_PROCESSEXT) (
  IN EFI_IPSEC2_PROTOCOL           *This,
  IN EFI_HANDLE                    NicHandle,
  IN UINT8                         IpVer,
  IN OUT VOID                      *IpHead,
  IN OUT UINT8                     *LastHead,
  IN OUT VOID                      **OptionsBuffer,
  IN OUT UINT32                    *OptionsLength,
  IN OUT EFI_IPSEC_FRAGMENT_DATA   **FragmentTable,
  IN OUT UINT32                    *FragmentCount,
  IN EFI_IPSEC_TRAFFIC_DIR         TrafficDirection,
  OUT EFI_EVENT                    *RecycleSignal
)
```

Parameters

This
Pointer to the *EFI_IPSEC2_PROTOCOL* instance.

NicHandle
Instance of the network interface.

IpVer
IP version.IPV4 or IPV6.

IpHead
Pointer to the IP Header it is either the *EFI_IP4_HEADER* or *EFI_IP6_HEADER*.On input, it contains the IP header. On output,

1. in tunnel mode and the traffic direction is inbound, the buffer will be reset to zero by IPsec;
2. in tunnel mode and the traffic direction is outbound, the buffer will reset to be the tunnel IP header.
3. in transport mode, the related fielders (like payload length, Next header) in IP header will be modified according to the condition.

LastHead
For IP4, it is the next protocol in IP header. For IP6 it is the Next Header of the last extension header.

OptionsBuffer
On input, it contains the options (extensions header) to be processed by IPsec. On output,

1. in tunnel mode and the traffic direction is outbound, it will be set to *NULL,* and that means this contents was wrapped after inner header and should not be concatenated after tunnel header again;
2. in transport mode and the traffic direction is inbound, if there are IP options (extension headers) protected by IPsec, IPsec will concatenate the those options after the input options (extension headers);
3. on other situations, the output of contents of *OptionsBuffer* might be same with input's. The caller should take the responsibility to free the buffer both on input and on output.

OptionsLength
On input, the input length of the options buffer. On output, the output length of the options buffer.

FragmentTable
Pointer to a list of fragments. On input, these fragments contain the IP payload. On output,

1. in tunnel mode and the traffic direction is inbound, the fragments contain the whole IP payload which is from the IP inner header to the last byte of the packet;*
2. in tunnel mode and the traffic direction is the outbound, the fragments contains the whole encapsulated payload which encapsulates the whole IP payload between the encapsulated header and encapsulated trailer fields.*
3. in transport mode and the traffic direction is inbound, the fragments contains the IP payload which is from the next layer protocol to the last byte of the packet;*
4. in transport mode and the traffic direction is outbound, the fragments contains the whole encapsulated payload which encapsulates the next layer protocol information between the encapsulated header and encapsulated trailer fields.*

FragmentCount
Number of fragments.

TrafficDirection
Traffic direction.

RecycleSignal
Event for recycling of resources.

Description

The *EFI_IPSEC_PROCESSEXT* process routine handles each inbound or outbound packet with the support of options (extension headers) processing. The behavior is that it can perform one of the following actions: bypass the packet, discard the packet, or protect the packet.

Status Codes Returned

EFI_SUCCESS	The packet was bypassed and all buffers remain the same.
EFI_SUCCESS	The packet was processed by IPsec successfully.
EFI_ACCESS_DENIED	The packet was discarded.
EFI_NOT_READY	The IKE negotiation is invoked and the packet was discarded.
EFI_INVALID_PARAMETER	One of more of following are **TRUE** If *OptionsBuffer* is **NULL;** If *OptionsLength* is **NULL;** If *FragmentTable* is **NULL;** If *FragmentCount* is **NULL;**

28.8 Network Protocol - EFI FTP Protocol

This section defines the EFI FTPv4 (File Transfer Protocol version 4) Protocol that interfaces over EFI FTPv4 Protocol

28.8.1 EFI_FTP4_SERVICE_BINDING_PROTOCOL Summary

Summary

The *EFI_FTP4_SERVICE_BINDING_PROTOCOL* is used to locate communication devices that are supported by an EFI FTPv4 Protocol driver and to create and destroy instances of the EFI FTPv4 Protocol child protocol driver that can use the underlying communication device.

GUID

```
#define EFI_FTP4_SERVICE_BINDING_PROTOCOL_GUID \
  {0xfaaecb1, 0x226e, 0x4782,\
    {0xaa, 0xce, 0x7d, 0xb9, 0xbc, 0xbf, 0x4d, 0xaf}}
```

Description

A network application or driver that requires FTPv4 I/O services can use one of the protocol handler services, such as *BS->LocateHandleBuffer(),* to search for devices that publish an EFI FTPv4 Service Binding Protocol GUID. Each device with a published EFI FTPv4 Service Binding Protocol GUID supports the EFI FTPv4 Protocol service and may be available for use.

After a successful call to the *EFI_FTP4_SERVICE_BINDING_PROTOCOL.CreateChild()* function, the newly created child EFI FTPv4 Protocol driver instance is in an unconfigured state; it is not ready to transfer data.

Before a network application terminates execution, every successful call to the *EFI_FTP4_SERVICE_BINDING_PROTOCOL.CreateChild()* function must be matched with a call to the *EFI_FTP4_SERVICE_BINDING_PROTOCOL.DestroyChild()* function.

Each instance of the EFI FTPv4 Protocol driver can support one file transfer operation at a time. To download two files at the same time, two instances of the EFI FTPv4 Protocol driver will need to be created.

NOTE: *Byte Order: f not specifically specified, the IP addresses used in the* EFI_FTP4_PROTOCOL *are in network byte order and the ports are in host byte order*.

28.8.2 EFI_FTP4_PROTOCOL

Summary

The EFI FTPv4 Protocol provides basic services for client-side FTP (File Transfer Protocol) operations.

GUID

```
#define EFI_FTP4_PROTOCOL_GUID \
  {0xeb338826, 0x681b, 0x4295,\
    {0xb3, 0x56, 0x2b, 0x36, 0x4c, 0x75, 0x7b, 0x09}}
```

Protocol Interface Structure

```
typedef struct _EFI_FTP4_PROTOCOL {
  EFI_FTP4_GET_MODE_DATA      GetModeData;
  EFI_FTP4_CONNECT            Connect;
  EFI_FTP4_CLOSE              Close;
```

(continues on next page)

(continued from previous page)

```
  EFI_FTP4_CONFIGURE          Configure;
  EFI_FTP4_READ_FILE          ReadFile;
  EFI_FTP4_WRITE_FILE         WriteFile;
  EFI_FTP4_READ_DIRECTORY     ReadDirectory;
  EFI_FTP4_POLL               Poll;
} EFI_FTP4_PROTOCOL;
```

Parameters

GetModeData
Reads the current operational settings. See the *GetModeData()* function description.

Connect
Establish control connection with the FTP server by using the TELNET protocol according to FTP protocol definition. See the *Connect()* function description

Close
Gracefully disconnecting a FTP control connection This function is a nonblocking operation. See the *Close()* function description.

Configure
Sets and clears operational parameters for an FTP child driver. See the *Configure()* function description.

ReadFile
Downloads a file from an FTPv4 server. See the *ReadFile()* function description.

WriteFile
Uploads a file to an FTPv4 server. This function may be unsupported in some EFI implementations. See the *WriteFile()* function description.

ReadDirectory
Download a related file "directory" from an FTPv4 server. This function may be unsupported in some implementations. See the *ReadDirectory()* function description.

Poll
Polls for incoming data packets and processes outgoing data packets. See the *Poll()* function description.

28.8.3 EFI_FTP4_PROTOCOL.GetModeData()

Summary

Gets the current operational settings.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_FTP4_GET_MODE_DATA)(
  IN EFI_FTP4_PROTOCOL              *This,
  OUT EFI_FTP4_CONFIG_DATA          *ModeData
  );
```

Parameters

This
Pointer to the *EFI_FTP4_PROTOCOL* instance.

ModeData
Pointer to storage for the EFI FTPv4 Protocol driver mode data. Type *EFI_FTP4_CONFIG_DATA* is defined in "Related Definitions" below. The string buffers for Username and Password in *EFI_FTP4_CONFIG_DATA* are allocated by the function, and the caller should take the responsibility to free the buffer later.

Description

The GetModeData() function reads the current operational settings of this EFI FTPv4 Protocol driver instance. *EFI_FTP4_CONFIG_DATA* is defined in the *EFI_FTP4_PROTOCOL.Configure*.

Status Codes Returned

EFI_SUCCESS	This function is called successfully.
EFI_INVALID_PARAMETER	One or more of the following are **TRUE**: • *This* is **NULL**. • *ModeData* is **NULL**.
EFI_NOT_STARTED	The EFI FTPv4 Protocol driver has not been started.
EFI_OUT_OF_RESOURCES	Could not allocate enough resource to finish the operation.
EFI_DEVICE_ERROR	An unexpected system or network error occurred.

28.8.4 EFI_FTP4_PROTOCOL.Connect()

Summary

Initiate a FTP connection request to establish a control connection with FTP server

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_FTP4_CONNECT) (
  IN EFI_FTP4_PROTOCOL           *This,
  IN EFI_FTP4_CONNECTION_TOKEN   *Token
  );
```

Parameters

This
Pointer to the *EFI_FTP4_PROTOCOL* instance.

Token
Pointer to the token used to establish control connection.

Related Definition

```
//*******************************************************
// EFI_FTP4_CONNECTION_TOKEN
//*******************************************************
typedef struct {
  EFI_EVENT               Event;
  EFI_STATUS              Status;
}   EFI_FTP4_CONNECTION_TOKEN;
```

Event

The Event to signal after the connection is established and *Status* field is updated by the EFI FTP v4 Protocol driver. The type of *Event* must be *EVENT_NOTIFY_SIGNAL,* and its Task Priority Level (TPL) must be lower than or equal to *TPL_CALLBACK*. If it is set to NULL, this function will not return until the function completes

Status

The variable to receive the result of the completed operation.

Status Codes Returned

EFI_SUCCESS	The FTP connection is established successfully.
EFI_ACCESS_DENIED	The FTP server denied the access the user's request to access it.
EFI_CONNECTION_RESET	The connect fails because the connection is reset either by instance itself or communication peer.
EFI_TIMEOUT	The connection establishment timer expired and no more specific information is available.
EFI_NETWORK_UNREACHABLE	The active open fails because an ICMP network unreachable error is received.
EFI_HOST_UNREACHABLE	The active open fails because an ICMP host unreachable error is received.
EFI_PROTOCOL_UNREACHABLE	The active open fails because an ICMP protocol unreachable error is received.
EFI_PORT_UNREACHABLE	The connection establishment timer times out and an ICMP port unreachable error is received.
EFI_ICMP_ERROR	The connection establishment timer timeout and some other ICMP error is received.
EFI_DEVICE_ERROR	An unexpected system or network error occurred.

Description

The Connect() function will initiate a connection request to the remote FTP server with the corresponding connection token. If this function returns EFI_SUCCESS, the connection sequence is initiated successfully. If the connection succeeds or failed due to any error, the Token->Event will be signaled and Token->Status will be updated accordingly.

Status Codes Returned

EFI_SUCCESS	The connection sequence is successfully initiated.
EFI_INVALID_PARAMETER	One or more of the following are **TRUE:** [2] This is **NULL.** [2] Token is **NULL.** [2] Token->Event is **NULL.**
EFI_NOT_STARTED	The EFI FTPv4 Protocol driver has not been started.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_OUT_OF_RESOURCES	Could not allocate enough resource to finish the operation.
EFI_DEVICE_ERROR	An unexpected system or network error occurred.

28.8.5 EFI_FTP4_PROTOCOL.Close()

Summary

Disconnecting a FTP connection gracefully.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_FTP4_CLOSE)(
  IN EFI_FTP4_PROTOCOL           *This,
  IN EFI_FTP4_CONNECTION_TOKEN   *Token
  );
```

Parameters

This
Pointer to the *EFI_FTP4_PROTOCOL* instance.

Token
Pointer to the token used to close control connection.

Description

The *Close()* function will initiate a close request to the remote FTP server with the corresponding connection token. If this function returns *EFI_SUCCESS,* the control connection with the remote FTP server is closed.

Status Codes Returned

EFI_SUCCESS	The close request is successfully initiated.
EFI_INVALID_PARAMETER	One or more of the following are **TRUE**: • This is **NULL**. • ConnectionToken is **NULL**. • ConnectionToken->Event is **NULL**.
EFI_NOT_STARTED	The EFI FTPv4 Protocol driver has not been started.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_OUT_OF_RESOURCES	Could not allocate enough resource to finish the operation.
EFI_DEVICE_ERROR	An unexpected system or network error occurred.

28.8.6 EFI_FTP4_PROTOCOL.Configure()

Summary

Sets or clears the operational parameters for the FTP child driver.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_FTP4_CONFIGURE) (
  IN EFI_FTP4_PROTOCOL           *This,
  IN EFI_FTP4_CONFIG_DATA        *FtpConfigData OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_FTP4_PROTOCOL* instance.

FtpConfigData
Pointer to configuration data that will be assigned to the FTP child driver instance. If *NULL,* the FTP child driver instance is reset to startup defaults and all pending transmit and receive requests are flushed.

Related Definition

```
//*******************************************************
// EFI_FTP4_CONFIG_DATA
//*******************************************************
typedef struct {
  UINT8              *Username;
  UINT8              *Password;
  BOOLEAN            Active;
  BOOLEAN            UseDefaultSetting;
  EFI_IPv4_ADDRESS   StationIp;
  EFI_IPv4_ADDRESS   SubnetMask;
  EFI_IPv4_ADDRESS   GatewayIp;
  EFI_IPv4_ADDRESS   ServerIp;
  UINT16             ServerPort;
  UINT16             AltDataPort;
  UINT8              RepType;
  UINT8              FileStruct;
  UINT8              TransMode;
}   EFI_FTP4_CONFIG_DATA;
```

Username
Pointer to a ASCII string that contains user name. The caller is responsible for freeing *Username* after *GetModeData()* is called.

Password
Pointer to a ASCII string that contains password. The caller is responsible for freeing *Password* after *GetModeData()* is called.

Active
Set it to **TRUE** to initiate an active data connection. Set it to **FALSE** to initiate a passive data connection.

UseDefaultSetting
Boolean value indicating if default network setting used.

StationIp
IP address of station if *UseDefaultSetting* is **FALSE**.

SubnetMask
Subnet mask of station if *UseDefaultSetting* is **FALSE**.

GatewayIp
IP address of gateway if *UseDefaultSetting* is **FALSE**.

ServerIp
IP address of FTPv4 server.

ServerPort
FTPv4 server port number of control connection, and the default value is 21 as convention.

ALtDataPort
FTPv4 server port number of data connection. If it is zero, use (*ServerPort* - 1) by convention.

RepType
A byte indicate the representation type. The right 4 bit is used for first parameter, the left 4 bit is use for second parameter

- For the first parameter, 0x0 = image, 0x1 = EBCDIC, 0x2 = ASCII, 0x3 = local
- For the second parameter, 0x0 = Non-print, 0x1 = Telnet format effectors, 0x2 = Carriage Control
- If it is a local type, the second parameter is the local byte byte size.
- If it is a image type, the second parameter is undefined.

FileStruct
Defines the file structure in FTP used. 0x00 = file, 0x01 = record, 0x02 = page

TransMode
Defines the transfer mode used in FTP. 0x00 = stream, 0x01 = Block, 0x02 = Compressed

Description

The *Configure()* function will configure the connected FTP session with the configuration setting specified in *FtpConfigData*. The configuration data can be reset by calling *Configure()* with *FtpConfigData* set to **NULL**.

Status Codes Returned

EFI_SUCCESS	The FTPv4 driver was configured successfully.
EFI_INVALID_PARAMETER	One or more following conditions are **TRUE:** • *This* is **NULL**. • *FtpConfigData.RepType* is invalid. • *FtpConfigData.FileStruct* in invalid. • *FtpConfigData.TransMode* is invalid. • IP address in *FtpConfigData* is invalid.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) has not finished yet.
EFI_UNSUPPORTED	One or more of the configuration parameters are not supported by this implementation.
EFI_OUT_OF_RESOURCES	The EFI FTPv4 Protocol driver instance data could not be allocated.

continues on next page

Table 28.56 – continued from previous page

EFI_DEVICE_ERROR	An unexpected system or network error occurred. The EFI FTPv4 Protocol driver instance is not configured.

28.8.7 EFI_FTP4_PROTOCOL.ReadFile()

Summary

Downloads a file from an FTPv4 server.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_FTP4_READ_FILE)(
  IN EFI_FTP4_PROTOCOL          *This,
  IN EFI_FTP4_COMMAND_TOKEN     *Token
  );
```

Parameters

This
Pointer to the *EFI_FTP4_PROTOCOL* instance.

Token
Pointer to the token structure to provide the parameters that are used in this operation. Type *EFI_FTP4_COMMAND_TOKEN* is defined in "Related Definitions" below.

Related Definition

```
//*******************************************************
// EFI_FTP4_COMMAND_TOKEN
//*******************************************************
typedef struct {
  EFI_EVENT                Event;
  UINT8                    *Pathname;
  UINT64                   DataBufferSize;
  VOID                     *DataBuffer;
  EFI_FTP4_DATA_CALLBACK   DataCallback;
  VOID                     *Context;
  EFI_STATUS               Status;
}   EFI_FTP4_COMMAND_TOKEN;
```

Event
The *Event* to signal after request is finished and *Status* field is updated by the EFI FTP v4 Protocol driver. The type of *Event* must be *EVT_NOTIFY_SIGNAL,* and its Task Priority Level (TPL) must be lower than or equal to *TPL_CALLBACK*. If it is set to *NULL,* related function must wait until the function completes

Pathname
Pointer to a null-terminated ASCII name string.

DataBuffersize
The size of data buffer in bytes

DataBuffer
Pointer to the data buffer. Data downloaded from FTP server through connection is downloaded here.

DataCallback
Pointer to a callback function. If it is receiving function that leads to inbound data, the callback function is called when databuffer is full. Then, old data in the data buffer should be flushed and new data is stored from the beginning of data buffer. If it is a transmit function that lead to outbound data and *DataBufferSize* of *Data* in *DataBuffer* has been transmitted, this callback function is called to supply additional data to be transmitted. The size of additional data to be transmitted is indicated in *DataBufferSize,* again. If there is no data remained, *DataBufferSize* should be set to 0

Context
Pointer to the parameter for *DataCallback*.

Status
The variable to receive the result of the completed operation.

EFI_SUCCESS — The FTP command is completed successfully.

EFI_ACCESS_DENIED — The FTP server denied the access to the requested file.

EFI_CONNECTION_RESET — The connect fails because the connection is reset either by instance itself or communication peer.

EFI_TIMEOUT — The connection establishment timer expired and no more specific information is available.

EFI_NETWORK_UNREACHABLE — The active open fails because an ICMP network unreachable error is received.

EFI_HOST_UNREACHABLE — The active open fails because an ICMP host unreachable error is received.

EFI_PROTOCOL_UNREACHABLE — The active open fails because an ICMP protocol unreachable error is received.

EFI_PORT_UNREACHABLE — The connection establishment timer times out and an ICMP port unreachable error is received.

EFI_ICMP_ERROR — The connection establishment timer timeout and some other ICMP error is received.

EFI_DEVICE_ERROR — An unexpected system or network error occurred.

Related Definition

```
//*******************************************************
// EFI_FTP4_DATA_CALLBACK
//*******************************************************
```

(continues on next page)

(continued from previous page)

```
typedef
EFI_STATUS
(EFIAPI *EFI_FTP4_DATA_CALLBACK)(
  IN EFI_FTP4_COMMAND_TOKEN   *Token,
  IN EFI_FTP4_PROTOCOL        *This,
  );
```

This
Pointer to the EFI_FTP4_PROTOCOL instance.

Token
Pointer to the token structure to provide the parameters that are used in this operation. Type EFI_FTP4_COMMAND_TOKEN is defined in "Related Definitions" above.

Description

The ReadFile() function is used to initialize and start an FTPv4 download process and optionally wait for completion. When the download operation completes, whether successfully or not, the Token.Status field is updated by the EFI FTPv4 Protocol driver and then Token.Event is signaled (if it is not NULL).

Data will be downloaded from the FTPv4 server into Token.DataBuffer. If the file size is larger than Token.DataBufferSize, Token.DataCallback will be called to allow for processing data and then new data will be placed at the beginning of Token.DataBuffer.

Status Codes Returned

EFI_SUCCESS	The data file is being downloaded successfully.
EFI_INVALID_PARAMETER	One or more of the parameters is not valid. • This is **NULL**. • Token is **NULL**. • Token. Pathname is **NULL**. • Token. DataBuffer is **NULL**. • Token. DataBufferSize is 0.
EFI_NOT_STARTED	The EFI FTPv4 Protocol driver has not been started.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.
EFI_DEVICE_ERROR	An unexpected network error or system error occurred.

28.8.8 EFI_FTP4_PROTOCOL.WriteFile()

Summary

Uploads a file from an FTPv4 server.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_FTP4_WRITE_FILE)(
```

(continues on next page)

(continued from previous page)

```
  IN EFI_FTP4_PROTOCOL           *This,
  IN EFI_FTP4_COMMAND_TOKEN      *Token
  );
```

Parameters

This
Pointer to the *EFI_FTP4_PROTOCOL* instance.

Token
Pointer to the token structure to provide the parameters that are used in this operation. Type *EFI_FTP4_COMMAND_TOKEN* is defined in " *EFI_FTP4_READ_FILE* " .

Description

The *WriteFile()* function is used to initialize and start an FTPv4 upload process and optionally wait for completion. When the upload operation completes, whether successfully or not, the *Token.Status* field is updated by the EFI FTPv4 Protocol driver and then *Token.Event* is signaled (if it is not **NULL**).

Data to be uploaded to server is stored into *Token.DataBuffer* . *Token.DataBufferSize* is the number bytes to be transferred. If the file size is larger than *Token.DataBufferSize, Token.DataCallback* will be called to allow for processing data and then new data will be placed at the beginning of *Token.DataBuffer*. *Token.DataBufferSize* is updated to reflect the actual number of bytes to be transferred. *Token.DataBufferSize* is set to 0 by the call back to indicate the completion of data transfer.

Status Codes Returned

EFI_SUCCESS	The data file is being uploaded successfully.
EFI_UNSUPPORTED	The operation is not supported by this implementation.
EFI_INVALID_PARAMETER	One or more of the parameters is not valid. • This is **NULL**. • Token is **NULL**. • Token. Pathname is **NULL**. • Token. DataBuffer is **NULL**. • Token. DataBufferSize is 0.
EFI_NOT_STARTED	The EFI FTPv4 Protocol driver has not been started.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.
EFI_DEVICE_ERROR	An unexpected network error or system error occurred.

28.8.9 EFI_FTP4_PROTOCOL.ReadDirectory()

Summary

Download a data file "directory" from a FTPv4 server. May be unsupported in some EFI implementations.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_FTP4_READ_DIRECTORY) (
  IN EFI_FTP4_PROTOCOL              *This,
  IN EFI_FTP4_COMMAND_TOKEN         *Token
  );
```

Parameters

This
Pointer to the *EFI_FTP4_PROTOCOL* instance.

Token
Pointer to the token structure to provide the parameters that are used in this operation. Type *EFI_FTP4_COMMAND_TOKEN* is defined in " *EFI_FTP4_READ_FILE* " .

Description

The *ReadDirectory()* function is used to return a list of files on the FTPv4 server that logically (or operationally) related to *Token.Pathname,* and optionally wait for completion. When the download operation completes, whether successfully or not, the *Token.Status* field is updated by the EFI FTPv4 Protocol driver and then *Token.Event* is signaled (if it is not **NULL**).

Data will be downloaded from the FTPv4 server into *Token.DataBuffer*. If the file size is larger than *Token.DataBufferSize, Token.DataCallback* will be called to allow for processing data and then new data will be placed at the beginning of *Token.DataBuffer*.

Status Codes Returned

EFI_SUCCESS	The file list information is being downloaded successfully.
EFI_UNSUPPORTED	The operation is not supported by this implementation.
EFI_INVALID_PARAMETER	One or more of the parameters is not valid. • *This* is **NULL**. • *Token* is **NULL.** • *Token*. *DataBuffer* is **NULL.** • *Token*. *DataBufferSize* is 0.
EFI_NOT_STARTED	The EFI FTPv4 Protocol driver has not been started.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.
EFI_DEVICE_ERROR	An unexpected network error or system error occurred.

28.8.10 EFI_FTP4_PROTOCOL.Poll()

Summary

Polls for incoming data packets and processes outgoing data packets.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_FTP4_POLL) (
  IN EFI_FTP4_PROTOCOL     *This
  );
```

Parameters

This
Pointer to the *EFI_FTP4_PROTOCOL* instance.

Description

The *Poll()* function can be used by network drivers and applications to increase the rate that data packets are moved between the communications device and the transmit and receive queues.

In some systems, the periodic timer event in the managed network driver may not poll the underlying communications device fast enough to transmit and/or receive all data packets without missing incoming packets or dropping outgoing packets. Drivers and applications that are experiencing packet loss should try calling the *Poll()* function more often.

Status Codes Returned

EFI_SUCCESS	Incoming or outgoing data was processed.
EFI_NOT_STARTED	This EFI FTPv4 Protocol instance has not been started.
EFI_INVALID_PARAMETER	This is **NULL**.
EFI_DEVICE_ERROR	An unexpected system or network error occurred.
EFI_TIMEOUT	Data was dropped out of the transmit and/or receive queue. Consider increasing the polling rate.

28.9 EFI TLS Protocols

28.9.1 EFI TLS Service Binding Protocol

28.9.1.1 EFI_TLS_SERVICE_BINDING_PROTOCOL

Summary

The EFI TLS Service Binding Protocol is used to locate EFI TLS Protocol drivers to create and destroy child of the driver to communicate with other host using TLS protocol.

GUID

```
#define EFI_TLS_SERVICE_BINDING_PROTOCOL_GUID \
  { \
    0x952cb795, 0xff36, 0x48cf, 0xa2, 0x49, 0x4d, 0xf4, 0x86, 0xd6, 0xab, 0x8d \
}
```

Description

The TLS consumer need locate EFI_TLS_SERVICE_BINDING_PROTOCOL and call CreateChild() to create a new child of EFI_TLS_PROTOCOL and EFI_TLS_CONFIGURATION_PROTOCOL instance. Then usE EFI_TLS_CONFIGURATION_PROTOCOL to set TLS configuration data, and use EFI_TLS_PROTOCOL to start TLS session. After use, the TLS consumer needs to call DestroyChild() to destroy it.

28.9.2 EFI TLS Protocol

28.9.2.1 EFI_TLS_PROTOCOL

Summary

This protocol provides the ability to manage TLS session.

GUID

```
#define EFI_TLS_PROTOCOL_GUID \
  { 0xca959f, 0x6cfa, 0x4db1, \
    {0x95, 0xbc, 0xe4, 0x6c, 0x47, 0x51, 0x43, 0x90 }}
```

Protocol Interface Structure

```
typedef struct _EFI_TLS_PROTOCOL {
  EFI_TLS_SET_SESSION_DATA          SetSessionData;
  EFI_TLS_GET_SESSION_DATA          GetSessionData;
  EFI_TLS_BUILD_RESPONSE_PACKET     BuildResponsePacket;
  EFI_TLS_PROCESS_PACKET            ProcessPacket;
  }   EFI_TLS_PROTOCOL;
```

Parameters

SetSessionData
Set TLS session data. See the *SetSessionData ()* function description.

GetSessionData
Get TLS session data. See the *GetSessionData ()* function description.

BuildResponsePacket
Build response packet according to TLS state machine. This function is only valid for alert, handshake and change_cipher_spec content type. See the *BuildResponsePacket ()* function description.

ProcessPacket
Decrypt or encrypt TLS packet during session. This function is only valid after session connected and for application_data content type. See the *ProcessPacket ()* function description.

Description

The *EFI_TLS_PROTOCOL* is used to create, destroy and manage TLS session. For detail of TLS, please refer to TLS related RFC.

28.9.3 EFI_TLS_PROTOCOL.SetSessionData ()

Summary

Set TLS session data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TLS_SET_SESSION_DATA)(
  IN EFI_TLS_PROTOCOL               *This,
  IN EFI_TLS_SESSION_DATA_TYPE      DataType,
  IN VOID                           *Data,
  IN UINTN                          DataSize
  );
```

Parameters

This
Pointer to the *EFI_TLS_PROTOCOL* instance.

DataType
TLS session data type. See *EFI_TLS_SESSION_DATA_TYPE*

Data
Pointer to session data.

DataSize
Total size of session data.

Description

The *SetSessionData()* function set data for a new TLS session. All session data should be set before *BuildResponsePacket()* invoked.

Related Definition

```
//*******************************************************
// EFI_TLS_SESSION_DATA_TYPE
//*******************************************************
typedef enum {
  EfiTlsVersion,
  EfiTlsConnectionEnd,
  EfiTlsCipherList,
  EfiTlsCompressionMethod,
  EfiTlsExtensionData,
  EfiTlsVerifyMethod,
  EfiTlsSessionID,
  EfiTlsSessionState,
  EfiTlsClientRandom,
  EfiTlsServerRandom,
  EfiTlsKeyMaterial,
  EfiTlsVerifyHost,
  EfiTlsSessionDataTypeMaximum
} EFI_TLS_SESSION_DATA_TYPE;
```

EfiTlsVersion
TLS session Version. The corresponding *Data* is of type *EFI_TLS_VERSION*.

EfiTlsConnectionEnd
TLS session as client or as server. The corresponding *Data* is of *EFI_TLS_CONNECTION_END*.

EfiTlsCipherList
A priority list of preferred algorithms for the TLS session. The corresponding Data is a list of EFI_TLS_CIPHER.

EfiTlsCompressionMethod
TLS session compression method. The corresponding Data is of type *EFI_TLS_COMPRESSION*.

EfiTlsExtensionData
TLS session extension data. The corresponding *Data* is a list of type *EFI_TLS_EXTENDION*.

EfiTlsVerifyMethod
TLS session verify method. The corresponding *Data* is of type *EFI_TLS_VERIFY*.

EfiTlsSessionID
TLS session data session ID. For *SetSessionData(),* it is TLS session ID used for session resumption. For *GetSessionData(),* it is the TLS session ID used for current session. The corresponding *Data* is of type *EFI_TLS_SESSION_ID*.

EfiTlsSessionState
TLS session data session state. The corresponding *Data* is of type *EFI_TLS_SESSION_STATE*.

EfiTlsClientRandom
TLS session data client random. The corresponding *Data* is of type *EFI_TLS_RANDOM*.

EfiTlsServerRandom
TLS session data server random. The corresponding *Data* is of type *EFI_TLS_RANDOM*.

EfiTlsKeyMaterial
TLS session data key material. The corresponding *Data* is of type *EFI_TLS_MASTER_SECRET*.

EfiTlsVerifyHost
TLS session hostname for validation which is used to verify whether the name within the peer certificate matches a given host name. This parameter is invalid when *EfiTlsVerifyMethod* is *EFI_TLS_VERIFY_NONE*. The corresponding Data is of type *EFI_TLS_VERIFY_HOST*.

```
//*******************************************************
// EFI_TLS_VERSION
//*******************************************************
typedef struct {
  UINT8        Major;
  UINT8        Minor;
}  EFI_TLS_VERSION;
```

Note
The TLS version definition is from latest TLS RFC.

```
//*******************************************************
// EFI_TLS_CONNECTION_END
//*******************************************************
typedef enum {
  EfiTlsClient,
  EfiTlsServer,
  } EFI_TLS_CONNECTION_END;
```

TLS connection end is to define TLS session as client or as server.

```
//*******************************************************
// EFI_TLS_CIPHER
//*******************************************************
typedef struct {
 UINT8             Data1;
  UINT8             Data2;
} EFI_TLS_CIPHER;
```

NOTE: *The definition of EFI_TLS_CIPHER is from RFC 5246 A.4.1.Hello Messages. The value of EFI_TLS_CIPHER is from TLS Cipher Suite Registry of IANA.*

```
//*******************************************************
// EFI_TLS_COMPRESSION
//*******************************************************
typedef UINT8 EFI_TLS_COMPRESSION;
```

NOTE: *The value of EFI_TLS_COMPRESSION definition is from RFC 3749.*

```
//*******************************************************
// EFI_TLS_EXTENSION
//*******************************************************
typedef struct {
  UINT16       ExtensionType;
  UINT16       Length;
  UINT8        Data [];
  } EFI_TLS_EXTENSION;
```

NOTE: *The definition of* EFI_TLS_EXTENSION *is from RFC 5246 A.4.1. Hello Messages.*

```
//*******************************************************
// EFI_TLS_VERIFY
//*******************************************************
typedef UINT32 EFI_TLS_VERIFY;
#define EFI_TLS_VERIFY_NONE                  0x0
#define EFI_TLS_VERIFY_PEER                  0x1
#define EFI_TLS_VERIFY_FAIL_IF_NO_PEER_CERT  0x2
#define EFI_TLS_VERIFY_CLIENT_ONCE           0x4
```

The consumer needs to use either *EFI_TLS_VERIFY_NONE* or *EFI_TLS_VERIFY_PEER*. *EFI_TLS_VERIFY_FAIL_IF_NO_PEER_CERT* and *EFI_TLS_VERIFY_CLIENT_ONCE* can be ORed with *EFI_TLS_VERIFY_PEER*. *EFI_TLS_VERIFY_FAIL_IF_NO_PEER_CERT* is only meaningful in the server mode, which means the TLS session will fail if the client certificate is absent. *EFI_TLS_VERIFY_CLIENT_ONCE* means the TLS session only verifies the client once, and doesn't request a certificate during re-negotiation.

```
//*******************************************************
// EFI_TLS_VERIFY_HOST
//*******************************************************
typedef struct {
  EFI_TLS_VERIFY_HOST_FLAG         Flags;
  CHAR8                            *HostName;
}   EFI_TLS_VERIFY_HOST;
```

Flags

The host name validation flags. The flags arguments can be ORed.

HostName
The specified host name to be verified.

```
//*******************************************************
// EFI_TLS_VERIFY_HOST_FLAG
//*******************************************************
typedef UINT32 EFI_TLS_VERIFY_HOST_FLAG;
#define EFI_TLS_VERIFY_FLAG_NONE                    0x00
#define EFI_TLS_VERIFY_FLAG_ALWAYS_CHECK_SUBJECT    0x01
#define EFI_TLS_VERIFY_FLAG_NO_WILDCARDS            0x02
#define EFI_TLS_VERIFY_FLAG_NO_PARTIAL_WILDCARDS    0x04
#define EFI_TLS_VERIFY_FLAG_MULTI_LABEL_WILDCARDS   0x08
#define EFI_TLS_VERIFY_FLAG_SINGLE_LABEL_SUBDOMAINS 0x10
#define EFI_TLS_VERIFY_FLAG_NEVER_CHECK_SUBJECT     0x20
```

EFI_TLS_VERIFY_FLAG_NONE means no additional flags set for hostname validation. Wildcards are supported and they match only in the left-most label.

EFI_TLS_VERIFY_FLAG_ALWAYS_CHECK_SUBJECT means to always check the Subject Distinguished Name (DN) in the peer certificate even if the certificate contains Subject Alternative Name (SAN).

EFI_TLS_VERIFY_FLAG_NO_WILDCARDS means to disable the match of all wildcards.

EFI_TLS_VERIFY_FLAG_NO_PARTIAL_WILDCARDS means to disable the "*" as wildcard in labels that have a prefix or suffix (e.g. "www*" or "*www").

EFI_TLS_VERIFY_FLAG_MULTI_LABEL_WILDCARDS allows the "*" to match more than one labels. Otherwise, only matches a single label.

EFI_TLS_VERIFY_FLAG_SINGLE_LABEL_SUBDOMAINS restricts to only match direct child sub-domains which start with ".". For example, a name of ".example.com" would match "www.example.com" with this flag, but would not match "www.sub.example.com".

EFI_TLS_VERIFY_FLAG_NEVER_CHECK_SUBJECT means never check the Subject Distinguished Name (DN) even there is no Subject Alternative Name (SAN) in the certificate.

If both *EFI_TLS_VERIFY_FLAG_ALWAYS_CHECK_SUBJECT* and *EFI_TLS_VERIFY_FLAG_NEVER_CHECK_SUBJECT* are specified, *EFI_INVALID_PARAMETER* will be returned. If *EFI_TLS_VERIFY_FLAG_NO_WILDCARDS* is set with *EFI_TLS_VERIFY_FLAG_NO_PARTIAL_WILDCARDS* or *EFI_TLS_VERIFY_FLAG_MULTI_LABEL_WILDCARDS, EFI_INVALID_PARAMETER* will be returned.

```
//*******************************************************
// EFI_TLS_RANDOM
//*******************************************************
typedef struct {
  UINT32          GmtUnixTime;
  UINT8           RandomBytes[28];
}   EFI_TLS_RANDOM;
```

NOTE: *The definition of* EFI_TLS_RANDOM *is from RFC 5246 A.4.1. Hello Messages.*

```
//*******************************************************
// EFI_TLS_MASTER_SECRET
//*******************************************************
typedef struct {
  UINT8 *Data[48];
}   EFI_TLS_MASTER_SECRET;
```

NOTE: *The definition of* EFI_TLS_MASTER_SECRETE *is from RFC 5246 8.1. Computing the Master Secret.*

```
//*******************************************************
// EFI_TLS_SESSION_ID
//*******************************************************
#define MAX_TLS_SESSION_ID_LENGTH 32
typedef struct {
  UINT16             Length;
  UINT8              Data[MAX_TLS_SESSION_ID_LENGTH];
}   EFI_TLS_SESSION_ID;
```

NOTE: *The definition of* EFI_TLS_SESSION_ID *is from RFC 5246 A.4.1. Hello Messages.*

```
//*******************************************************
// EFI_TLS_SESSION_STATE
//*******************************************************
Typedef enum {
  EfiTlsSessionNotStarted,
  EfiTlsSessionHandShaking,
  EfiTlsSessionDataTransferring,
  EfiTlsSessionClosing,
  EfiTlsSessionError,
  EfiTlsSessionStateMaximum
}   EFI_TLS_SESSION_STATE;
```

The definition of *EFI_TLS_SESSION_STATE* is below:

When a new child of TLS protocol is created, the initial state of TLS session is *EfiTlsSessionNotStarted*.

The consumer can call *BuildResponsePacket()* with NULL to get ClientHello to start the TLS session. Then the status is *EfiTlsSessionHandShaking*.

During handshake, the consumer need call *BuildResponsePacket()* with input data from peer, then get response packet and send to peer. After handshake finish, the TLS session status becomes *EfiTlsSessionDataTransferring,* and consume can use *ProcessPacket()* for data transferring.

Finally, if consumer wants to active close TLS session, consumer need call SetSessionData to set TLS session state to *EfiTlsSessionClosing,* and call *BuildResponsePacket()* with NULL to get CloseNotify alert message, and sent it out.

If any error happen during parsing ApplicationData content type, EFI_ABORT will be returned by *ProcessPacket(),* and TLS session state will become *EfiTlsSessionError*. Then consumer need call *BuildResponsePacket()* with NULL to get alert message and sent it out.

Status Codes Returned

EFI_SUCCESS	The TLS session data is set successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *This* is **NULL** • *Data* is **NULL**. • *DataSize* is 0.
EFI_UNSUPPORTED	The *DataType* is unsupported.

continues on next page

Table 28.61 – continued from previous page

EFI_ACCESS_DENIED	If the *DataType* is one of below: • *EfiTlsClientRandom* • *EfiTlsServerRandom* • *EfiTlsKeyMaterial*
EFI_NOT_READY	Current TLS session state is NOT *EfiTlsSessionStateNotStarted*.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.

28.9.4 EFI_TLS_PROTOCOL.GetSessionData ()

Summary

Get TLS session data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TLS_GET_SESSION_DATA)(
  IN EFI_TLS_PROTOCOL            *This,
  IN EFI_TLS_SESSION_DATA_TYPE   DataType,
  IN OUT VOID                    *Data, OPTIONAL
  IN OUT UINTN                   *DataSize
);
```

Parameters

This
Pointer to the *EFI_TLS_PROTOCOL* instance.

DataType
TLS session data type. See EFI_TLS_SESSION_DATA_TYPE

Data
Pointer to session data.

DataSize
Total size of session data. On input, it means the size of *Data* buffer. On output, it means the size of copied *Data* buffer if *EFI_SUCCESS,* and means the size of desired *Data* buffer if *EFI_BUFFER_TOO_SMALL.*

Description

The GetSessionData() function return the TLS session information.

Status Codes Returned

EFI_SUCCESS	The TLS session data is got successfully.

continues on next page

Table 28.62 – continued from previous page

EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • This is NULL. • DataSize is NULL. • Data is NULL if *DataSize is not zero.
EFI_UNSUPPORTED	The DataType is unsupported.
EFI_NOT_FOUND	The TLS session data is not found.
EFI_NOT_READY	The DataType is not ready in current session state.
EFI_BUFFER_TOO_SMALL	The buffer is too small to hold the data.

28.9.5 EFI_TLS_PROTOCOL.BuildResponsePacket ()

Summary

Build response packet according to TLS state machine. This function is only valid for alert, handshake and change_cipher_spec content type.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TLS_BUILD_RESPONSE_PACKET)(
  IN EFI_TLS_PROTOCOL         *This,
  IN UINT8                    *RequestBuffer, OPTIONAL
  IN UINTN                    RequestSize, OPTIONAL
  OUT UINT8                   *Buffer, OPTIONAL
  IN OUT UINTN                *BufferSize
);
```

Parameters

This
Pointer to the *EFI_TLS_PROTOCOL* instance.

RequestBuffer
Pointer to the most recently received TLS packet. NULL means TLS need initiate the TLS session and response packet need to be ClientHello.

RequestSize
Packet size in bytes for the most recently received TLS packet. 0 is only valid when *RequestBuffer* is NULL.

Buffer
Pointer to the buffer to hold the built packet.

BufferSize
Pointer to the buffer size in bytes. On input, it is the buffer size provided by the caller. On output, it is the buffer size in fact needed to contain the packet.

Description

The *BuildResponsePacket()* function builds TLS response packet in response to the TLS request packet specified by *RequestBuffer* and *RequestSize*. If *RequestBuffer* is NULL and *RequestSize* is 0, and TLS session status is *EfiTlsSessionNotStarted,* the TLS session will be initiated and the response packet needs to be ClientHello. If *RequestBuffer* is NULL and *RequestSize* is 0, and TLS session status is *EfiTlsSessionClosing,* the TLS session will be closed and

response packet needs to be CloseNotify. If *RequestBuffer* is NULL and *RequestSize* is 0, and TLS session status is *EfiTlsSessionError,* the TLS session has errors and the response packet needs to be Alert message based on error type.

Status Codes Returned

EFI_SUCCESS	The required TLS packet is built successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *This* is **NULL**. • *RequestBuffer* is **NULL** but *RequestSize* is NOT 0. • *RequestSize* is 0 but *RequestBuffer* is **NOT NULL**. • *BufferSize* is **NULL**. • *Buffer* is **NULL**.if *BufferSize* is not zero.
EFI_BUFFER_TOO_SMALL	*BufferSize* is too small to hold the response packet.
EFI_NOT_READY	Current TLS session state is NOT ready to build *ResponsePacket*.
EFI_ABORTED	Something wrong build response packet.

28.9.6 EFI_TLS_PROTOCOL.ProcessPacket ()

Summary

Decrypt or encrypt TLS packet during session. This function is only valid after session connected and for application_data content type.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TLS_PROCESS_PACKET)(
  IN EFI_TLS_PROTOCOL                *This,
  IN OUT EFI_TLS_FRAGMENT_DATA       **FragmentTable,
  IN UINT32                          *FragmentCount,
  IN EFI_TLS_CRYPT_MODE              CryptMode
  );
```

Parameters

This
Pointer to the *EFI_TLS_PROTOCOL* instance.

FragmentTable
Pointer to a list of fragment. The caller will take responsible to handle the original *FragmentTable* while it may be reallocated in TLS driver. If *CryptMode* is *EfiTlsEncrypt,* on input these fragments contain the TLS header and plain text TLS APP payload; on output these fragments contain the TLS header and cypher text TLS APP payload. If *CryptMode* is *EfiTlsDecrypt,* on input these fragments contain the TLS header and cypher text TLS APP payload; on output these fragments contain the TLS header and plain text TLS APP payload.

FragmentCount
Number of fragment.

CryptMode
Crypt mode.

Description

The *ProcessPacket ()* function process each inbound or outbound TLS APP packet.

Related Definition

```
//*******************************************************
// EFI_TLS_FRAGMENT_DATA
//*******************************************************
typedef struct {
  UINT32       FragmentLength;
  VOID         *FragmentBuffer;
}   EFI_TLS_FRAGMENT_DATA;
```

FragmentLength
Length of data buffer in the fragment.

FragmentBuffer
Pointer to the data buffer in the fragment.

```
//*******************************************************
// EFI_TLS_CRYPT_MODE
//*******************************************************
typedef enum {
  EfiTlsEncrypt,
  EfiTlsDecrypt,
}   EFI_TLS_CRYPT_MODE;
```

EfiTlsEncrypt
Encrypt data provided in the fragment buffers.

EfiTlsDecrypt
Decrypt data provided in the fragment buffers.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *This* is **NULL**. • *FragmentTable* is **NULL**. • *FragmentCount* is **NULL**. • *CryptoMode* is invalid.
EFI_NOT_READY	Current TLS session state is NOT EfiTlsSessionDataTransferring.
EFI_ABORTED	Something wrong decryption the message. TLS session status will become EfiTlsSessionError. The caller need call *BuildResponsePacket()* to generate Error Alert message and send it out.
EFI_OUT_OF_RESOURCES	No enough resource to finish the operation.

28.9.7 EFI TLS Configuration Protocol

28.9.7.1 EFI_TLS_CONFIGURATION_PROTOCOL

Summary

This protocol provides a way to set and get TLS configuration.

GUID

```
#define EFI_TLS_CONFIGURATION_PROTOCOL_GUID \
  { 0x1682fe44, 0xbd7a, 0x4407, \
    {0xb7, 0xc7, 0xdc, 0xa3, 0x7c, 0xa3, 0x92, 0x2d }}
```

Protocol Interface Structure

```
typedef struct _EFI_TLS_CONFIGURATION_PROTOCOL {
  EFI_TLS_CONFIGURATION_SET_DATA          SetData;
  EFI_TLS_CONFIGURATION_GET_DATA          GetData;
}   EFI_TLS_CONFIGURATION_PROTOCOL;
```

Parameters

SetData
Set TLS configuration data. See the *SetData()* function description.

GetData
Get TLS configuration data. See the *GetData()* function description.

Description

The *EFI_TLS_CONFIGURATION_PROTOCOL* is designed to provide a way to set and get TLS configuration, such as Certificate, private key file.

28.9.8 EFI_TLS_CONFIGURATION_PROTOCOL.SetData()

Summary

Set TLS configuration data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TLS_CONFIGURATION_SET_DATA)(
  IN EFI_TLS_CONFIGURATION_PROTOCOL       *This,
  IN EFI_TLS_CONFIG_DATA_TYPE              DataType,
  IN VOID                                 *Data,
  IN UINTN                                DataSize
  );
```

Parameters

This
Pointer to the *EFI_TLS_CONFIGURATION_PROTOCOL* instance.

DataType
Configuration data type. See EFI_TLS_CONFIG_DATA_TYPE

Data
Pointer to configuration data.

DataSize
Total size of configuration data.

Description

The *SetData()* function sets TLS configuration to non-volatile storage or volatile storage.

Related Definition

```
//*******************************************************
// EFI_TLS_CONFIG_DATA_TYPE
//*******************************************************
typedef enum {
  EfiTlsConfigDataTypeHostPublicCert,
  EfiTlsConfigDataTypeHostPrivateKey,
  EfiTlsConfigDataTypeCACertificate,
  EfiTlsConfigDataTypeCertRevocationList,
  EfiTlsConfigDataTypeMaximum
} EFI_TLS_CONFIG_DATA_TYPE;
```

EfiTlsConfigDataTypeHostPublicCert
Local host configuration data: public certificate data.This data should be DER-encoded binary X.509 certificate or PEM-encoded X.509 certificate.

EfiTlsConfigDataTypeHostPrivateKey
Local host configuration data: private key data.

EfiTlsConfigDataTypeCACertificate
CA certificate to verify peer. This data should be PEM-encoded RSA or PKCS#8 private key.

EfiTlsConfigDataTypeCertRevocationList
CA-supplied Certificate Revocation List data. This data should be DER-encoded CRL data.

Status Codes Returned

EFI_SUCCESS	The TLS configuration data is set successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *This* is **NULL**. • *Data* is **NULL**. • *DataSize* is 0.
EFI_UNSUPPORTED	The *DataType* is unsupported.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.

28.9.9 EFI_TLS_CONFIGURATION_PROTOCOL.GetData()

Summary

Get TLS configuration data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_TLS_CONFIGURATION_GET_DATA)(
  IN EFI_TLS_CONFIGURATION_PROTOCOL          *This,
  IN EFI_TLS_CONFIG_DATA_TYPE                DataType,
  IN OUT VOID                                *Data, OPTIONAL
  IN OUT UINTN                               *DataSize
  );
```

Parameters

This
Pointer to the *EFI_TLS_CONFIGURATION_PROTOCOL* instance.

DataType
Configuration data type. See EFI_TLS_CONFIG_DATA_TYPE

Data
Pointer to configuration data.

DataSize
Total size of configuration data. On input, it means the size of *Data* buffer. On output, it means the size of copied *Data* buffer if *EFI_SUCCESS,* and means the size of desired *Data* buffer if *EFI_BUFFER_TOO_SMALL*.

Description

The *GetData()* function gets TLS configuration.

Status Codes Returned

EFI_SUCCESS	The TLS configuration data is got successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *This* is **NULL**. • *DataSize* is **NULL** • *Data* is **NULL** if *DataSize* is not zero.
EFI_UNSUPPORTED	The *DataType* is unsupported.
EFI_NOT_FOUND	The TLS configuration data is not found.
EFI_BUFFER_TOO_SMALL	The buffer is too small to hold the data.